ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

TÀI LIỆU ÔN TẬP TRỌNG TÂM

KIẾN TRÚC MÁY TÍNH (PHIÊN BẢN RISC-V)

Mã học phần: IT3030, IT3034, IT3283

Phạm vi và Cấu trúc tài liệu (Chương 1 – Chương 4)

- **Chương 1:** Giới thiệu chung và Đánh giá hiệu năng máy tính.
- **Chương 2:** Hệ thống máy tính (Cấu trúc CPU, Bộ nhớ, Vào-Ra, Ngắt và Liên kết Bus).
- **Chương 3:** Kiến trúc tập lệnh RISC-V (Thanh ghi, Hợp ngữ, Mã máy 32-bit, Bản đồ bộ nhớ).
- **Chương 4:** Số học máy tính và Logic số (Hệ đếm, Cổng logic, Mã bù 2, Tràn số, Chuẩn IEEE 754).

Tài liệu phân tách nghiêm ngặt kiến thức theo đúng giới hạn từng chương của bài giảng chính thức, kèm quy trình giải từng bước và lời giải chi tiết cho toàn bộ ngân hàng câu hỏi Giữa kỳ và Cuối kỳ.

Hà Nội – 2026

Mục lục

1 GIỚI THIỆU CHUNG VÀ HIỆU NĂNG MÁY TÍNH **3**

1.1 Phần 1: Lý thuyết trọng tâm 3

1.1.1 Máy tính, chương trình và phân loại máy tính 3

1.1.2 Kiến trúc máy tính, phân lớp và các mức ngôn ngữ 4

1.1.3 Đánh giá hiệu năng máy tính (Trọng tâm tính toán) 6

1.2 Phần 2: Toàn bộ bài tập Chương 1 và Hướng dẫn giải 9

1.2.1 Nhóm 1: Khái niệm cơ bản và Các thành phần của máy tính 9

1.2.2 Nhóm 2: Tính chu kỳ xung nhịp (T_0), CPI và Thời gian thực thi (t_{CPU}) 9

1.2.3 Nhóm 3: Bài tập về chỉ số tốc độ thực thi lệnh MIPS 14

1.2.4 Nhóm 4: Tính CPI trung bình và Thời gian thực thi với nhiều lớp lệnh 15

1.2.5 Nhóm 5: So sánh hiệu năng và cân bằng thời gian giữa các bộ xử lý . . 20

2 HỆ THỐNG MÁY TÍNH **27**

2.1 Phần 1: Lý thuyết trọng tâm 27

2.1.1 Các thành phần cơ bản của máy tính 27

2.1.2 Ba hoạt động cơ bản của máy tính 29

2.1.3 Liên kết trong máy tính (Hệ thống Bus và Liên kết điểm-điểm) 30

2.2 Phần 2: Toàn bộ bài tập Chương 2 và Hướng dẫn giải 34

2.2.1 Nhóm 1: Cấu trúc CPU, Bộ nhớ và Hệ thống vào-ra 34

2.2.2 Nhóm 2: Chu trình lệnh, các thanh ghi PC / IR và Hoạt động Ngắt . . 35

2.2.3 Nhóm 3: Liên kết hệ thống, Các loại Bus và Tính toán không gian địa chỉ 37

3 KIẾN TRÚC TẬP LỆNH RISC-V **42**

3.1 Phần 1: Lý thuyết trọng tâm 42

3.1.1 Tổng quan về Kiến trúc tập lệnh và Tập 32 thanh ghi 42

3.1.2 Tổ chức dữ liệu trong bộ nhớ và Quy tắc Đầu nhỏ (Little-Endian) . . . 44

3.1.3 Các nhóm lệnh hợp ngữ RISC-V cốt lõi 44

3.1.4 Mã máy 32-bit của RISC-V, Lệnh nén 16-bit và 4 Phương thức định địa chỉ . . . 48
3.2 Phần 2: Toàn bộ bài tập Chương 3 và Hướng dẫn giải . . . 51
3.2.1 Nhóm 1: Thanh ghi, Khuôn dạng lệnh, Phương thức định địa chỉ và Kiểm tra hợp lệ . . . 51
3.2.2 Nhóm 2: Thực thi lệnh Số học, Logic, Dịch bit và Khởi tạo thanh ghi . 56
3.2.3 Nhóm 3: Lệnh So sánh (`slt, slti, sltu, blt, bltu`) và Rẽ nhánh . 63
3.2.4 Nhóm 4: Truy cập bộ nhớ (`lw, lh, lb, lbu, sw`) và Thứ tự Little-Endian . . . 65
3.2.5 Nhóm 5: Chuyển đổi giữa Ngôn ngữ C và Hợp ngữ RISC-V (Biểu thức, Mảng, Con trỏ, Rẽ nhánh) . . . 69
3.2.6 Nhóm 6: Phân tích Vòng lặp và Đếm tổng số lệnh thực thi . . . 81
3.2.7 Nhóm 7: Chương trình con (Thủ tục), Ngăn xếp (Stack) và Lệnh `jal / jalr` . . . 87
3.2.8 Nhóm 8: Dịch Mã máy – Hợp ngữ, Tính địa chỉ đích và Giải mã Bản đồ bộ nhớ (Memory Map) . . . 90

4 SỐ HỌC MÁY TÍNH 99
4.1 Phần 1: Lý thuyết trọng tâm . . . 99
4.1.1 Hệ đếm và Chuyển đổi giữa các hệ đếm . . . 99
4.1.2 Đại số Boolean, Cổng logic và Mạch số cơ bản . . . 100
4.1.3 Biểu diễn Số nguyên không dấu và Số nguyên có dấu (Mã bù 2) . . . 101
4.1.4 Cộng / Trừ / Nhân / Chia số nguyên và Hiện tượng Tràn số học (Overflow) . . . 103
4.1.5 Số thực dấu phẩy động chuẩn IEEE 754 . . . 105
4.2 Phần 2: Toàn bộ bài tập Chương 4 và Hướng dẫn giải . . . 107
4.2.1 Nhóm 1: Hệ đếm, Chuyển đổi cơ số và Xác định số bit biểu diễn . . . 107
4.2.2 Nhóm 2: Nhận diện Cổng logic, Mạch tổ hợp và Bộ cộng 1-bit . . . 108
4.2.3 Nhóm 3: Biểu diễn và So sánh Số nguyên có dấu (Mã bù 2) . . . 110
4.2.4 Nhóm 4: Phép cộng/trừ số nguyên và Hiện tượng Tràn số học (Overflow) 115
4.2.5 Nhóm 5: Lệnh Chia và Lấy phần dư số nguyên (`div, rem`) . . . 121
4.2.6 Nhóm 6: Số thực dấu phẩy động chuẩn IEEE 754 (32-bit) . . . 122

CHƯƠNG 1

GIỚI THIỆU CHUNG VÀ HIỆU NĂNG MÁY TÍNH

1.1 Phần 1: Lý thuyết trọng tâm

1.1.1 Máy tính, chương trình và phân loại máy tính

1. Khái niệm cơ bản

- **Máy tính (Computer):** Là thiết bị điện tử hoạt động theo chương trình, có khả năng nhập dữ liệu (input data), lưu trữ dữ liệu, xử lý dữ liệu và xuất ra thông tin (output information).
- **Chương trình (Program):** Là dãy các lệnh được lập trình sẵn và được lưu trong bộ nhớ để điều khiển máy tính thực hiện theo.

2. Bốn thành phần cơ bản của mọi máy tính

Mọi máy tính đều được cấu thành từ 4 bộ phận cơ bản:

1. **Bộ xử lý trung tâm (CPU – Central Processing Unit):** Điều khiển hoạt động của máy tính và xử lý dữ liệu.
2. **Bộ nhớ chính (Main Memory):** Chứa các lệnh và dữ liệu của chương trình hiện đang được thực hiện. (Các thiết bị lưu trữ như ổ cứng HDD, SSD thuộc bộ nhớ ngoài, dùng để lưu trữ phần mềm và dữ liệu lâu dài khi chương trình chưa chạy).
3. **Hệ thống vào-ra (Input/Output – I/O):** Trao đổi thông tin giữa máy tính với môi trường bên ngoài.
4. **Liên kết hệ thống (System Interconnection):** Kết nối và vận chuyển thông tin giữa CPU, bộ nhớ chính và hệ thống vào-ra.

3. Phân loại máy tính

Dựa theo quy mô, công dụng và giá thành, máy tính được chia thành 5 nhóm:

- **Máy tính cá nhân (PC – Personal Computers):** Gồm máy tính để bàn (Desktop) và máy tính xách tay (Laptop), phục vụ mục đích đa dụng cho cá nhân.

- **Thiết bị di động cá nhân (PMD – Personal Mobile Devices):** Điện thoại thông minh (Smartphones), máy tính bảng (Tablets).
- **Máy tính nhúng / IoT (Embedded Computers / Internet of Things):** Máy tính được đặt ẩn bên trong một thiết bị khác và được thiết kế chuyên dụng cho một chức năng cụ thể. Máy tính nhúng có kết nối Internet được gọi là thiết bị IoT.
- **Máy chủ (Servers):** Dùng trong mạng máy tính để quản lý tài nguyên và cung cấp dịch vụ cho nhiều người dùng; có hiệu năng và độ tin cậy cao, giá thành từ hàng nghìn đến hàng triệu USD.
- **Máy tính quy mô kho / Siêu máy tính (WSC – Warehouse-Scale Computers / Supercomputers):** Kết nối hàng nghìn máy chủ với nhau (Clusters). Siêu máy tính là một dạng WSC dùng cho các bài toán tính toán lớn trong khoa học và kỹ thuật, giá thành từ hàng triệu đến hàng trăm triệu USD.

1.1.2 Kiến trúc máy tính, phân lớp và các mức ngôn ngữ

1. Ba khía cạnh của Kiến trúc máy tính

- **Kiến trúc tập lệnh (ISA – Instruction Set Architecture):** Nghiên cứu máy tính theo cách nhìn của người lập trình hợp ngữ; là giao diện giữa phần mềm và phần cứng. Các kiến trúc tập lệnh thông dụng hiện nay gồm:
  - *Intel x86:* Ra đời từ năm 1978, dùng phổ biến trong Laptop, Desktop, Servers.
  - *ARM:* Ra đời từ năm 1985, dùng phổ biến trong Smartphones, Tablets.
  - *RISC-V:* Kiến trúc tập lệnh mở (miễn phí bản quyền), phát triển từ năm 2010 tại Đại học UC Berkeley.
- **Tổ chức máy tính (Computer Organization) hay Vi kiến trúc (Microarchitecture):** Nghiên cứu việc thiết kế máy tính ở mức cao để thực thi kiến trúc tập lệnh (thiết kế CPU, hệ thống nhớ, cấu trúc bus).
- **Phần cứng (Hardware):** Nghiên cứu thiết kế mạch chi tiết và công nghệ đóng gói vật lý của máy tính. Cùng một kiến trúc tập lệnh có thể có nhiều sản phẩm phần cứng và tổ chức khác nhau.

2. Bảy ý tưởng lớn trong thiết kế Kiến trúc máy tính

1. Sử dụng trừu tượng hóa để đơn giản hóa thiết kế (Use abstraction to simplify design).
2. Tối ưu hóa cho trường hợp thường gặp (Make the common case fast).
3. Tăng hiệu năng nhờ xử lý song song (Performance via parallelism).
4. Tăng hiệu năng nhờ kỹ thuật đường ống lệnh (Performance via pipelining).

5. Tăng hiệu năng nhờ dự đoán (Performance via prediction).
6. Phân cấp bộ nhớ (Hierarchy of memories).
7. Đảm bảo độ tin cậy nhờ dự phòng (Dependability via redundancy).

3. Các yếu tố quyết định hiệu năng của chương trình

- **Thuật toán:** Quyết định số phép toán cần thực hiện.
- **Ngôn ngữ lập trình, trình biên dịch và kiến trúc tập lệnh:** Quyết định số lệnh máy (Instruction Count – IC) được tạo ra để thực hiện chương trình.
- **Bộ xử lý và hệ thống bộ nhớ:** Quyết định tốc độ thực hiện các lệnh máy.
- **Hệ thống vào-ra và hệ điều hành:** Quyết định tốc độ thực hiện các thao tác vào-ra dữ liệu.

4. Phân lớp máy tính và các mức chuyển đổi mã chương trình

- **Ba lớp của hệ thống máy tính:** Từ trong ra ngoài gồm Phần cứng (Bộ xử lý, bộ nhớ, mô-đun vào-ra) → Phần mềm hệ thống (Hệ điều hành quản lý tài nguyên và Chương trình dịch) → Phần mềm ứng dụng.
- **Quá trình dịch chương trình qua 3 mức ngôn ngữ:**
  1. **Ngôn ngữ bậc cao (High-Level Language – HLL):** Như ngôn ngữ C, gần với bài toán cần giải quyết, độc lập với phần cứng.
  2. **Trình biên dịch (Compiler):** Dịch chương trình từ Ngôn ngữ bậc cao sang Hợp ngữ (Assembly Language – mô tả lệnh dưới dạng văn bản gợi nhớ, phụ thuộc vào từng kiến trúc tập lệnh cụ thể).
  3. **Trình hợp dịch (Assembler):** Dịch từ Hợp ngữ sang Ngôn ngữ máy (Machine Language / Mã máy – được mã hóa hoàn toàn bằng các bit nhị phân 0 và 1 để phần cứng trực tiếp thực thi).

5. Sự tiến hóa của công nghệ máy tính

- **Thế hệ đèn điện tử chân không (1950s):** Máy tính ENIAC (1946 – xử lý theo số thập phân) và máy tính IAS (1952 – do John von Neumann thiết kế theo nguyên lý chương trình lưu trữ trong bộ nhớ "stored program", xử lý theo số nhị phân).
- **Thế hệ Transistors (1960s) → Vi mạch SSI, MSI, LSI (1970s) → Vi mạch VLSI (1980s) → Vi mạch ULSI (1990s – nay)**.
- **Sự phát triển độ rộng bit của vi xử lý:** 4-bit (Intel 4004 năm 1971) → 8-bit (1972) → 16-bit (1978, máy tính IBM-PC ra đời năm 1981) → 32-bit (1985) → 64-bit (2001) → Đa lõi (Multicores – từ năm 2006, tích hợp nhiều nhân CPU trên cùng một chip).

1.1.3 Đánh giá hiệu năng máy tính (Trọng tâm tính toán)

Bảng quy đổi đơn vị chuẩn khi giải bài tập Hiệu năng

Khi thay số vào công thức, luôn đổi tất cả đại lượng về đơn vị chuẩn: **giây (s)** đối với thời gian, **Hertz (Hz)** đối với tần số, và **số lệnh** đối với khối lượng lệnh.

Đại lượng	**Đơn vị trong đề bài**	**Quy đổi về đơn vị chuẩn**
Tần số xung nhịp (f_0)	1 MHz (MegaHertz)	$= 10^6$ Hz (1 triệu chu kỳ/giây)
	1 GHz (GigaHertz)	$= 10^9$ Hz (1 tỷ chu kỳ/giây)
Thời gian (T_0, t_{CPU})	1 ms (mili-giây)	$= 10^{-3}$ s
	1 μs (micro-giây)	$= 10^{-6}$ s
	1 ns (nano-giây)	$= 10^{-9}$ s
	1 ps (pico-giây)	$= 10^{-12}$ s
Số lượng lệnh (IC)	1 triệu lệnh	$= 10^6$ lệnh
	1 tỷ lệnh (hoặc 1.0E9)	$= 10^9$ lệnh ($x\text{E}y = x \times 10^y$)

Quan hệ rút gọn giữa GHz và ns: Vì 1 GHz $= 10^9$ Hz và 1 ns $= 10^{-9}$ s nên:

$$T_0 \text{ (tính bằng ns)} = \frac{1}{f_0 \text{ (tính bằng GHz)}}$$

1. Chu kỳ xung nhịp (T_0) và Tần số xung nhịp (f_0)

- **Chu kỳ xung nhịp (T_0 – Clock period):** Là thời gian kéo dài của một chu kỳ xung nhịp (đơn vị: s, ms, ns, ps).
- **Tần số xung nhịp (f_0 – Clock rate / Clock speed):** Là số chu kỳ xung nhịp diễn ra trong 1 giây (đơn vị: Hz, MHz, GHz).
- **Công thức liên hệ:**

$$T_0 = \frac{1}{f_0} \quad \text{và} \quad f_0 = \frac{1}{T_0}$$

2. Tổng số chu kỳ xung nhịp (n) và Thời gian thực thi của CPU (t_{CPU})

- Gọi IC (Instruction Count) là tổng số lệnh máy được thực hiện của chương trình; CPI (Cycles Per Instruction) là số chu kỳ xung nhip trung bình để thực hiện một lệnh.
- **Tổng số chu kỳ xung nhịp (n) cần thiết để chạy xong chương trình:**

$$n = IC \times CPI$$

- **Thời gian CPU thực hiện chương trình (t_{CPU}):**

$$t_{CPU} = n \times T_0 = \frac{n}{f_0} = IC \times CPI \times T_0 = \frac{IC \times CPI}{f_0}$$

- **Các công thức suy diễn ngược (khi đề bài cho trước t_{CPU}):**

$$n = t_{CPU} \times f_0 \quad ; \quad CPI = \frac{t_{CPU} \times f_0}{IC} \quad ; \quad IC = \frac{t_{CPU} \times f_0}{CPI}$$

3. Công thức tính CPI trung bình (CPI_{TB}) khi có nhiều lớp lệnh

Nếu chương trình gồm K loại lệnh khác nhau, mỗi loại lệnh i có số chu kỳ riêng là CPI_i:

- **Trường hợp 1:** Đề bài cho số lượng lệnh cụ thể $IC_1, IC_2, \ldots, IC_K$ của từng loại:

$$IC = \sum_{i=1}^{K} IC_i \quad ; \quad n = \sum_{i=1}^{K} (CPI_i \times IC_i) \quad ; \quad CPI_{TB} = \frac{n}{IC} = \frac{\sum_{i=1}^{K} (CPI_i \times IC_i)}{\sum_{i=1}^{K} IC_i}$$

- **Trường hợp 2:** Đề bài cho tỷ lệ phần trăm $p_1\%, p_2\%, \ldots, p_K\%$ của từng loại lệnh (với $\sum p_i\% = 100\% = 1$):

$$CPI_{TB} = \sum_{i=1}^{K} (p_i\% \times CPI_i)$$

4. Định nghĩa Hiệu năng (P) và So sánh tốc độ giữa hai máy tính

- Hiệu năng (P – Performance) là nghịch đảo của thời gian thực hiện chương trình:

$$P = \frac{1}{t_{CPU}}$$

  Máy tính nào có thời gian thực hiện chương trình **nhỏ hơn** thì máy tính đó **nhanh hơn** (có hiệu năng cao hơn).

- Khi đề bài hỏi **"Máy A nhanh hơn máy B bao nhiêu lần?"**, ta lấy hiệu năng của máy A chia cho hiệu năng của máy B (tức là lấy **thời gian của máy B chia cho thời gian của máy A**):

$$k = \frac{P_A}{P_B} = \frac{t_B}{t_A}$$

- Nếu hai máy A và B có **cùng kiến trúc tập lệnh (ISA)** và chạy cùng một chương trình thì số lệnh thực thi trên hai máy là bằng nhau ($IC_A = IC_B = IC$). Khi đó IC tự triệt tiêu:

$$\frac{P_A}{P_B} = \frac{t_B}{t_A} = \frac{CPI_B \times T_B}{CPI_A \times T_A} = \frac{CPI_B / f_B}{CPI_A / f_A}$$

5. Chỉ số tốc độ thực thi MIPS và MFLOPS

- **MIPS (Millions of Instructions Per Second):** Số triệu lệnh máy thực hiện được trong 1 giây:

$$MIPS = \frac{IC}{t_{CPU} \times 10^6} = \frac{f_0}{CPI \times 10^6}$$

  - Nếu thay f_0 theo đơn vị **MHz** (10^6 Hz): $MIPS = \dfrac{f_0 \text{ (MHz)}}{CPI}$.
  - Nếu thay f_0 theo đơn vị **GHz** (10^9 Hz): $MIPS = \dfrac{f_0 \text{ (GHz)} \times 1000}{CPI}$.
  - Suy ngược ra CPI khi biết $MIPS$: $CPI = \dfrac{f_0}{MIPS \times 10^6} = \dfrac{f_0 \text{ (GHz)} \times 1000}{MIPS}$.

- **Hạn chế của chỉ số MIPS:** Chỉ đo tốc độ thực thi lệnh mà không phản ánh khả năng của từng lệnh; không dùng để so sánh hai máy khác kiến trúc tập lệnh (khác ISA); thay đổi theo từng chương trình trên cùng một máy; đôi khi một chương trình có nhiều lệnh đơn giản hơn lại cho chỉ số MIPS cao hơn nhưng tổng thời gian chạy thực tế lại lâu hơn.

- **MFLOPS (Millions of Floating-Point Operations Per Second):** Số triệu phép toán dấu phẩy động thực hiện được trong 1 giây:

$$MFLOPS = \frac{\text{Số phép toán dấu phẩy động thực thi}}{t_{CPU} \times 10^6}$$

  Các bội số lớn hơn: GFLOPS (10^9), TFLOPS (10^{12}), PFLOPS (10^{15}).

1.2 Phần 2: Toàn bộ bài tập Chương 1 và Hướng dẫn giải

1.2.1 Nhóm 1: Khái niệm cơ bản và Các thành phần của máy tính

Bài tập 1.1 (Đề Cuối kỳ 2020.2, 2021.1, 2024.2 & Ôn tập Cuối kỳ 2025.1, 2025.2) Hãy chỉ ra tên của thành phần phần cứng trong cấu trúc máy tính thực hiện chức năng chính: "Chứa toàn bộ các lệnh và dữ liệu của chương trình hiện đang được thực hiện"?

A. Bộ nhớ chính (RAM)

B. Tập thanh ghi (Register File)

C. Bộ nhớ đệm nhanh (Cache memory)

D. Bộ nhớ ngoài (Ổ đĩa SSD/HDD)

Hướng dẫn giải:

- **Kiến thức áp dụng:** Theo mô hình máy tính von Neumann (Slide Chương 1, trang 6 và trang 16), mọi chương trình khi đang được thực hiện đều phải nằm trong Bộ nhớ chính (Main Memory / RAM). Bộ nhớ ngoài (SSD/HDD) chỉ lưu trữ phần mềm và dữ liệu khi chưa chạy.
- **Kết luận:** Đáp án đúng là **A (Bộ nhớ chính)**.

1.2.2 Nhóm 2: Tính chu kỳ xung nhịp (T_0), CPI và Thời gian thực thi (t_{CPU})

Quy trình 3 bước giải bài toán tính thời gian thực thi của CPU

1. **Bước 1 (Đổi đơn vị):** Đổi số lệnh IC ra đơn vị lệnh (1 triệu = 10^6, 1 tỷ = 10^9); đổi tần số f_0 ra Hz (1 MHz = 10^6 Hz, 1 GHz = 10^9 Hz).Lưu ý: Chỉ lấy số lượng **lệnh mã máy** sau khi biên dịch, không lấy số dòng lệnh ngôn ngữ bậc cao.
2. **Bước 2 (Áp dụng công thức):** Tính thời gian theo đơn vị giây (s):

$$t_{CPU} = \frac{IC \times CPI}{f_0}$$

3. **Bước 3 (Đổi về đơn vị của đáp án):**
   - Đổi từ giây (s) sang mili-giây (ms): Nhân kết quả với 10^3 (vì 10^{-3} s = 1 ms).
   - Đổi từ giây (s) sang nano-giây (ns): Nhân kết quả với 10^9 (vì 10^{-9} s = 1 ns).

Bài tập 1.2 (Slide Chương 1 – Trang 24)
Một bộ xử lý có tần số xung nhịp $f_0 = 4$ GHz. Hãy tính chu kỳ xung nhịp T_0 của bộ xử lý đó theo đơn vị nano-giây (ns).

Hướng dẫn giải:

- **Bước 1:** Đổi tần số về đơn vị Hz: $f_0 = 4$ GHz $= 4 \times 10^9$ Hz.
- **Bước 2:** Áp dụng công thức tính chu kỳ xung nhịp:

$$T_0 = \frac{1}{f_0} = \frac{1}{4 \times 10^9} = 0.25 \times 10^{-9} \text{ s}$$

- **Bước 3:** Đổi sang đơn vị nano-giây (1 ns $= 10^{-9}$ s): $T_0 = 0.25$ ns.
- **Kết luận:** Đáp số là **0.25 ns**.

Bài tập 1.3 (Đề Cuối kỳ 2020.2 & Đề Cuối kỳ 2021.1)
Biến thể 1 (Đề 2020.2): Bộ xử lý trung tâm có tốc độ xung nhịp cố định là 2.9 GHz. Hãy cho biết chu kỳ xung nhịp tương ứng của bộ xử lý đó là bao nhiêu?
A. 7.9 ns B. 0.13 ps C. 0.13 ms D. 0.345 ns

Biến thể 2 (Ôn tập Cuối kỳ 2025.1 & 2025.2 – Câu 1): Bộ xử lý có tốc độ xung nhịp là 7.9 GHz. Cho biết chu kỳ xung nhịp tương ứng là bao nhiêu?
A. 7.9 ns B. 0.13 ps C. 0.13 ms D. 0.13 ns

Hướng dẫn giải:

- **Xét Biến thể 1 ($f_0 = 2.9$ GHz $= 2.9 \times 10^9$ Hz):**

$$T_0 = \frac{1}{f_0} = \frac{1}{2.9 \times 10^9} \text{ s} \approx 0.3448 \times 10^{-9} \text{ s} \approx 0.345 \text{ ns}$$

- **Xét Biến thể 2 ($f_0 = 7.9$ GHz $= 7.9 \times 10^9$ Hz):**

$$T_0 = \frac{1}{f_0} = \frac{1}{7.9 \times 10^9} \text{ s} \approx 0.1266 \times 10^{-9} \text{ s} \approx 0.13 \text{ ns}$$

- **Kết luận:** Biến thể 1 chọn **D (0.345 ns)**; Biến thể 2 chọn **D (0.13 ns)**.

Bài tập 1.4 (Đề Giữa kỳ 2018.2 – Mã đề 198 Câu 2 & Mã đề 199 Câu 17)
Một phần mềm viết bằng ngôn ngữ hợp ngữ với 30 lệnh, được biên dịch thành 40 lệnh mã máy. Biết rằng CPI trung bình là 2, và CPU có tốc độ xung nhịp 4 GHz. Hỏi thời gian thực hiện phần mềm là bao lâu?

A. 60 ns

B. 80 ns

C. 20 ns

D. 15 ns

Hướng dẫn giải:

- **Bước 1:** Xác định các thông số chuẩn. CPU trực tiếp thực thi các **lệnh mã máy**, do đó tổng số lệnh thực thi là $IC = 40$ lệnh (con số 30 lệnh hợp ngữ trước khi dịch là dữ kiện gây nhiễu). Ta có $CPI = 2$ và tần số $f_0 = 4$ GHz $= 4 \times 10^9$ Hz.
- **Bước 2:** Tính thời gian thực thi của CPU:

$$t_{CPU} = \frac{IC \times CPI}{f_0} = \frac{40 \times 2}{4 \times 10^9} = \frac{80}{4} \times 10^{-9} \text{ s} = 20 \times 10^{-9} \text{ s}$$

- **Bước 3:** Đổi sang đơn vị nano-giây: $t_{CPU} = 20$ ns.
- **Kết luận:** Đáp án đúng là **C (20 ns)**.

Bài tập 1.5 (Đề Giữa kỳ 2023.2 – Đề 1, Câu 1)
Máy tính có bộ xử lý với tốc độ xung nhịp là $f = 4$ GHz. Giả sử có chương trình được thực hiện trên máy tính đó với số lệnh $n = 5$ triệu lệnh và CPI trung bình bằng 2. Hãy cho biết thời gian của CPU (t_{CPU}) để hoàn thành chương trình đó (tính theo ms)?

Hướng dẫn giải:

- **Bước 1:** Đổi đơn vị:
  - Số lượng lệnh: $IC = 5$ triệu lệnh $= 5 \times 10^6$ lệnh.
  - Chỉ số $CPI = 2$.
  - Tần số xung nhịp: $f_0 = 4$ GHz $= 4 \times 10^9$ Hz.
- **Bước 2:** Tính thời gian thực thi của CPU:

$$t_{CPU} = \frac{IC \times CPI}{f_0} = \frac{5 \times 10^6 \times 2}{4 \times 10^9} = \frac{10}{4} \times 10^{-3} \text{ s} = 2.5 \times 10^{-3} \text{ s}$$

- **Bước 3:** Vì 10^{-3} s $= 1$ ms, nên $t_{CPU} = 2.5$ ms.
- **Kết luận:** Đáp số là **2.5 ms**.

Bài tập 1.6 (Đề Cuối kỳ 2024.1 – #54583d & Ôn tập Cuối kỳ 2025.2 – P1, Câu 4)
Máy tính có bộ xử lý với tần số xung nhịp 5 GHz và CPI trung bình bằng 2. Giả sử có chương trình được thực hiện trên máy tính đó với 6 triệu lệnh. Hãy cho biết thời gian của bộ xử lý để hoàn thành chương trình đó?

A. 24 ms

B. 60 ms

C. 2.4 ms

D. 240 ms

Hướng dẫn giải:

- **Bước 1:** Đổi đơn vị: $IC = 6 \times 10^6$ lệnh; $CPI = 2$; $f_0 = 5$ GHz $= 5 \times 10^9$ Hz.
- **Bước 2:** Tính thời gian thực thi của CPU:

$$t_{CPU} = \frac{IC \times CPI}{f_0} = \frac{6 \times 10^6 \times 2}{5 \times 10^9} = \frac{12}{5} \times 10^{-3} \text{ s} = 2.4 \times 10^{-3} \text{ s} = 2.4 \text{ ms}$$

- **Kết luận:** Đáp án đúng là **C (2.4 ms)**.

Bài tập 1.7 (Đề Cuối kỳ 2020.2 & 2021.1)
Một hệ thống máy tính tích hợp lõi vi xử lý hoạt động ở tần số xung nhịp cố định là 1.8 GHz và sở hữu số chu kỳ lệnh trung bình trên mỗi câu lệnh là $CPI = 2.1$. Giả sử có một chương trình ứng dụng bao gồm tổng cộng 3 triệu lệnh cần thực thi trọn vẹn trên hệ thống này. Hãy tính tổng thời gian xử lý cần thiết để CPU hoàn thành chương trình trên?

A. 3.5 ms

B. 11.3 ms

C. 35.0 ms

D. 3500.0 ms

Hướng dẫn giải:

- **Bước 1:** Đổi đơn vị: $IC = 3 \times 10^6$ lệnh; $CPI = 2.1$; $f_0 = 1.8$ GHz $= 1.8 \times 10^9$ Hz.
- **Bước 2:** Tính thời gian thực thi:

$$t_{CPU} = \frac{IC \times CPI}{f_0} = \frac{3 \times 10^6 \times 2.1}{1.8 \times 10^9} = \frac{6.3}{1.8} \times 10^{-3} \text{ s} = 3.5 \times 10^{-3} \text{ s} = 3.5 \text{ ms}$$

- **Kết luận:** Đáp án đúng là **A (3.5 ms)**.

Bài tập 1.8 (Đề Cuối kỳ 2022.2 & Cuối kỳ 2024.2 – Trang 12)
Cho một chương trình có 6 triệu lệnh được thực hiện trên máy tính có bộ xử lý với tốc độ xung nhịp 2 GHz và CPI trung bình là 1.5. Hãy cho biết thời gian của bộ xử lý để hoàn thành chương trình đó?

A. 72 ms

B. 450 ms

C. 4.5 ms

D. 45 ms

Hướng dẫn giải:

- **Bước 1:** Đổi đơn vị: $IC = 6 \times 10^6$ lệnh; $CPI = 1.5$; $f_0 = 2$ GHz $= 2 \times 10^9$ Hz.
- **Bước 2:** Tính thời gian thực thi:

$$t_{CPU} = \frac{IC \times CPI}{f_0} = \frac{6 \times 10^6 \times 1.5}{2 \times 10^9} = \frac{9}{2} \times 10^{-3} \text{ s} = 4.5 \times 10^{-3} \text{ s} = 4.5 \text{ ms}$$

- **Kết luận:** Đáp án đúng là **C (4.5 ms)**.

Bài tập 1.9 (Đề Giữa kỳ 2019.2 – Mã đề 243 Câu 11, Mã đề 244 Câu 8 & Giữa kỳ 2022.2 – Đề 0008 Câu 1)
Hãy tính chỉ số CPI trung bình của phần mềm trong 3 đề thi dưới đây:

a) **(Mã đề 243 – Câu 11):** Bộ xử lý có tần số xung nhịp 2 GHz thực hiện phần mềm gồm 400 000 lệnh máy trong 0.0003 giây.

b) **(Mã đề 244 – Câu 8):** Bộ xử lý có tần số xung nhịp 300 MHz thực hiện phần mềm gồm 1 000 000 lệnh máy trong 0.004 giây.

c) **(Giữa kỳ 2022.2 – Đề 0008, Câu 1):** Bộ xử lý có tần số xung nhịp 50 MHz thực hiện phần mềm gồm 800 000 lệnh máy trong 0.04 giây.

Hướng dẫn giải:
Từ công thức gốc $t_{CPU} = \dfrac{IC \times CPI}{f_0}$, ta rút ra công thức tính CPI:

$$CPI = \frac{t_{CPU} \times f_0}{IC}$$

- **Giải câu a:** Với $f_0 = 2$ GHz $= 2 \times 10^9$ Hz, $IC = 400\,000 = 4 \times 10^5$ lệnh, $t_{CPU} = 0.0003$ s $= 3 \times 10^{-4}$ s:

$$CPI = \frac{3 \times 10^{-4} \times 2 \times 10^9}{4 \times 10^5} = \frac{6 \times 10^5}{4 \times 10^5} = 1.5$$

- **Giải câu b:** Với $f_0 = 300$ MHz $= 300 \times 10^6$ Hz, $IC = 1\,000\,000 = 10^6$ lệnh, $t_{CPU} = 0.004$ s:

$$CPI = \frac{0.004 \times 300 \times 10^6}{10^6} = 0.004 \times 300 = 1.2$$

- **Giải câu c:** Với $f_0 = 50$ MHz $= 50 \times 10^6$ Hz, $IC = 800\,000 = 0.8 \times 10^6$ lệnh, $t_{CPU} = 0.04$ s:

$$CPI = \frac{0.04 \times 50 \times 10^6}{0.8 \times 10^6} = \frac{2}{0.8} = 2.5$$

- **Kết luận:** Câu a: $CPI = 1.5$; Câu b: $CPI = 1.2$; Câu c: $CPI = 2.5$.

1.2.3 Nhóm 3: Bài tập về chỉ số tốc độ thực thi lệnh MIPS

Bài tập 1.10 (Slide Chương 1 – Trang 35 & Trang 36)

a) Tính chỉ số $MIPS$ của bộ xử lý với $f_0 = 2$ GHz và $CPI = 4$.

b) Tính chỉ số CPI của bộ xử lý với $f_0 = 1$ GHz và tốc độ đạt 400 MIPS.

Hướng dẫn giải:

- **Giải câu a:** Đổi $f_0 = 2$ GHz $= 2 \times 10^9$ Hz. Áp dụng công thức:

$$MIPS = \frac{f_0}{CPI \times 10^6} = \frac{2 \times 10^9}{4 \times 10^6} = \frac{2000}{4} = 500 \text{ MIPS}$$

- **Giải câu b:** Đổi $f_0 = 1$ GHz $= 10^9$ Hz. Áp dụng công thức suy ra CPI:

$$CPI = \frac{f_0}{MIPS \times 10^6} = \frac{10^9}{400 \times 10^6} = \frac{1000}{400} = 2.5$$

- **Kết luận:** Câu a: **500 MIPS**; Câu b: **CPI = 2.5**.

Bài tập 1.11 (Đề Cuối kỳ 2020.2, 2022.2 – #1fd28d, 2024.1 – #3e5442, 2024.2 & Ôn tập 2025.2)

Hãy tính số triệu lệnh thực hiện được trong một giây (đơn vị MIPS) cho 3 câu hỏi trắc nghiệm sau:

a) **(Đề CK 2020.2):** Tần số xung nhịp $f_0 = 1.2$ GHz và $CPI = 1.5$.
A. 1 MIPS B. 1800 MIPS C. 800 MIPS D. 1250 MIPS

b) **(Đề CK 2022.2 & 2024.2 – #1fd28d):** Tần số xung nhịp $f_0 = 2.5$ GHz và $CPI = 2$.
A. 1250 MIPS B. 1.25 MIPS C. 5 MIPS D. 500 MIPS

c) **(Đề CK 2024.1 – #3e5442 & Ôn tập 2025.2 Câu 21):** Tần số xung nhịp $f_0 = 3$ GHz và $CPI = 1.5$.
A. 2 MIPS B. 4.5 MIPS C. 2000 MIPS D. 500 MIPS

Hướng dẫn giải:
Áp dụng công thức $MIPS = \dfrac{f_0}{CPI \times 10^6} = \dfrac{f_0 \text{ (GHz)} \times 1000}{CPI}$:

- **Câu a:** $MIPS = \dfrac{1.2 \times 1000}{1.5} = \dfrac{1200}{1.5} = 800$ MIPS $\rightarrow$ Chọn **C**.
- **Câu b:** $MIPS = \dfrac{2.5 \times 1000}{2} = \dfrac{2500}{2} = 1250$ MIPS $\rightarrow$ Chọn **A**.
- **Câu c:** $MIPS = \dfrac{3 \times 1000}{1.5} = \dfrac{3000}{1.5} = 2000$ MIPS $\rightarrow$ Chọn **C**.

Bài tập 1.12 (Đề Cuối kỳ 2021.2 – Mã đề B1 Câu 1.1 & Mã đề B5 Câu 1.2)

a) **(Mã đề B1 – Câu 1.1):** Máy tính A có bộ xử lý với tần số xung nhịp 1.5 GHz, có khả năng thực hiện 600 triệu lệnh trên một giây. Tính CPI của nó?

b) **(Mã đề B5 – Câu 1.2):** Cho máy tính A với bộ xử lý có $MIPS = 900$. Hãy tính CPI của máy tính đó nếu biết chu kỳ xung nhịp của bộ xử lý là $T_A = 300$ ps.

Hướng dẫn giải:

- **Giải câu a:** Thực hiện 600 triệu lệnh/giây tức là $MIPS = 600$. Với $f_0 = 1.5$ GHz $= 1.5 \times 10^9$ Hz:
$$CPI = \frac{f_0}{MIPS \times 10^6} = \frac{1.5 \times 10^9}{600 \times 10^6} = \frac{1500}{600} = 2.5$$
- **Giải câu b:**
  - Bước 1: Đổi chu kỳ xung nhịp sang giây: $T_A = 300$ ps $= 300 \times 10^{-12}$ s.
  - Bước 2: Tìm tần số xung nhịp $f_A = \dfrac{1}{T_A} = \dfrac{1}{300 \times 10^{-12}}$ Hz $= \dfrac{10^{12}}{300}$ Hz.
  - Bước 3: Thay vào công thức tính CPI:
$$CPI = \frac{f_A}{MIPS \times 10^6} = \frac{10^{12}}{300 \times 900 \times 10^6} = \frac{10^6}{270\,000} = \frac{100}{27} \approx 3.704$$
- **Kết luận:** Câu a: **CPI = 2.5**; Câu b: **CPI** $= \frac{100}{27} \approx$ **3.704**.

1.2.4 Nhóm 4: Tính CPI trung bình và Thời gian thực thi với nhiều lớp lệnh

Quy trình giải bài toán có nhiều nhóm lệnh (Class A

- **Dạng 1 (Cho số lượng lệnh IC_i của từng nhóm):**
  1. Tính tổng số chu kỳ: $n = (IC_1 \times CPI_1) + (IC_2 \times CPI_2) + \ldots$
  2. Tính tổng số lệnh: $IC = IC_1 + IC_2 + \ldots$
  3. Lấy $CPI_{TB} = \dfrac{n}{IC}$ và thời gian $t_{CPU} = \dfrac{n}{f_0}$.
- **Dạng 2 (Cho tỷ lệ phần trăm $p_i\%$ của từng nhóm):**
  1. Nếu đề bài ẩn tỷ lệ của một nhóm (ghi là "còn lại"), lấy 100% trừ đi các nhóm đã biết để tìm tỷ lệ phần trăm của nhóm đó.
  2. Tính $CPI_{TB} = (p_1\% \times CPI_1) + (p_2\% \times CPI_2) + \ldots$ (nhớ đổi $p\%$ ra số thập phân, ví dụ $35\% = 0.35$).

Bài tập 1.13 (Slide Chương 1 – Trang 32, Đề Giữa kỳ 2018.2 – Mã đề 204 Câu 19 & Cuối kỳ 2021.2 – Mã đề B4 Câu 1)

Tính chỉ số CPI trung bình cho các dãy lệnh được cho theo bảng số lượng lệnh dưới đây:

a) **(Đề Giữa kỳ 2018.2 – Mã đề 204, Câu 19):** Phần mềm X có 3 nhóm lệnh A, B, C với CPI tương ứng là $1, 2, 3$ và số lệnh tương ứng là $4, 1, 1$.

b) **(Slide Chương 1 – Trang 32):** Ba loại lệnh A, B, C có CPI lần lượt là $1, 2, 3$. Dãy lệnh 1 có số lệnh $(A, B, C) = (20, 10, 20)$; Dãy lệnh 2 có số lệnh $(A, B, C) = (40, 10, 10)$.

c) **(Đề Cuối kỳ 2021.2 – Mã đề B4, Câu 1):** Ba loại lệnh A, B, C có CPI lần lượt là $1, 2, 4$.

- Dãy lệnh 1 có số lệnh $(A, B, C) = (1, 2, 2)$.
- Dãy lệnh 2 có số lệnh $(A, B, C) = (5, 2, 1)$.
- Dãy lệnh 3 có số lệnh $(A, B, C) = (4, 1, 1)$.

Hướng dẫn giải:
Áp dụng công thức $CPI_{TB} = \dfrac{\sum(CPI_i \times IC_i)}{\sum IC_i}$:

- **Giải câu a:**

$$CPI_{TB} = \frac{(1 \times 4) + (2 \times 1) + (3 \times 1)}{4 + 1 + 1} = \frac{4 + 2 + 3}{6} = \frac{9}{6} = 1.5$$

- **Giải câu b:**
  - Dãy lệnh 1: $CPI_{TB1} = \dfrac{(1 \times 20) + (2 \times 10) + (3 \times 20)}{20 + 10 + 20} = \dfrac{20 + 20 + 60}{50} = \dfrac{100}{50} = 2.0.$
  - Dãy lệnh 2: $CPI_{TB2} = \dfrac{(1 \times 40) + (2 \times 10) + (3 \times 10)}{40 + 10 + 10} = \dfrac{40 + 20 + 30}{60} = \dfrac{90}{60} = 1.5.$
- **Giải câu c:**
  - Dãy lệnh 1: $CPI_{TB1} = \dfrac{(1 \times 1) + (2 \times 2) + (4 \times 2)}{1 + 2 + 2} = \dfrac{1 + 4 + 8}{5} = \dfrac{13}{5} = 2.6.$
  - Dãy lệnh 2: $CPI_{TB2} = \dfrac{(1 \times 5) + (2 \times 2) + (4 \times 1)}{5 + 2 + 1} = \dfrac{5 + 4 + 4}{8} = \dfrac{13}{8} = 1.625.$
  - Dãy lệnh 3: $CPI_{TB3} = \dfrac{(1 \times 4) + (2 \times 1) + (4 \times 1)}{4 + 1 + 1} = \dfrac{4 + 2 + 4}{6} = \dfrac{10}{6} \approx 1.67.$

Bài tập 1.14 (Đề Giữa kỳ 2025.2 – Đề 0013 Câu 75, Câu 944 & Đề 0012 Câu 110)

a) **(Đề 0013 – Câu 75):** Một phần mềm gồm: 15% lệnh tốn 3 chu kỳ, còn lại là lệnh 4 chu kỳ, 35% lệnh 5 chu kỳ và 10% lệnh 2 chu kỳ. CPI trung bình của phần mềm là bao nhiêu?

b) **(Đề 0013 – Câu 944):** Một phần mềm gồm: 10% lệnh tốn 4 chu kỳ, 40% lệnh 6 chu kỳ, 30% lệnh 8 chu kỳ, còn lại là lệnh 10 chu kỳ. CPI trung bình của phần mềm là bao nhiêu?

c) **(Đề 0012 – Câu 110):** Bộ xử lý RISC-V có tần số xung nhịp 1 GHz thực hiện phần mềm gồm 200 000 lệnh máy với $CPI = 3$, 200 000 lệnh máy với $CPI = 4$ và 300 000 lệnh máy với $CPI = 2$. Hỏi phần mềm đó chạy trong bao lâu?

Hướng dẫn giải:

- **Giải câu a:**
  - Bước 1: Tính tỷ lệ phần trăm của nhóm lệnh "còn lại" (nhóm 4 chu kỳ):

$$p_{\text{còn lại}} = 100\% - (15\% + 35\% + 10\%) = 100\% - 60\% = 40\% = 0.40$$

  - Bước 2: Tính CPI trung bình:

$$CPI_{TB} = (0.15 \times 3) + (0.40 \times 4) + (0.35 \times 5) + (0.10 \times 2) = 0.45 + 1.60 + 1.75 + 0.20 = 4.0$$

- **Giải câu b:**
  - Bước 1: Tỷ lệ phần trăm của nhóm lệnh "còn lại" (10 chu kỳ):

$$p_{\text{còn lại}} = 100\% - (10\% + 40\% + 30\%) = 20\% = 0.20$$

  - Bước 2: Tính CPI trung bình:

$$CPI_{TB} = (0.10 \times 4) + (0.40 \times 6) + (0.30 \times 8) + (0.20 \times 10) = 0.4 + 2.4 + 2.4 + 2.0 = 7.2$$

- **Giải câu c:**
  - Bước 1: Tính tổng số chu kỳ xung nhịp (n) của cả phần mềm:

$$n = (200\,000 \times 3) + (200\,000 \times 4) + (300\,000 \times 2) = 600\,000 + 800\,000 + 600\,000 = 2\,000\,000 = 2 \times 10^6 \text{ chu}$$

  - Bước 2: Với tần số $f_0 = 1$ GHz $= 10^9$ Hz, thời gian chạy phần mềm là:

$$t_{CPU} = \frac{n}{f_0} = \frac{2 \times 10^6}{10^9} = 2 \times 10^{-3} \text{ s} = 0.002 \text{ s} = 2 \text{ ms}$$

- **Kết luận:** Câu a: **4** (hoặc **4.0**); Câu b: **7.2**; Câu c: **2 ms** (hoặc **0.002 s**).

Bài tập 1.15 (Đề Giữa kỳ 2024.1 – #a6b720 & Ôn tập Cuối kỳ 2025.2 – P1, Câu 10)

Một bộ xử lý chạy một chương trình với tần số xung nhịp 4 GHz. Chương trình gồm ba loại lệnh với các đặc điểm sau:

- Loại A: chiếm 40% tổng số lệnh với CPI là 1

- Loại B: chiếm 35% tổng số lệnh với CPI là 3
- Loại C: chiếm 25% tổng số lệnh với CPI là 5

Tổng số lệnh cho chương trình là 2 tỷ lệnh. Tổng thời gian thực thi là bao nhiêu?

A. 1.1 giây

B. 1.25 giây

C. 1.35 giây

D. 1.4 giây

Hướng dẫn giải:

- **Bước 1:** Tính CPI trung bình của chương trình:

$$CPI_{TB} = (0.40 \times 1) + (0.35 \times 3) + (0.25 \times 5) = 0.40 + 1.05 + 1.25 = 2.7$$

- **Bước 2:** Đổi đơn vị: Tổng số lệnh $IC = 2$ tỷ $= 2 \times 10^9$ lệnh; tần số $f_0 = 4$ GHz $= 4 \times 10^9$ Hz.
- **Bước 3:** Tính tổng thời gian thực thi:

$$t_{CPU} = \frac{IC \times CPI_{TB}}{f_0} = \frac{2 \times 10^9 \times 2.7}{4 \times 10^9} = \frac{5.4}{4} = 1.35 \text{ giây}$$

- **Kết luận:** Đáp án đúng là **C (1.35 giây)**.

Bài tập 1.16 (Ôn tập Cuối kỳ 2025.2 – P1, Câu 55 & Câu 70)
Cho một bộ xử lý 2 GHz thực thi tập lệnh gồm năm lớp lệnh A, B, C, D, E với CPI lần lượt là $2, 3, 4, 6$ và 10. Máy tính đang thực hiện một chương trình trong đó các lớp lệnh A, B, C, D, E chiếm lần lượt $30\%, 30\%, 15\%, 15\%$ và 10% tổng số lệnh.

a) **(Câu 70):** Tìm CPI trung bình của bộ xử lý này.

b) **(Câu 55):** Nếu thời gian thực thi của chương trình này là 20 giây, hãy tìm tổng số lệnh (IC). (Viết đáp án không chứa dấu phân tách chữ số).

Hướng dẫn giải:

- **Giải câu a:** Tính CPI trung bình:

$$CPI_{TB} = (0.30 \times 2) + (0.30 \times 3) + (0.15 \times 4) + (0.15 \times 6) + (0.10 \times 10)$$

$$CPI_{TB} = 0.6 + 0.9 + 0.6 + 0.9 + 1.0 = 4.0$$

- **Giải câu b:** Ta có $t_{CPU} = 20$ s và $f_0 = 2$ GHz $= 2 \times 10^9$ Hz. Từ công thức $t_{CPU} =$

$\frac{IC \times CPI_{TB}}{f_0}$, rút ra tổng số lệnh IC:

$$IC = \frac{t_{CPU} \times f_0}{CPI_{TB}} = \frac{20 \times 2 \times 10^9}{4} = 10 \times 10^9 = 10\,000\,000\,000 \text{ lệnh}$$

- **Kết luận:** Câu a điền **4**; Câu b điền **10000000000**.

Bài tập 1.17 (Đề Giữa kỳ 2021.1 – IT3030E, Problem 3)
Cho một chương trình gồm 2000 lệnh thuộc 4 lớp lệnh: 750 lệnh số học (arithmetics), 250 lệnh ghi bộ nhớ (stores), 500 lệnh đọc bộ nhớ (loads) và 500 lệnh rẽ nhánh (branches). Giả sử số chu kỳ xung nhịp (CPI) cho các lệnh số học, load, store và branch lần lượt là $1, 5, 5$ và 2.

a) Tính thời gian thực thi của chương trình trên bộ xử lý có tốc độ 2 GHz.

b) Tìm CPI trung bình của chương trình này.

c) Nếu số lượng lệnh load trong chương trình giảm đi 50%, hãy tính độ tăng tốc (speedup) và CPI mới.

Hướng dẫn giải:

- **Giải câu a:**
  - Bước 1: Tính tổng số chu kỳ xung nhịp ban đầu ($n_{\text{cũ}}$):

    $$n_{\text{cũ}} = (750{\times}1){+}(250{\times}5){+}(500{\times}5){+}(500{\times}2) = 750{+}1250{+}2500{+}1000 = 5500 \text{ chu kỳ}$$

  - Bước 2: Thời gian thực thi trên bộ xử lý $f_0 = 2$ GHz $= 2 \times 10^9$ Hz:

    $$t_{\text{cũ}} = \frac{n_{\text{cũ}}}{f_0} = \frac{5500}{2 \times 10^9} = 2750 \times 10^{-9} \text{ s} = 2.75 \times 10^{-6} \text{ s} = 2.75\ \mu\text{s}$$

- **Giải câu b:** Chỉ số CPI trung bình ban đầu:

  $$CPI_{\text{cũ}} = \frac{n_{\text{cũ}}}{IC_{\text{cũ}}} = \frac{5500}{2000} = 2.75$$

- **Giải câu c:**
  - Khi số lệnh load giảm 50%, số lệnh load chỉ còn: $500 \times 50\% = 250$ lệnh (giảm đi 250 lệnh).
  - Tổng số lệnh mới: $IC_{\text{mới}} = 2000 - 250 = 1750$ lệnh.
  - Tổng số chu kỳ mới (giảm đi $250 \times 5 = 1250$ chu kỳ):

    $$n_{\text{mới}} = 5500 - 1250 = 4250 \text{ chu kỳ}$$

– Chỉ số CPI trung bình mới:

$$CPI_{\text{mới}} = \frac{n_{\text{mới}}}{IC_{\text{mới}}} = \frac{4250}{1750} = \frac{17}{7} \approx 2.43$$

– Độ tăng tốc (Speedup) do tần số f_0 không đổi nên bằng tỷ số chu kỳ cũ chia cho chu kỳ mới:

$$\text{Speedup} = \frac{t_{\text{cũ}}}{t_{\text{mới}}} = \frac{n_{\text{cũ}}}{n_{\text{mới}}} = \frac{5500}{4250} = \frac{22}{17} \approx 1.294 \text{ lần}$$

1.2.5 Nhóm 5: So sánh hiệu năng và cân bằng thời gian giữa các bộ xử lý

Bài tập 1.18 (Slide Chương 1 – Trang 29, Đề Giữa kỳ 2018.2 – Mã đề 198 Câu 12 & Đề Cuối kỳ 2021.2 – Mã đề B1 Câu 1.2)
Hai máy tính A và B có cùng kiến trúc tập lệnh (ISA). Hãy xác định máy nào nhanh hơn và nhanh hơn bao nhiêu lần trong 3 trường hợp thông số dưới đây:

a) **(Slide Chương 1 – Trang 29):** Máy A có $T_A = 250$ ps, $CPI_A = 2.0$; Máy B có $T_B = 500$ ps, $CPI_B = 1.2$.

b) **(Đề Cuối kỳ 2021.2 – Mã đề B1, Câu 1.2):** Máy A có $T_A = 250$ ps, $CPI_A = 2.0$; Máy B có $T_B = 750$ ps, $CPI_B = 1.2$.

c) **(Đề Giữa kỳ 2018.2 – Mã đề 198 Câu 12 & Mã đề 199 Câu 4):** Máy A có $T_A = 300$ ps, $CPI_A = 2.0$; Máy B có $T_B = 500$ ps, $CPI_B = 1.1$.

Hướng dẫn giải:
Vì hai máy A và B có cùng kiến trúc tập lệnh (ISA), nên khi chạy cùng một chương trình thì số lệnh thực thi là như nhau: $IC_A = IC_B = IC$. Thời gian thực thi trên mỗi máy là $t = IC \times CPI \times T_0$:

- **Giải câu a:**
  – $t_A = IC \times 2.0 \times 250$ ps $= 500 \times IC$ (ps).
  – $t_B = IC \times 1.2 \times 500$ ps $= 600 \times IC$ (ps).
  – Vì $t_A < t_B$ ($500 < 600$) nên **máy A nhanh hơn máy B**, và nhanh hơn số lần là: $\frac{t_B}{t_A} = \frac{600}{500} = \mathbf{1.2}$ lần.
- **Giải câu b:**
  – $t_A = IC \times 2.0 \times 250$ ps $= 500 \times IC$ (ps).
  – $t_B = IC \times 1.2 \times 750$ ps $= 900 \times IC$ (ps).

- Vì $t_A < t_B$ (500 < 900) nên **máy A nhanh hơn máy B**, và nhanh hơn số lần là: $\frac{t_B}{t_A} = \frac{900}{500} = \mathbf{1.8}$ lần.

- **Giải câu c:**
  - $t_A = IC \times 2.0 \times 300$ ps $= 600 \times IC$ (ps).
  - $t_B = IC \times 1.1 \times 500$ ps $= 550 \times IC$ (ps).
  - Vì $t_B < t_A$ (550 < 600) nên **máy B nhanh hơn máy A** (chọn đáp án "B nhanh hơn A").

Bài tập 1.19 (Đề Cuối kỳ 2023.1, 2023.2 – #2262f, Giữa kỳ 2024.1 – #8cf496 & Ôn tập 2025.2 – Câu 12)

a) **(Đề CK 2023.1 – #2262f & Ôn tập 2025.2 Câu 12):** Xét hai bộ xử lý cùng thực hiện một công việc. P1 có tốc độ xung nhịp 4 GHz, CPI trung bình là 0.9 và yêu cầu thực thi 5.0E9 lệnh. P2 có tốc độ xung nhịp 3 GHz, CPI trung bình là 0.75 và yêu cầu thực thi 1.0E9 lệnh. Hỏi P2 nhanh hơn P1 bao nhiêu lần khi thực hiện công việc này?

b) **(Đề GK 2024.1 – #8cf496):** Xét hai bộ xử lý cùng thực hiện một công việc. P1 có tốc độ xung nhịp 1.5 GHz, CPI trung bình là 1.8 và yêu cầu thực thi 3.0E9 lệnh. P2 có tốc độ xung nhịp 2 GHz, CPI trung bình là 1.2 và yêu cầu thực thi 4.0E9 lệnh. Hỏi P2 nhanh hơn P1 bao nhiêu lần?

Hướng dẫn giải:

- **Giải câu a:**
  - Bước 1: Tính thời gian thực thi của P1 (t_1):
$$t_1 = \frac{IC_1 \times CPI_1}{f_1} = \frac{5.0 \times 10^9 \times 0.9}{4 \times 10^9} = \frac{4.5}{4} = 1.125 \text{ giây}$$
  - Bước 2: Tính thời gian thực thi của P2 (t_2):
$$t_2 = \frac{IC_2 \times CPI_2}{f_2} = \frac{1.0 \times 10^9 \times 0.75}{3 \times 10^9} = \frac{0.75}{3} = 0.25 \text{ giây}$$
  - Bước 3: Đề hỏi "P2 nhanh hơn P1 bao nhiêu lần", ta lấy thời gian của P1 chia cho thời gian của P2:
$$\frac{P_2}{P_1} = \frac{t_1}{t_2} = \frac{1.125}{0.25} = 4.5 \text{ lần}$$

- **Giải câu b:**
  - Bước 1: Tính thời gian thực thi của P1 (t_1):
$$t_1 = \frac{IC_1 \times CPI_1}{f_1} = \frac{3.0 \times 10^9 \times 1.8}{1.5 \times 10^9} = \frac{5.4}{1.5} = 3.6 \text{ giây}$$

– Bước 2: Tính thời gian thực thi của P2 (t_2):

$$t_2 = \frac{IC_2 \times CPI_2}{f_2} = \frac{4.0 \times 10^9 \times 1.2}{2 \times 10^9} = \frac{4.8}{2} = 2.4 \text{ giây}$$

– Bước 3: Tỷ số tốc độ của P2 so với P1:

$$\frac{P_2}{P_1} = \frac{t_1}{t_2} = \frac{3.6}{2.4} = 1.5 \text{ lần}$$

- **Kết luận:** Câu a điền **4.5**; Câu b điền **1.5**.

Bài tập 1.20 (Đề Giữa kỳ 2021.1 – IT3030E, Problem 2)
Giả sử có hai bộ xử lý P1 và P2 cùng thực hiện một kiến trúc tập lệnh (ISA) gồm 4 lớp lệnh A, B, C, D. Tốc độ xung nhịp và CPI của từng lớp lệnh được cho trong bảng sau:

Bộ xử lý	Clock Rate	Class A	Class B	Class C	Class D
P1	2.5 GHz	2	1.5	2	1
P2	3 GHz	1	2	1	1

Cho một chương trình có 1 000 000 lệnh được chia thành các lớp như sau: 10% lớp A, 20% lớp B, 50% lớp C và 20% lớp D.

a) Tính CPI trung bình (global CPI) cho mỗi bộ xử lý.

b) Tìm tổng số chu kỳ xung nhịp yêu cầu trong cả hai trường hợp.

c) Bộ xử lý nào nhanh hơn?

Hướng dẫn giải:

- **Giải câu a (Tính CPI trung bình):**
  – Đối với P1:

$$CPI_1 = (0.10 \times 2) + (0.20 \times 1.5) + (0.50 \times 2) + (0.20 \times 1) = 0.2 + 0.3 + 1.0 + 0.2 = 1.7$$

  – Đối với P2:

$$CPI_2 = (0.10 \times 1) + (0.20 \times 2) + (0.50 \times 1) + (0.20 \times 1) = 0.1 + 0.4 + 0.5 + 0.2 = 1.2$$

- **Giải câu b (Tính tổng số chu kỳ $n = IC \times CPI$ với $IC = 10^6$ lệnh):**
  – Số chu kỳ của P1: $n_1 = 10^6 \times 1.7 = 1.7 \times 10^6$ chu kỳ.
  – Số chu kỳ của P2: $n_2 = 10^6 \times 1.2 = 1.2 \times 10^6$ chu kỳ.

- **Giải câu c (So sánh thời gian thực thi):**

- Thời gian thực thi trên P1: $t_1 = \dfrac{n_1}{f_1} = \dfrac{1.7 \times 10^6}{2.5 \times 10^9} = 0.68 \times 10^{-3}$ s $= 0.68$ ms.
- Thời gian thực thi trên P2: $t_2 = \dfrac{n_2}{f_2} = \dfrac{1.2 \times 10^6}{3 \times 10^9} = 0.40 \times 10^{-3}$ s $= 0.40$ ms.
- Vì $t_2 < t_1$ (0.40 ms $<$ 0.68 ms) nên **bộ xử lý P2 nhanh hơn P1** (nhanh hơn $\dfrac{0.68}{0.40} = 1.7$ lần).

Bài tập 1.21 (Slide Chương 1 – Trang 26)
Hai máy tính A và B cùng chạy một chương trình:

- Máy tính A có tần số xung nhịp $f_A = 2$ GHz, thời gian CPU thực hiện chương trình là $t_A = 10$ s.
- Máy tính B cần đạt thời gian thực hiện chương trình là $t_B = 6$ s. Biết rằng số chu kỳ xung nhịp khi chạy chương trình trên máy B (n_B) bằng 1.2 lần số chu kỳ xung nhịp khi chạy chương trình trên máy A (n_A).

Hãy xác định tần số xung nhịp cần thiết cho máy B (f_B)?

Hướng dẫn giải:

- **Bước 1:** Tính tổng số chu kỳ xung nhịp khi chạy chương trình trên máy A:

$$n_A = t_A \times f_A = 10 \text{ s} \times (2 \times 10^9 \text{ Hz}) = 20 \times 10^9 \text{ chu kỳ}$$

- **Bước 2:** Tính tổng số chu kỳ xung nhịp khi chạy chương trình trên máy B:

$$n_B = 1.2 \times n_A = 1.2 \times 20 \times 10^9 = 24 \times 10^9 \text{ chu kỳ}$$

- **Bước 3:** Tính tần số xung nhịp f_B cần thiết để máy B hoàn thành trong $t_B = 6$ s:

$$f_B = \frac{n_B}{t_B} = \frac{24 \times 10^9}{6} = 4 \times 10^9 \text{ Hz} = 4 \text{ GHz}$$

- **Kết luận:** Đáp số là $\mathbf{f_B = 4}$ GHz.

Bài tập 1.22 (Tổng hợp 4 mã đề tự luận Đề Cuối kỳ 2021.2 – Mã đề B2, B3, B6, B7 Câu 1)
Giả sử có một chương trình chạy trên hai máy tính A và B. Hãy xác định khi dịch trên máy B thì chương trình đó được dịch thành bao nhiêu lệnh (IC_B) để cả hai máy có cùng thời gian thực hiện của CPU ($t_A = t_B$) trong 4 mã đề sau:

a) **(Mã đề B2):** Máy A có $CPI_A = 1.3, f_A = 600$ MHz, $IC_A = 100\,000$ lệnh. Máy B có $CPI_B = 2.5, f_B = 750$ MHz.

b) **(Mã đề B3):** Máy A có $CPI_A = 1.2, f_A = 500$ MHz, $IC_A = 90\,000$ lệnh. Máy B có

$CPI_B = 2.4, f_B = 600$ MHz.

c) **(Mã đề B6):** Máy A có $CPI_A = 1.4, f_A = 750$ MHz, $IC_A = 80\,000$ lệnh. Máy B có $CPI_B = 2.8, f_B = 1.5$ GHz.

d) **(Mã đề B7):** Máy A có $CPI_A = 2.4, f_A = 800$ MHz, $IC_A = 70\,000$ lệnh. Máy B có $CPI_B = 3.6, f_B = 1$ GHz.

Hướng dẫn giải:
Phương trình cân bằng thời gian thực thi giữa hai máy ($t_A = t_B$):

$$\frac{IC_A \times CPI_A}{f_A} = \frac{IC_B \times CPI_B}{f_B} \iff IC_B = \frac{IC_A \times CPI_A \times f_B}{f_A \times CPI_B}$$

(Lưu ý: Phải đưa f_A và f_B về cùng một đơn vị là MHz trước khi thay số; với 1 GHz = 1000 MHz và 1.5 GHz = 1500 MHz):

- **Giải câu a (Đề B2):**

$$IC_B = \frac{100\,000 \times 1.3 \times 750}{600 \times 2.5} = \frac{97\,500\,000}{1500} = 65\,000 \text{ lệnh}$$

- **Giải câu b (Đề B3):**

$$IC_B = \frac{90\,000 \times 1.2 \times 600}{500 \times 2.4} = \frac{64\,800\,000}{1200} = 54\,000 \text{ lệnh}$$

- **Giải câu c (Đề B6 – đổi $f_B = 1.5$ GHz $= 1500$ MHz):**

$$IC_B = \frac{80\,000 \times 1.4 \times 1500}{750 \times 2.8} = \frac{168\,000\,000}{2100} = 80\,000 \text{ lệnh}$$

- **Giải câu d (Đề B7 – đổi $f_B = 1$ GHz $= 1000$ MHz):**

$$IC_B = \frac{70\,000 \times 2.4 \times 1000}{800 \times 3.6} = \frac{168\,000\,000}{2880} = \frac{175\,000}{3} \approx 58\,333 \text{ lệnh}$$

Bài tập 1.23 (Đề Cuối kỳ 2021.2 – Mã đề B8, Câu 1)
Giả sử có 3 bộ xử lý P1, P2, P3 có cùng kiến trúc tập lệnh (ISA). Tần số xung nhịp và CPI của mỗi bộ xử lý được cho trong bảng sau:

Bộ xử lý	**Tần số xung nhịp (f_0)**	**CPI**
P1	2 GHz	1.5
P2	1.5 GHz	1.0
P3	3 GHz	2.5

1.1. Bộ xử lý nào có hiệu suất cao nhất?

1.2. Mỗi bộ xử lý thực thi một chương trình trong 10 s. Tính số chu kỳ và số lệnh của mỗi bộ xử lý.

1.3. Người ta muốn giảm thời gian thực thi đi 30%, nhưng điều này lại làm tăng CPI lên 20%. Tính tần số xung nhịp mới của mỗi bộ xử lý lúc này.

Hướng dẫn giải:

- **Giải ý 1.1 (Tìm bộ xử lý có hiệu suất cao nhất):**
  Vì cả 3 bộ xử lý có cùng kiến trúc tập lệnh (ISA), bộ xử lý nào thực hiện được nhiều lệnh nhất trong 1 giây (tức tỷ số $\frac{f_0}{CPI}$ lớn nhất) sẽ có hiệu suất cao nhất:
  - P1: $\frac{f_1}{CPI_1} = \frac{2\text{ GHz}}{1.5} \approx 1.33 \times 10^9$ lệnh/giây (1333 MIPS).
  - P2: $\frac{f_2}{CPI_2} = \frac{1.5\text{ GHz}}{1.0} = 1.50 \times 10^9$ lệnh/giây (1500 MIPS).
  - P3: $\frac{f_3}{CPI_3} = \frac{3\text{ GHz}}{2.5} = 1.20 \times 10^9$ lệnh/giây (1200 MIPS).

  Vậy **bộ xử lý P2 có hiệu suất cao nhất**.
- **Giải ý 1.2 (Tính số chu kỳ $n = t \times f_0$ và số lệnh $IC = \frac{n}{CPI}$ trong $t = 10$ s):**
  - Bộ xử lý P1: $n_1 = 10 \times 2 \times 10^9 = 20 \times 10^9$ chu kỳ; $IC_1 = \frac{20 \times 10^9}{1.5} \approx 1.33 \times 10^{10}$ lệnh.
  - Bộ xử lý P2: $n_2 = 10 \times 1.5 \times 10^9 = 15 \times 10^9$ chu kỳ; $IC_2 = \frac{15 \times 10^9}{1.0} = 1.50 \times 10^{10}$ lệnh.
  - Bộ xử lý P3: $n_3 = 10 \times 3 \times 10^9 = 30 \times 10^9$ chu kỳ; $IC_3 = \frac{30 \times 10^9}{2.5} = 1.20 \times 10^{10}$ lệnh.
- **Giải ý 1.3 (Tính tần số xung nhịp mới $f_{\text{mới}}$):**
  - Giảm thời gian đi 30% nghĩa là thời gian mới bằng 70% thời gian cũ: $t_{\text{mới}} = (1 - 0.30) \times t_{\text{cũ}} = 0.7 \times 10\text{ s} = 7\text{ s}$.
  - Tăng CPI lên 20% nghĩa là CPI mới bằng 120% CPI cũ: $CPI_{\text{mới}} = (1 + 0.20) \times CPI_{\text{cũ}} = 1.2 \times CPI_{\text{cũ}}$.
  - Do số lệnh IC không đổi, tần số xung nhịp mới liên hệ với tần số cũ theo công thức:
    $$f_{\text{mới}} = \frac{IC \times CPI_{\text{mới}}}{t_{\text{mới}}} = \frac{IC \times 1.2 \times CPI_{\text{cũ}}}{0.7 \times t_{\text{cũ}}} = \frac{1.2}{0.7} \times f_{\text{cũ}} = \frac{12}{7} \times f_{\text{cũ}}$$
  - Thay số cho từng bộ xử lý:
    * $f_{1\text{ mới}} = \frac{12}{7} \times 2\text{ GHz} = \frac{24}{7} \approx \mathbf{3.43}$ GHz.
    * $f_{2\text{ mới}} = \frac{12}{7} \times 1.5\text{ GHz} = \frac{18}{7} \approx \mathbf{2.57}$ GHz.

* $f_{3\text{ mới}} = \dfrac{12}{7} \times 3\text{ GHz} = \dfrac{36}{7} \approx \mathbf{5.14}\text{ GHz}$.

CHƯƠNG 2

HỆ THỐNG MÁY TÍNH

2.1 Phần 1: Lý thuyết trọng tâm

2.1.1 Các thành phần cơ bản của máy tính

Mọi hệ thống máy tính đều hoạt động dựa trên sự phối hợp của các mô-đun phần cứng chính: Bộ xử lý trung tâm (CPU), Bộ nhớ máy tính và Hệ thống vào-ra, được kết nối với nhau thông qua Liên kết hệ thống (Bus).

1. Bộ xử lý trung tâm (CPU – Central Processing Unit)

- **Chức năng:** Điều khiển hoạt động của toàn bộ máy tính và trực tiếp xử lý dữ liệu.
- **Nguyên tắc hoạt động:** CPU hoạt động theo chương trình nằm trong bộ nhớ chính và là thành phần có tốc độ xử lý nhanh nhất trong hệ thống.
- **Ba thành phần cơ bản bên trong CPU:**
  1. **Đơn vị điều khiển (CU – Control Unit):** Giải mã lệnh và phát các tín hiệu điều khiển hoạt động của máy tính theo chương trình đã định sẵn.
  2. **Đơn vị số học và logic (ALU – Arithmetic and Logic Unit):** Thực hiện các phép toán số học (cộng, trừ, nhân, chia, so sánh) và các phép toán logic (AND, OR, XOR, NOT, dịch bit).
  3. **Tập thanh ghi (RF – Register File):** Gồm các thanh ghi nằm ngay bên trong CPU, có tốc độ truy cập nhanh nhất, chịu trách nhiệm lưu giữ các thông tin tạm thời phục vụ cho hoạt động tức thời của CPU.

2. Bộ nhớ máy tính (Memory System)

- **Chức năng và thao tác:** Lưu trữ chương trình và dữ liệu dưới dạng nhị phân. Có hai thao tác cơ bản với bộ nhớ là thao tác **Ghi (Write)** và thao tác **Đọc (Read)**.
- **Phân biệt Bộ nhớ trong và Bộ nhớ ngoài:**
  - *Bộ nhớ trong:* Gồm **Bộ nhớ chính** (RAM, ROM) và **Bộ nhớ đệm** (Cache).
  - *Bộ nhớ ngoài (Thiết bị lưu trữ – Storage):* Gồm ổ đĩa cứng từ tính (HDD), ổ thể rắn bán dẫn (SSD), ổ nhớ Flash, thẻ nhớ, đĩa quang (CD, DVD).

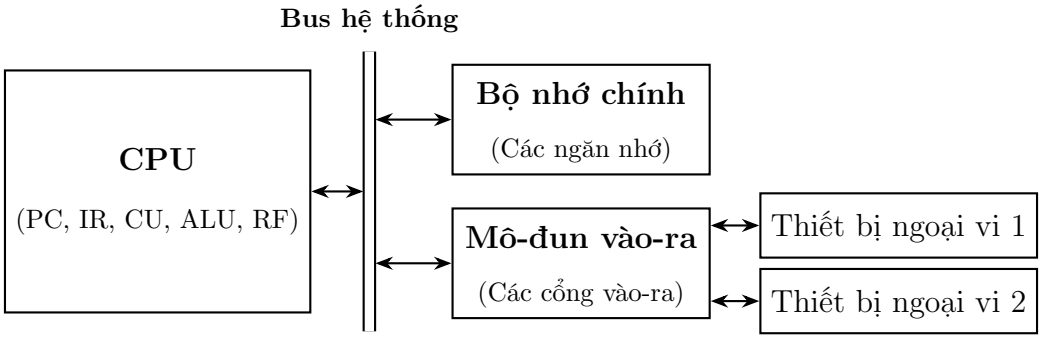
- **Đặc điểm của 3 thành phần bộ nhớ chính:**

  1. **Bộ nhớ chính (Main Memory):** Tồn tại trên mọi máy tính, chứa các lệnh và dữ liệu của chương trình đang được thực hiện. Bộ nhớ chính sử dụng bộ nhớ bán dẫn, được tổ chức thành các **ngăn nhớ** được đánh địa chỉ (thường đánh địa chỉ theo từng byte).
     - *Tính chất quan trọng của ngăn nhớ:* Mỗi ngăn nhớ trong máy tính có **nội dung** (giá trị dữ liệu) có thể thay đổi, nhưng **địa chỉ vật lý** của ngăn nhớ đó luôn cố định. CPU muốn đọc hoặc ghi một ngăn nhớ thì bắt buộc phải biết địa chỉ của ngăn nhớ đó.
  2. **Bộ nhớ đệm (Cache Memory):** Là bộ nhớ bán dẫn có tốc độ nhanh được đặt đệm giữa CPU và bộ nhớ chính nhằm **tăng tốc độ CPU truy cập bộ nhớ**. Cache có dung lượng nhỏ hơn bộ nhớ chính, thường được chia thành các mức (L1, L2, L3) và tích hợp trên cùng chip bộ xử lý.
  3. **Thiết bị lưu trữ (Storage / Bộ nhớ ngoài):** Dùng để lưu giữ lâu dài tài nguyên phần mềm và dữ liệu; có dung lượng rất lớn nhưng tốc độ chậm. Về mặt liên kết hệ thống, thiết bị lưu trữ được kết nối với máy tính dưới dạng các **thiết bị vào-ra**.

3. Hệ thống vào-ra (Input/Output System)

- **Chức năng và thao tác:** Trao đổi thông tin giữa máy tính với thế giới bên ngoài thông qua hai thao tác cơ bản là **Vào dữ liệu (Input)** và **Ra dữ liệu (Output)**.
- **Hai thành phần chính của hệ thống vào-ra:**

  1. **Các thiết bị vào-ra (I/O Devices / Thiết bị ngoại vi – Peripherals):** Làm nhiệm vụ chuyển đổi dữ liệu giữa bên trong và bên ngoài máy tính. Gồm 4 nhóm: Thiết bị vào (bàn phím, chuột), Thiết bị ra (màn hình, máy in), Thiết bị lưu trữ (HDD, SSD), và Thiết bị truyền thông (card mạng, modem). *Lưu ý:* Các thiết bị ngoại vi (như bàn phím, màn hình) **không nối trực tiếp** với CPU hay Bus hệ thống mà phải kết nối thông qua Mô-đun vào-ra.
  2. **Các mô-đun vào-ra (I/O Modules):** Làm nhiệm vụ nối ghép các thiết bị vào-ra với máy tính (qua Bus hệ thống). Mỗi mô-đun vào-ra có một hoặc một vài **cổng vào-ra (I/O Ports)**, mỗi cổng được đánh một địa chỉ xác định. CPU muốn trao đổi dữ liệu với thiết bị ngoại vi nào thì phải thông qua địa chỉ của cổng vào-ra tương ứng.



2.1.2 Ba hoạt động cơ bản của máy tính

1. Hoạt động thực hiện chương trình (Chu trình lệnh)

- Máy tính thực hiện chương trình bằng cách lặp đi lặp lại **chu trình lệnh** gồm hai bước chính: **Tìm nạp lệnh (Fetch instruction)** và **Thực hiện lệnh (Execute instruction)**. Hoạt động này chỉ dừng lại khi gặp lệnh thực hiện bị lỗi, gặp lệnh dừng hoặc tắt máy.
- **Quá trình Tìm nạp lệnh (Fetch):**
  - **Bộ đếm chương trình PC (Program Counter):** Là thanh ghi của CPU luôn chứa **địa chỉ của lệnh tiếp theo** sẽ được nạp vào để thực hiện (trên kiến trúc Intel x86, thanh ghi này được gọi là **IP – Instruction Pointer**).
  - Bắt đầu mỗi chu trình lệnh, CPU phát địa chỉ từ thanh ghi **PC** ra bộ nhớ chính để tìm ngăn nhớ chứa lệnh.
  - Lệnh đọc từ bộ nhớ chính được nạp vào **Thanh ghi lệnh IR (Instruction Register)** của CPU.
  - Sau khi lệnh được nạp vào IR, nếu chương trình thực hiện tuần tự, CPU sẽ **tự động tăng nội dung thanh ghi PC** để trỏ sang lệnh kế tiếp (với tập lệnh 32-bit cố định của RISC-V, mỗi lệnh chiếm 4 byte nên $PC \leftarrow PC + 4$).
- **Quá trình Giải mã và Thực hiện lệnh (Decode & Execute):**
  - Ngay sau khi lấy lệnh vào thanh ghi **IR**, CPU tiến hành **giải mã lệnh** đang nằm trong thanh ghi **IR** và phát các tín hiệu điều khiển thực hiện thao tác mà lệnh yêu cầu.
  - **Ba kiểu thao tác cơ bản của lệnh:**
    1. Trao đổi dữ liệu giữa CPU với bộ nhớ chính hoặc với mô-đun vào-ra.
    2. Thực hiện các phép toán số học hoặc phép toán logic với các dữ liệu (trên ALU).
    3. Chuyển điều khiển trong chương trình (rẽ nhánh hoặc nhảy đến vị trí khác, bản chất là thay đổi giá trị của thanh ghi PC bằng địa chỉ đích).

2. Hoạt động Ngắt (Interrupt)

- **Định nghĩa:** Ngắt là cơ chế cho phép CPU **tạm dừng chương trình đang thực hiện** để chuyển sang thực hiện một chương trình con có sẵn trong bộ nhớ gọi là **chương trình con xử lý ngắt (Interrupt Handler)**.
- **Các loại nguyên nhân gây ngắt:**
  - Ngắt do lỗi khi thực hiện lệnh (ví dụ: tràn số học, chia cho 0, mã lệnh sai).
  - Ngắt do lỗi phần cứng.
  - Ngắt do bộ định thời (Timer).
  - Ngắt do bộ điều khiển vào-ra gửi tín hiệu ngắt đến CPU để yêu cầu trao đổi dữ liệu.
- **Quy trình CPU xử lý ngắt từ mô-đun vào-ra:**
  1. Sau khi hoàn thành mỗi một lệnh máy, CPU kiểm tra tín hiệu ngắt. Nếu không có ngắt, CPU tìm nạp lệnh tiếp theo.
  2. Nếu có tín hiệu ngắt, CPU **tạm dừng (suspend)** chương trình đang thực hiện.
  3. **Cất ngữ cảnh** (lưu địa chỉ trở về trên PC và các thông tin liên quan đến chương trình bị ngắt vào ngăn xếp).
  4. Thiết lập bộ đếm chương trình **PC** trỏ đến địa chỉ bắt đầu của chương trình con xử lý ngắt tương ứng.
  5. Thực hiện chương trình con xử lý ngắt, sau đó **khôi phục ngữ cảnh** và trở về tiếp tục thực hiện chương trình đang bị tạm dừng.
- **Xử lý khi có nhiều yêu cầu ngắt:** Gồm hai phương pháp là **Xử lý ngắt tuần tự** (thực hiện lần lượt hết chương trình xử lý ngắt X rồi mới thực hiện chương trình xử lý ngắt Y) và **Xử lý ngắt ưu tiên** (chương trình xử lý ngắt Y có mức ưu tiên cao hơn được phép ngắt tạm thời chương trình xử lý ngắt X đang chạy).

3. Hoạt động Vào-Ra (Input/Output Operations)

- Là hoạt động trao đổi dữ liệu giữa mô-đun vào-ra với bên trong máy tính.
- **Hai kiểu hoạt động vào-ra cơ bản:**
  1. CPU trực tiếp trao đổi dữ liệu với mô-đun vào-ra thông qua các lệnh vào-ra trong chương trình (Vào-ra bằng chương trình hoặc Vào-ra điều khiển bằng ngắt).
  2. CPU trao quyền điều khiển hệ thống bus cho phép mô-đun vào-ra trao đổi dữ liệu trực tiếp với bộ nhớ chính mà không đi qua CPU (**DMA – Direct Memory Access**).

2.1.3 Liên kết trong máy tính (Hệ thống Bus và Liên kết điểm-điểm)

1. Luồng thông tin kết nối tới từng mô-đun

- **Mô-đun nhớ:** Nhận địa chỉ để xác định ngăn nhớ; nhận dữ liệu khi ghi; đưa dữ liệu hoặc lệnh ra khi đọc (bộ nhớ không phân biệt lệnh và dữ liệu); nhận tín hiệu điều khiển Đọc (Read) và Ghi (Write).
- **Mô-đun vào-ra:** Nhận địa chỉ để xác định cổng vào-ra; chuyển dữ liệu hai chiều giữa bên trong và bên ngoài; nhận tín hiệu điều khiển Đọc/Ghi từ CPU; phát tín hiệu điều khiển đến thiết bị ngoại vi và phát tín hiệu yêu cầu ngắt đến CPU.
- **CPU:** Phát địa chỉ đến mô-đun nhớ hoặc mô-đun vào-ra; đọc lệnh từ bộ nhớ; đọc/ghi dữ liệu với bộ nhớ hoặc mô-đun vào-ra; phát tín hiệu điều khiển đọc/ghi và nhận tín hiệu yêu cầu ngắt.

2. Liên kết Bus trong máy tính

Bus là tập hợp các đường dây kết nối dùng để vận chuyển thông tin giữa các mô-đun của máy tính với nhau. **Độ rộng bus** là số đường dây của bus có thể truyền các bit thông tin đồng thời (khái niệm độ rộng bus **chỉ dùng cho Bus địa chỉ và Bus dữ liệu**, không cố định cho Bus điều khiển).

Bảng tổng hợp 3 loại Bus chức năng trong Bus hệ thống

Loại Bus	**Hướng truyền**	**Chức năng và Công thức tính toán**
Bus địa chỉ (Address Bus)	**Một chiều** (Từ CPU/DMA đến Bộ nhớ và I/O)	Vận chuyển địa chỉ để xác định vị trí ngăn nhớ hay cổng vào-ra. Nếu có độ rộng N bit ($A_{N-1} \dots A_0$), số địa chỉ tối đa (không gian địa chỉ) là: 2^N **địa chỉ** (từ $00\dots0_{(2)}$ đến $11\dots1_{(2)}$).
Bus dữ liệu (Data Bus)	**Hai chiều** (Giữa CPU, Bộ nhớ và I/O)	Vận chuyển **lệnh** từ bộ nhớ đến CPU và vận chuyển **dữ liệu** giữa các thành phần. Nếu có độ rộng M bit ($D_{M-1} \dots D_0$, thường là $8, 16, 32, 64$ bit), mỗi thời điểm truyền đồng thời được M bit ($= M/8$ byte).
Bus điều khiển (Control Bus)	**Tùy từng đường tín hiệu**	Vận chuyển các tín hiệu điều khiển đọc/ghi, điều khiển ngắt và điều khiển xin nhượng bus. Độ rộng phụ thuộc vào thiết kế cụ thể của từng bộ xử lý (không xác định cố định theo bus dữ liệu).

3. Các tín hiệu điều khiển điển hình trên Bus điều khiển

- **Nhóm tín hiệu điều khiển đọc/ghi (do CPU phát ra):**
  - `MEMR` (Memory Read): Điều khiển đọc dữ liệu/lệnh từ ngăn nhớ có địa chỉ xác định

đưa lên bus dữ liệu chuyển vào CPU.

- `MEMW` (Memory Write): Điều khiển ghi dữ liệu trên bus dữ liệu (từ CPU) vào ngăn nhớ có địa chỉ xác định.
- `IOR` (I/O Read): Điều khiển đọc dữ liệu từ cổng vào-ra có địa chỉ xác định đưa lên bus dữ liệu chuyển vào CPU.
- `IOW` (I/O Write): Điều khiển ghi dữ liệu trên bus dữ liệu (từ CPU) ra cổng vào-ra có địa chỉ xác định.

- **Nhóm tín hiệu điều khiển ngắt:**
  - `INTR` (Interrupt Request): Từ bộ điều khiển vào-ra gửi đến yêu cầu ngắt CPU để trao đổi vào-ra (tín hiệu này có thể bị che/cấm).
  - `INTA` (Interrupt Acknowledge): Từ CPU phát ra báo cho bộ điều khiển vào-ra biết CPU chấp nhận ngắt.
  - `NMI` (Non-Maskable Interrupt): Tín hiệu ngắt không thể che gửi đến CPU.
  - `Reset`: Tín hiệu từ bên ngoài gửi đến để khởi động lại máy tính.
- **Nhóm tín hiệu điều khiển bus (phục vụ DMA):**
  - `BRQ` (Bus Request): Từ mô-đun vào-ra (DMAC) gửi đến yêu cầu CPU chuyển nhượng quyền sử dụng bus.
  - `BGT` (Bus Grant): Từ CPU phát ra chấp nhận chuyển nhượng quyền sử dụng bus.
  - `Lock/Unlock`: Tín hiệu cấm hoặc cho phép xin chuyển nhượng bus.

4. Hạn chế của Bus dùng chung (Shared Bus) và Liên kết điểm-điểm

- **Hạn chế của Bus dùng chung:** Cần có bộ phân xử bus; tại một thời điểm chỉ phục vụ được một yêu cầu trao đổi dữ liệu nên độ trễ lớn; khó đồng bộ hóa khi tốc độ truyền cao.
- **Giải pháp khắc phục:** Sử dụng cấu trúc **Đa bus (Multiple bus)** hoặc **Liên kết điểm-điểm (Point-to-point interconnection)**.
- **Các liên kết điểm-điểm và bus ngoại vi phổ biến:**
  - **QPI (Quick Path Interconnect):** Đa kết nối trực tiếp từng cặp giữa các lõi xử lý (Cores) và Chipset, không cần phân xử bus, truyền dữ liệu dạng gói theo kiến trúc phân lớp.
  - **PCI Express (PCIe):** Liên kết điểm-điểm tốc độ cao thay thế cho bus dùng chung PCI truyền thống để nối ghép các thiết bị vào-ra băng thông lớn (card đồ họa, ổ SSD NVMe).
  - **SATA:** Chuẩn kết nối chuyên dụng chỉ dùng để kết nối với các ổ lưu trữ vật lý (HDD, SSD SATA).

- **USB (Universal Serial Bus):** Chuẩn bus ngoại vi nối tiếp đa dụng kết nối giữa mô-đun vào-ra và các thiết bị ngoại vi.

2.2 Phần 2: Toàn bộ bài tập Chương 2 và Hướng dẫn giải

2.2.1 Nhóm 1: Cấu trúc CPU, Bộ nhớ và Hệ thống vào-ra

Bài tập 2.1 (Đề Giữa kỳ 2018.2 – Mã đề 204, Câu 5, Câu 6 & Câu 13)
Hãy chọn phương án đúng cho 3 câu hỏi lý thuyết cơ bản sau:

a) **(Câu 5):** Đơn vị nào bên trong CPU chịu trách nhiệm lưu giữ các thông tin tạm thời phục vụ cho hoạt động của CPU?
A. Cache B. Bộ nhớ chính C. Tập thanh ghi (RF) D. Biến nhớ

b) **(Câu 6):** Loại bộ nhớ nào sau đây **không** thuộc về bộ nhớ trong?
A. Bộ nhớ ROM B. Bộ nhớ ổ cứng HDD C. Bộ nhớ chính D. Bộ nhớ cache

c) **(Câu 13):** Ngăn nhớ nằm ở chỗ nào trong hệ thống máy tính?
A. CPU B. Thiết bị ngoại vi C. Module vào-ra D. Bộ nhớ chính

Hướng dẫn giải:

- **Giải câu a:** Theo Slide Chương 2 (trang 7), bên trong CPU có 3 thành phần là CU, ALU và Tập thanh ghi (RF). Trong đó, **Tập thanh ghi (RF)** chứa các thông tin tạm thời phục vụ trực tiếp cho hoạt động của CPU → Chọn **C**.
- **Giải câu b:** ROM, RAM (bộ nhớ chính) và Cache đều là bộ nhớ bán dẫn thuộc bộ nhớ trong. **Ổ cứng HDD** là thiết bị lưu trữ từ tính thuộc **bộ nhớ ngoài** → Chọn **B**.
- **Giải câu c:** Theo Slide Chương 2 (trang 9), **Bộ nhớ chính** được tổ chức thành các ngăn nhớ được đánh địa chỉ → Chọn **D**.

Bài tập 2.2 (Đề Giữa kỳ 2022.2 – Đề 0009 Câu 11, Đề 0008 Câu 11 & Ôn tập Cuối kỳ 2025.2 – Câu 41)

a) **(Đề 0009 – Câu 11):** Điền từ còn thiếu vào 2 chỗ trống trong phát biểu sau: "Mỗi ngăn nhớ trong máy tính có ..... thay đổi được, nhưng ..... của nó luôn cố định."

b) **(Đề 0008 – Câu 11 & Ôn tập 2025.2 – Câu 41):** Loại bộ nhớ giúp CPU tăng tốc độ truy cập bộ nhớ chính có tên là gì? (Hoặc: Tác dụng chính của bộ nhớ Cache là gì?)
A. Giảm chi phí cho hệ thống máy tính B. Tăng dung lượng bộ nhớ chính
C. Tăng tốc độ truy cập đến bộ nhớ chính D. Tăng tốc độ truy cập đến ổ đĩa cứng

Hướng dẫn giải:

- **Giải câu a:** Theo Slide Chương 2 (trang 9), mỗi ngăn nhớ có **nội dung** (hoặc giá trị) có thể thay đổi khi ghi dữ liệu mới, nhưng **địa chỉ** (hoặc địa chỉ vật lý) của ngăn nhớ đó luôn cố định.

- **Giải câu b:** Bộ nhớ đặt đệm giữa CPU và bộ nhớ chính để tăng tốc độ CPU truy cập bộ nhớ chính là **Bộ nhớ đệm (Cache)** → Điền **Cache** (hoặc chọn đáp án **C**).

Bài tập 2.3 (Đề Giữa kỳ 2018.2 – Mã đề 199, Câu 20)
Bàn phím được kết nối trực tiếp với bộ phận chức năng nào trong hệ thống máy tính?

A. Với bộ nhớ chính

B. Với mô-đun vào-ra

C. Với CPU

D. Với bus hệ thống

Hướng dẫn giải:

- **Kiến thức áp dụng:** Theo sơ đồ Hệ thống vào-ra (Slide Chương 2, trang 12–14), bàn phím là một thiết bị vào-ra (thiết bị ngoại vi). Các thiết bị ngoại vi không thể nối trực tiếp vào CPU hay Bus hệ thống mà bắt buộc phải kết nối thông qua các cổng của **mô-đun vào-ra**.
- **Kết luận:** Đáp án đúng là **B (Với mô-đun vào-ra)**.

2.2.2 Nhóm 2: Chu trình lệnh, các thanh ghi PC / IR và Hoạt động Ngắt

Bài tập 2.4 (Đề Giữa kỳ 2018.2 – Mã đề 198 Câu 17, Mã đề 204 Câu 4 & Câu 18, Giữa kỳ 2022.2 – Đề 0008 Câu 10 & Ôn tập 2025.2 – Câu 56, 76)
Hãy trả lời chuỗi câu hỏi về các thanh ghi hệ thống trong quá trình Tìm nạp và Giải mã lệnh:

a) **(Đề GK 2022.2 – Đề 0008 & Mã đề 204 Câu 4):** Trong chu trình hoạt động cơ bản của máy tính, ở giai đoạn tìm nạp lệnh (nhận lệnh từ bộ nhớ chính), CPU sẽ tác động tới 2 thanh ghi hệ thống nào? Viết tên đầy đủ hoặc viết tắt.

b) **(Ôn tập 2025.2 Câu 76 & Mã đề 198 Câu 17):** Thanh ghi nào trong CPU luôn giữ địa chỉ của lệnh tiếp theo sẽ được lấy từ bộ nhớ và được tự động tăng mỗi khi CPU nhận xong một lệnh?
A. Thanh ghi Stack Pointer (SP) B. Thanh ghi Program Counter (PC)
C. Thanh ghi Instruction Register (IR) D. Thanh ghi Memory Address Register (MAR)

c) **(Mã đề 204 – Câu 18):** Trong quá trình giải mã lệnh, CPU sẽ tiến hành giải mã lệnh đang nằm trên thanh ghi nào?
A. Thanh ghi lệnh IR B. Thanh ghi con trỏ lệnh PC/IP C. Thanh ghi dữ liệu D. Thanh ghi trạng thái

d) **(Ôn tập 2025.2 – Câu 56):** Trong chu trình thực hiện lệnh cơ bản (Instruction Cycle), giai đoạn nào diễn ra ngay sau khi CPU lấy lệnh (Fetch) từ bộ nhớ?
A. Giai đoạn Ghi lại kết quả (Write Back) B. Giai đoạn Thực thi (Execute)
C. Giai đoạn Giải mã (Decode), nơi CPU xác định loại lệnh và các toán hạng D. Giai đoạn Điều khiển ngắt

Hướng dẫn giải:

- **Giải câu a:** Hai thanh ghi phục vụ quá trình tìm nạp lệnh là **PC (Program Counter – Bộ đếm chương trình)** và **IR (Instruction Register – Thanh ghi lệnh)**.
- **Giải câu b:** Thanh ghi giữ địa chỉ của lệnh tiếp theo và tự động tăng sau mỗi lần nạp lệnh là **Thanh ghi Bộ đếm chương trình (Program Counter – PC)** → Chọn **B**.
- **Giải câu c:** Lệnh sau khi đọc từ bộ nhớ lên được chứa trong thanh ghi lệnh **IR**, do đó Đơn vị điều khiển (CU) sẽ giải mã lệnh đang nằm trên **thanh ghi lệnh IR** → Chọn **A**.
- **Giải câu d:** Ngay sau khi tìm nạp lệnh (Fetch), CPU phải tiến hành **Giai đoạn Giải mã (Decode)** để biết lệnh đó yêu cầu thực hiện thao tác gì và lấy toán hạng ở đâu → Chọn **C**.

Bài tập 2.5 (Đề Giữa kỳ 2018.2 – Mã đề 198 Câu 14, Mã đề 199 Câu 8 & Ôn tập 2025.2 – Câu 15)

a) **(Mã đề 198 Câu 14):** Đáp án nào mô tả đúng về cơ chế ngắt (Interrupt)?
A. Tạm dừng chương trình đang thực hiện để chuyển đến thực hiện chương trình con khác (chương trình con xử lý ngắt).
B. Kết thúc phần mềm đang chạy.
C. Tạm dừng phần mềm đang hoạt động vĩnh viễn.
D. Hủy bỏ lệnh máy đang thực hiện dở mà không quay lại.

b) **(Ôn tập 2025.2 – Câu 15):** Với phương pháp vào-ra bằng chương trình, thành phần nào của máy tính trực tiếp điều khiển vào-ra?
A. DMAC B. Thiết bị vào-ra C. CPU D. RAM

Hướng dẫn giải:

- **Giải câu a:** Theo định nghĩa tại Slide Chương 2 (trang 22), ngắt là cơ chế cho phép CPU tạm dừng chương trình đang thực hiện để chuyển sang thực hiện chương trình con xử lý ngắt có sẵn trong bộ nhớ, sau đó quay trở về chương trình cũ → Chọn **A**.
- **Giải câu b:** Theo Slide Chương 2 (trang 26), khi vào-ra bằng chương trình, **CPU** trực tiếp điều khiển trao đổi dữ liệu với mô-đun vào-ra bằng các lệnh trong chương trình → Chọn **C (CPU)**.

2.2.3 Nhóm 3: Liên kết hệ thống, Các loại Bus và Tính toán không gian địa chỉ

Quy trình giải các bài toán về Độ rộng Bus địa chỉ và Dung lượng RAM

1. **Công thức cơ bản:** Nếu Bus địa chỉ có độ rộng N bit và bộ nhớ đánh địa chỉ theo từng byte:
$$\text{Dung lượng không gian địa chỉ tối đa} = 2^N \text{ byte}$$
2. **Đổi nhanh giữa đơn vị dung lượng và lũy thừa của 2:**
   - 1 KiB $= 2^{10}$ byte $\implies$ x KiB $= 2^k \times 2^{10} = 2^{10+k}$ byte.
   - 1 MiB $= 2^{20}$ byte $\implies$ x MiB $= 2^k \times 2^{20} = 2^{20+k}$ byte.
   - 1 GiB $= 2^{30}$ byte $\implies$ x GiB $= 2^k \times 2^{30} = 2^{30+k}$ byte.

   (Với $2 = 2^1$; $4 = 2^2$; $8 = 2^3$; $16 = 2^4$; $32 = 2^5$; $64 = 2^6$; $128 = 2^7$; $256 = 2^8$; $512 = 2^9$).
3. **Bài toán "Lắp thêm 1 thanh RAM lớn nhất và không thừa":**
   - Bước 1: Tính tổng không gian địa chỉ tối đa mà CPU quản lý được: $S_{\max} = 2^N$ (MiB hoặc GiB).
   - Bước 2: Tính phần dung lượng còn trống: $S_{\text{trống}} = S_{\max} - S_{\text{đã cắm}}$.
   - Bước 3: Vì dung lượng của **1 thanh RAM vật lý** luôn phải là một lũy thừa của 2 ($1, 2, 4, 8, 16, 32, 64$ MiB...), ta chọn thanh RAM có dung lượng là **lũy thừa của 2 lớn nhất mà nhỏ hơn hoặc bằng $S_{\text{trống}}$**.

Bài tập 2.6 (Đề Giữa kỳ 2018.2 – Mã đề 198 Câu 1 & Câu 10, Ôn tập 2025.2 – Câu 13 & Câu 26)

Hãy chọn phương án đúng cho 4 câu hỏi về đặc điểm của Bus hệ thống:

a) **(Mã đề 198 – Câu 1):** Một CPU có bus dữ liệu và bus địa chỉ có độ rộng đều bằng 32 bit. Hỏi độ rộng của bus điều khiển bằng bao nhiêu?
A. 64 bit B. 2^{32} bit C. Không xác định D. 32 bit

b) **(Mã đề 198 – Câu 10):** Loại bus nào vận chuyển nội dung lệnh từ bộ nhớ tới CPU?
A. Bus điều khiển B. Bus vào-ra C. Bus dữ liệu D. Bus địa chỉ

c) **(Ôn tập 2025.2 – Câu 13):** Bus dữ liệu (Data Bus) trong kiến trúc máy tính chịu trách nhiệm truyền tải loại thông tin nào?
A. Chỉ các dữ liệu giữa các thành phần trong hệ thống máy tính.
B. Các lệnh và dữ liệu giữa các thành phần trong hệ thống máy tính.
C. Địa chỉ bộ nhớ của dữ liệu cần truy cập.
D. Tín hiệu điều khiển để đồng bộ hóa hoạt động.

d) **(Ôn tập 2025.2 – Câu 26):** Bus Địa chỉ (Address Bus) có đặc điểm nào sau đây, đặc biệt là về hướng truyền?
A. Là bus một chiều (unidirectional), truyền địa chỉ từ CPU/DMA đến bộ nhớ/I/O.
B. Là bus hai chiều (bidirectional), truyền cả địa chỉ và dữ liệu.
C. Là bus hai chiều, truyền tín hiệu điều khiển và dữ liệu.
D. Là bus một chiều, truyền tín hiệu điều khiển từ bộ nhớ đến CPU.

Hướng dẫn giải:

- **Giải câu a:** Theo Slide Chương 2 (trang 31), khái niệm độ rộng bus chỉ áp dụng cho bus địa chỉ và bus dữ liệu. Số lượng đường dây điều khiển phụ thuộc vào thiết kế riêng của từng hệ thống chứ không phụ thuộc vào độ rộng bus dữ liệu hay địa chỉ → Chọn **C (Không xác định)**.
- **Giải câu b và câu c:** Theo Slide Chương 2 (trang 34), Bus dữ liệu có 2 chức năng: (1) Vận chuyển lệnh từ bộ nhớ đến CPU, và (2) Vận chuyển dữ liệu giữa các thành phần của máy tính với nhau → Câu b chọn **C (Bus dữ liệu)**; Câu c chọn **B (Các lệnh và dữ liệu)**.
- **Giải câu d:** Chỉ có CPU (hoặc DMAC) phát địa chỉ ra để định vị ngăn nhớ hoặc cổng vào-ra, do đó Bus địa chỉ là bus một chiều → Chọn **A**.

Bài tập 2.7 (Đề Giữa kỳ 2019.2 – Mã đề 243 Câu 5, 6, 13 & Mã đề 244 Câu 7, 9, 13)

a) **(Mã đề 243 Câu 6 & Mã đề 244 Câu 9):** Với lệnh hợp ngữ ghi bộ nhớ `sw t2, 4(t4)` (hoặc `sw s2, 8(s4)`), giá trị tổng `4 + t4` (hoặc `8 + s4`) sẽ được truyền đi từ CPU tới bộ nhớ chính trên đường bus nào?
A. Bus dữ liệu B. Bus địa chỉ C. Bus điều khiển D. Bus ngoại vi

b) **(Mã đề 243 Câu 5 & Mã đề 244 Câu 7):** Loại bus ngoại vi nào dưới đây **chỉ dùng** để kết nối với ổ lưu trữ vật lý (HDD, SSD)?
A. HDMI B. PCIe C. SATA D. USB

c) **(Mã đề 243 Câu 13 & Mã đề 244 Câu 13):** Trong kiến trúc máy tính, khái niệm USB nói về điều gì?
A. Ổ lưu trữ B. Bus ngoại vi giữa mô-đun I/O và thiết bị C. Khe cắm mở rộng RAM D. Nhà sản xuất linh kiện

Hướng dẫn giải:

- **Giải câu a:** Trong lệnh `sw t2, 4(t4)`, giá trị `4 + t4` là **địa chỉ hiệu dụng** của ngăn nhớ cần ghi dữ liệu, nên giá trị này được CPU phát ra trên **Bus địa chỉ** (còn nội dung của thanh ghi `t2` mới truyền trên Bus dữ liệu) → Chọn **B (Bus địa chỉ)**.
- **Giải câu b:** Chuẩn **SATA** (Serial ATA) là chuẩn kết nối chuyên dụng chỉ dùng cho các

ổ đĩa lưu trữ (ổ cứng HDD, SSD SATA, ổ đĩa quang) → Chọn **C (SATA)**.

- **Giải câu c:** USB (Universal Serial Bus) là chuẩn **bus ngoại vi giữa mô-đun vào-ra và thiết bị** → Chọn **B**.

Bài tập 2.8 (Tổng hợp các câu tính độ rộng Bus địa chỉ từ Đề Giữa kỳ 2019.2, 2022.2 & Cuối kỳ 2020.2, 2022.2, 2025.2)
Biết bộ nhớ chính được đánh địa chỉ theo từng byte. Hãy thực hiện các tính toán sau:

a) **(Đề GK 2022.2 – Đề 0008, Câu 3):** Máy tính có bus dữ liệu 32 bit và bus địa chỉ 23 bit. Hỏi không gian địa chỉ bộ nhớ có kích thước bao nhiêu byte (hoặc MiB)?

b) **(Đề GK 2022.2 – Đề 0006, Câu 3):** Nếu `0x3FF` là địa chỉ bộ nhớ lớn nhất mà CPU có thể quản lý được, thì bus địa chỉ có độ rộng bằng bao nhiêu bit?

c) **(Đề CK 2020.2 & Ôn tập 2025.2 Câu 11):** Không gian địa chỉ bộ nhớ là 2 GiB, độ rộng bus địa chỉ là bao nhiêu bit?

d) **(Đề GK 2019.2 – Mã đề 243 Câu 16 & Mã đề 244 Câu 4):** Không gian địa chỉ bộ nhớ là 64 GiB (Mã đề 243) hoặc 128 GiB (Mã đề 244), độ rộng bus địa chỉ là bao nhiêu bit?

e) **(Đề CK 2022.2 – #bb55ef & CK 2024.2):** Một hệ thống máy tính gồm có 4 MiB bộ nhớ cache, 8 GiB bộ nhớ chính, 1 TiB bộ nhớ thể rắn SSD. Hỏi bus địa chỉ của hệ thống cần có độ rộng tối thiểu bao nhiêu bit?

Hướng dẫn giải:

- **Giải câu a:** Không gian địa chỉ chỉ phụ thuộc vào độ rộng bus địa chỉ ($N = 23$ bit):

$$\text{Kích thước} = 2^{23} \text{ byte} = 2^3 \times 2^{20} \text{ byte} = \mathbf{8} \text{ MiB (hoặc } 8\,388\,608 \text{ byte)}$$

- **Giải câu b:** Đổi địa chỉ lớn nhất `0x3FF` từ hệ Hexa sang hệ nhị phân:

$$\texttt{0x3FF} = 0011\,1111\,1111_{(2)} \rightarrow \text{gồm 10 bit 1 liên tiếp } (= 2^{10} - 1)$$

  Địa chỉ chạy từ `0x000` đến `0x3FF` gồm $2^{10} = 1024$ địa chỉ $\implies$ Độ rộng bus địa chỉ là **10 bit**.

- **Giải câu c:** Ta có 2 GiB $= 2^1 \times 2^{30}$ byte $= 2^{31}$ byte $\implies$ Độ rộng bus địa chỉ là **31 bit**.

- **Giải câu d:**
  - Với 64 GiB $= 2^6 \times 2^{30}$ byte $= 2^{36}$ byte $\implies$ Độ rộng bus địa chỉ là **36 bit**.
  - Với 128 GiB $= 2^7 \times 2^{30}$ byte $= 2^{37}$ byte $\implies$ Độ rộng bus địa chỉ là **37 bit**.

- **Giải câu e:**

- Bộ nhớ Cache tự động ánh xạ từ bộ nhớ chính; ổ cứng SSD là thiết bị lưu trữ ngoài giao tiếp qua mô-đun vào-ra theo khối chứ không đánh địa chỉ từng byte trên bus địa chỉ CPU.
- Bus địa chỉ hệ thống chỉ cần định vị trực tiếp cho 8 GiB bộ nhớ chính (RAM):

$$8 \text{ GiB} = 2^3 \times 2^{30} \text{ byte} = 2^{33} \text{ byte} \implies \text{Độ rộng tối thiểu là } \mathbf{33} \text{ bit}$$

Bài tập 2.9 (Đề Giữa kỳ 2018.2 – Mã đề 204 Câu 16 & Đề Giữa kỳ 2019.2 – Mã đề 243 Câu 14, Mã đề 244 Câu 6)

a) **(Mã đề 204 – Câu 16):** Một hệ thống máy tính cần sử dụng lượng bộ nhớ chính lên tới 128 KiB, cổng vào-ra 64 KiB (dùng chung không gian địa chỉ), và ổ cứng SSD 256 KiB. Hỏi bus địa chỉ phải có độ rộng tối thiểu bằng bao nhiêu bit?
A. 17 bit B. 18 bit C. 19 bit D. 20 bit

b) **(Mã đề 243 – Câu 14):** Bus hệ thống của một máy tính có đường bus địa chỉ 24 bit. Biết rằng máy tính đã cắm 1 thanh RAM 8 MiB. Hỏi máy tính có thể lắp thêm **1 thanh RAM** kích thước **lớn nhất** là bao nhiêu và **không thừa**? (Giả định máy tính không có cổng vào-ra).
A. 2 MiB B. 4 MiB C. 16 MiB D. 8 MiB

c) **(Mã đề 244 – Câu 6):** Tương tự câu b, nhưng máy tính có đường bus địa chỉ 26 bit và đã cắm 1 thanh RAM 8 MiB. Hỏi có thể lắp thêm **1 thanh RAM** kích thước lớn nhất là bao nhiêu và không thừa?
A. 8 MiB B. 64 MiB C. 32 MiB D. 16 MiB

Hướng dẫn giải:

- **Giải câu a:**
  - Ổ cứng SSD là bộ nhớ ngoài nên không tính vào không gian địa chỉ trực tiếp của CPU.
  - Tổng không gian địa chỉ cần đánh cho Bộ nhớ chính và Cổng vào-ra là: 128 KiB + 64 KiB = 192 KiB.
  - Với 17 bit chỉ đánh địa chỉ được $2^{17} = 128$ KiB (thiếu). Để đánh địa chỉ cho 192 KiB (nằm giữa 128 KiB và 256 KiB = 2^{18}), độ rộng bus địa chỉ tối thiểu phải là **18 bit** → Chọn **B (18 bit)**.
- **Giải câu b (Bus địa chỉ 24 bit):**
  - Bước 1: Tổng không gian bộ nhớ tối đa mà 24 bit quản lý được là: 2^{24} byte = $2^4 \times 2^{20}$ byte = 16 MiB.
  - Bước 2: Máy đã cắm 8 MiB, vậy dung lượng địa chỉ còn trống là: 16 MiB − 8 MiB = 8 MiB.

- Bước 3: Lắp thêm 1 thanh RAM có kích thước **8 MiB** là vừa khít và không thừa → Chọn **D (8 MiB)**.

- **Giải câu c (Bus địa chỉ 26 bit):**
  - Bước 1: Tổng không gian bộ nhớ tối đa mà 26 bit quản lý được là: 2^{26} byte = $2^6 \times 2^{20}$ byte = 64 MiB.
  - Bước 2: Máy đã cắm 8 MiB, vậy phần không gian địa chỉ còn trống là: 64 MiB − 8 MiB = 56 MiB.
  - Bước 3: Vì dung lượng của **1 thanh RAM** thương mại phải là lũy thừa của 2 $(8, 16, 32, 64$ MiB): nếu lắp thanh 64 MiB thì tổng là $8 + 64 = 72$ MiB > 64 MiB (bị thừa 8 MiB không nhận diện được). Do đó thanh RAM đơn lẻ lớn nhất lắp vào mà không bị thừa là thanh **32 MiB** (tổng $8 + 32 = 40$ MiB ≤ 64 MiB) → Chọn **C (32 MiB)**.

Bài tập 2.10 (Đề Cuối kỳ 2021.2 – Mã đề B3, Câu 4.1 & 4.2)
Máy tính dùng 36 bit địa chỉ để đánh địa chỉ cho bộ nhớ theo byte, 16 bit để đánh địa chỉ cho cổng vào-ra riêng biệt. Hãy cho biết:

4.1. Số byte nhớ tối đa được đánh địa chỉ? Địa chỉ đầu và địa chỉ cuối dưới dạng Hexa?

4.2. Số cổng vào-ra tối đa được đánh địa chỉ vào-ra riêng biệt? Địa chỉ đầu và địa chỉ cuối dưới dạng Hexa?

Hướng dẫn giải:

- **Giải ý 4.1 (Không gian địa chỉ bộ nhớ 36 bit):**
  - Số byte nhớ tối đa được đánh địa chỉ: 2^{36} byte = $2^6 \times 2^{30}$ byte = **64** GiB.
  - Vì mỗi chữ số Hexa biểu diễn cho 4 bit nhị phân nên 36 bit ứng với $36 \div 4 = 9$ chữ số Hexa:
    - Địa chỉ đầu (tất cả 36 bit bằng 0): `0x000000000` (9 chữ số 0).
    - Địa chỉ cuối (tất cả 36 bit bằng 1): `0xFFFFFFFFF` (9 chữ số F).

- **Giải ý 4.2 (Không gian địa chỉ cổng vào-ra 16 bit):**
  - Số cổng vào-ra tối đa: $2^{16} =$ **65 536** cổng (hay 64K cổng).
  - Vì 16 bit ứng với $16 \div 4 = 4$ chữ số Hexa:
    - Địa chỉ cổng đầu: `0x0000`.
    - Địa chỉ cổng cuối: `0xFFFF`.

CHƯƠNG 3

KIẾN TRÚC TẬP LỆNH RISC-V

3.1 Phần 1: Lý thuyết trọng tâm

3.1.1 Tổng quan về Kiến trúc tập lệnh và Tập 32 thanh ghi

1. Đặc trưng của kiến trúc tập lệnh thu gọn RISC và RISC-V

- **Phân biệt CISC và RISC:**
  - *CISC (Complex Instruction Set Computer):* Máy tính với tập lệnh phức tạp (điển hình là Intel x86, Motorola 680x0). Độ dài lệnh thay đổi từ 1 đến 15 byte, cho phép lệnh tính toán trực tiếp với toán hạng nằm trong bộ nhớ.
  - *RISC (Reduced Instruction Set Computer):* Máy tính với tập lệnh thu gọn (điển hình là RISC-V, MIPS, ARM).
- **Các đặc trưng cốt lõi của kiến trúc RISC (và RISC-V 32-bit):**
  1. Các lệnh chuẩn có độ dài cố định bằng 32 bit (= 4 byte). Do đó, địa chỉ của mọi lệnh máy chuẩn trong bộ nhớ luôn là **bội số của 4** (chữ số Hexa tận cùng là `0, 4, 8, C`).
  2. Các lệnh số học và logic chỉ thao tác với các toán hạng nằm trong **thanh ghi** (hoặc hằng số nằm ngay trong lệnh).
  3. Chỉ truy cập bộ nhớ dữ liệu thông qua các lệnh chuyên biệt là **Load** (đọc từ bộ nhớ vào thanh ghi) và **Store** (ghi từ thanh ghi ra bộ nhớ).

2. Tập 32 thanh ghi đa dụng của RISC-V 32-bit (RV32I)

Bộ xử lý RISC-V 32-bit có 32 thanh ghi số nguyên, mỗi thanh ghi rộng 32 bit, đánh số từ `x0` đến `x31`. Vì $32 = 2^5$, mỗi thanh ghi được mã hóa trong lệnh máy bằng đúng **5 bit** (từ `00000` đến `11111`).

Bảng tra cứu 32 thanh ghi của RISC-V và Quy ước bảo toàn khi gọi hàm

Tên gọi	Số hiệu	Chức năng sử dụng trong chương trình	Quy ước bảo toàn
`zero`	`x0`	Luôn chứa hằng số 0 (thanh ghi chỉ đọc, mọi thao tác ghi vào `x0` đều bị bỏ qua)	Không áp dụng
`ra`	`x1`	Return Address: Chứa địa chỉ trở về khi gọi chương trình con	Caller-saved
`sp`	`x2`	Stack Pointer: Con trỏ ngăn xếp (luôn trỏ vào đỉnh ngăn xếp)	Callee-saved
`gp`	`x3`	Global Pointer: Con trỏ vùng dữ liệu tĩnh toàn cục	Không áp dụng
`tp`	`x4`	Thread Pointer: Con trỏ luồng thực thi	Không áp dụng
`t0 - t2`	`x5 - x7`	Temporaries: Các thanh ghi chứa giá trị trung gian tạm thời	Caller-saved
`s0 / fp`	`x8`	Saved register / Frame pointer: Lưu biến hoặc làm con trỏ khung	Callee-saved
`s1`	`x9`	Saved register: Thanh ghi lưu biến của chương trình	Callee-saved
`a0 - a1`	`x10 - x11`	Arguments / Return values: Truyền tham số đầu vào và nhận kết quả trả về của hàm	Caller-saved
`a2 - a7`	`x12 - x17`	Arguments: Truyền các tham số đầu vào thứ 3 đến thứ 8 của hàm	Caller-saved
`s2 - s11`	`x18 - x27`	Saved registers: Các thanh ghi lưu biến bền vững của chương trình	Callee-saved
`t3 - t6`	`x28 - x31`	Temporaries: Các thanh ghi chứa giá trị trung gian tạm thời	Caller-saved

Quy tắc bảo toàn thanh ghi khi gọi chương trình con:

- **Nhóm tạm thời (`t0 - t6`, `a0 - a7`, `ra`):** Thuộc nhóm **Caller-saved** (hàm gọi tự bảo vệ trên ngăn xếp nếu muốn dùng lại sau lời gọi hàm; hàm được gọi có quyền thay đổi tự do).
- **Nhóm lưu trữ (`s0 - s11`, `sp`):** Thuộc nhóm **Callee-saved** (hàm được gọi bắt buộc phải cất giá trị cũ vào ngăn xếp trước khi sử dụng và khôi phục lại nguyên trạng trước khi thoát hàm).

3.1.2 Tổ chức dữ liệu trong bộ nhớ và Quy tắc Đầu nhỏ (Little-Endian)

1. Các đơn vị dữ liệu và Quy tắc căn chỉnh biên địa chỉ (Alignment)

- **Byte (8 bit):** Chiếm 1 ngăn nhớ. Địa chỉ có thể là số chẵn hoặc số lẻ bất kỳ.
- **Halfword – Nửa từ (16 bit = 2 byte):** Chiếm 2 ngăn nhớ liên tiếp. Địa chỉ hiệu dụng bắt buộc phải là **số chẵn** (chia hết cho 2, chữ số Hexa cuối là `0, 2, 4, 6, 8, A, C, E`).
- **Word – Từ nhớ (32 bit = 4 byte):** Chiếm 4 ngăn nhớ liên tiếp. Địa chỉ của một Word là địa chỉ của byte đầu tiên (byte có địa chỉ nhỏ nhất) thuộc Word đó và bắt buộc phải **chia hết cho 4** (chữ số Hexa cuối cùng chỉ có thể là `0, 4, 8` hoặc `C`).

2. Thứ tự lưu trữ các byte trong bộ nhớ: Đầu nhỏ (Little-Endian) vs. Đầu to (Big-Endian)

- **Đầu nhỏ (Little-Endian):** Byte có ý nghĩa thấp nhất (LSB – 2 chữ số Hexa ngoài cùng bên phải) được lưu ở ngăn nhớ có **địa chỉ nhỏ nhất**; byte có ý nghĩa cao nhất (MSB – 2 chữ số Hexa ngoài cùng bên trái) được lưu ở ngăn nhớ có **địa chỉ lớn nhất**.
  - *Các kiến trúc sử dụng:* **RISC-V** và **Intel x86** sử dụng Little-Endian.
- **Đầu to (Big-Endian):** Byte có ý nghĩa cao nhất (MSB) được lưu ở ngăn nhớ có địa chỉ nhỏ nhất; byte có ý nghĩa thấp nhất (LSB) được lưu ở ngăn nhớ có địa chỉ lớn nhất.
  - *Các kiến trúc sử dụng:* Motorola 680x0, SunSPARC dùng Big-Endian. Riêng kiến trúc **ARM** hỗ trợ cả Little-Endian hoặc Big-Endian tùy phiên bản cấu hình.

Word 32-bit = `0x1A2B3C4D` lưu tại địa chỉ `0x10010008`

Thứ tự Đầu nhỏ (Little-Endian – RISC-V)

0x4D (LSB)	0x10010008
0x3C	0x10010009
0x2B	0x1001000A
0x1A (MSB)	0x1001000B

Thứ tự Đầu to (Big-Endian)

0x1A (MSB)	0x10010008
0x2B	0x10010009
0x3C	0x1001000A
0x4D (LSB)	0x1001000B

3.1.3 Các nhóm lệnh hợp ngữ RISC-V cốt lõi

1. Nhóm lệnh số học và Nạp hằng số vào thanh ghi

- `add rd, rs1, rs2`: Cộng hai thanh ghi ($rd = rs1 + rs2$).
- `sub rd, rs1, rs2`: Trừ hai thanh ghi ($rd = rs1 - rs2$).
- `addi rd, rs1, imm`: Cộng thanh ghi với hằng số 12-bit có dấu ($rd = rs1 + \text{SignExt}(imm)$).

- *Giới hạn của hằng số* `imm`*:* Vì `imm` là số nguyên có dấu 12 bit, giá trị hợp lệ bắt buộc phải nằm trong đoạn $[-\mathbf{2048}, +\mathbf{2047}]$ (viết ở hệ Hexa 12-bit là từ `0x800` đến `0x7FF`, hoặc mở rộng dấu 32-bit là từ `0xFFFFF800` đến `0x000007FF`). Nếu truyền số ≥ 2048 (như `0x800`) vào lệnh `addi`, trình dịch sẽ báo lỗi.
- RISC-V **không có lệnh** `subi`; để trừ một hằng số, ta dùng lệnh `addi` với hằng số âm.

- `lui rd, upimm` (Load Upper Immediate): Đưa hằng số 20-bit (`upimm`) vào 20 bit cao của thanh ghi đích `rd` (từ bit 31 đến bit 12) và xóa 12 bit thấp (bit 11 đến bit 0) về 0.

Quy tắc nạp hằng số 32-bit bất kỳ vào thanh ghi (Bản chất của lệnh giả `li`)

Để nạp hằng số 32-bit dạng `0xABCDE123` vào thanh ghi `rd` mà không dùng lệnh giả `li`, ta tách thành 20 bit cao (5 chữ số Hexa đầu `0xABCDE`) và 12 bit thấp (3 chữ số Hexa cuối `0x123`), rồi kết hợp `lui` với `addi`:

1. **Trường hợp 1 (Chữ số Hexa thứ 3 từ phải sang nhỏ hơn 8 – từ `0` đến `7`):** Bit số 11 bằng 0 nên phần 12-bit thấp là số dương. Giữ nguyên 5 chữ số Hexa đầu cho lệnh `lui`:
   - Ví dụ nạp `0x20241120`: `lui t0, 0x20241` → `addi t0, t0, 0x120`.
2. **Trường hợp 2 (Chữ số Hexa thứ 3 từ phải sang từ `8` đến `F`):** Bit số 11 bằng 1 nên lệnh `addi` sẽ mở rộng dấu và hiểu 12-bit thấp là một **số âm** (bị trừ lùi đi $2^{12} =$ `0x1000`). Để bù lại, ta bắt buộc phải **cộng thêm 1 vào 5 chữ số Hexa đầu** trong lệnh `lui`:
   - Ví dụ nạp `0x12345EDC` (chữ thứ 3 từ phải sang là `E` ≥ 8, với `0xEDC` $= -292$): Dùng `lui s1, 0x12346` (tăng `12345` lên `12346`), sau đó `addi s1, s1, -292`.

2. Nhóm lệnh Đọc và Ghi bộ nhớ (Load / Store)

- **Thao tác với Word (32 bit = 4 byte):**
  - `lw rd, imm(rs1)`: Đọc 1 word (4 byte) tại địa chỉ `rs1` + SignExt(`imm`) nạp vào thanh ghi `rd`.
  - `sw rs2, imm(rs1)`: Ghi 1 word (4 byte) từ thanh ghi nguồn `rs2` ra bộ nhớ tại địa chỉ `rs1` + SignExt(`imm`).
- **Thao tác với Halfword (16 bit = 2 byte):**
  - `lh rd, imm(rs1)`: Đọc 2 byte từ bộ nhớ vào 16 bit thấp của `rd`, mở rộng **có dấu** (Sign-extend: lấy bit số 15 điền vào toàn bộ 16 bit cao của `rd`).
  - `lhu rd, imm(rs1)`: Đọc 2 byte từ bộ nhớ vào 16 bit thấp của `rd`, mở rộng **không dấu** (Zero-extend: luôn điền `0x0000` vào 16 bit cao của `rd`).

  - `sh rs2, imm(rs1)`: Ghi 2 byte bên phải (16 bit thấp) của thanh ghi `rs2` ra bộ nhớ.

- **Thao tác với Byte (8 bit = 1 byte):**
  - `lb rd, imm(rs1)`: Đọc 1 byte từ bộ nhớ vào 8 bit thấp của `rd`, mở rộng **có dấu** (Sign-extend: nếu byte đọc lên có chữ số Hexa đầu từ `8` đến `F` thì 24 bit cao của `rd` điền toàn `0xFFFFFF`; nếu từ `0` đến `7` thì điền `0x000000`).
  - `lbu rd, imm(rs1)`: Đọc 1 byte từ bộ nhớ vào 8 bit thấp của `rd`, mở rộng **không dấu** (Zero-extend: luôn điền `0x000000` vào 24 bit cao của `rd`).
  - `sb rs2, imm(rs1)`: Chỉ ghi 1 byte bên phải (8 bit thấp) của thanh ghi `rs2` ra bộ nhớ.

3. Nhóm lệnh Logic và Dịch bit

- **Các phép toán logic từng bit (Bitwise):**
  - `and rd, rs1, rs2` và `andi rd, rs1, imm`: Phép AND (1 AND 1 = 1, còn lại bằng 0). Dùng để xóa một số bit về 0 và trích xuất các bit cần giữ.
  - `or rd, rs1, rs2` và `ori rd, rs1, imm`: Phép OR (0 OR 0 = 0, còn lại bằng 1). Dùng để thiết lập một số bit lên 1.
  - `xor rd, rs1, rs2` và `xori rd, rs1, imm`: Phép XOR (hai bit khác nhau thì bằng 1, giống nhau thì bằng 0). Dùng để đảo giá trị các bit (thay thế cho lệnh `not rd, rs1` bằng `xori rd, rs1, -1`) hoặc xóa thanh ghi về 0 (`xor s0, s0, s0`).
  - *Lưu ý đặc biệt của RISC-V:* Hằng số 12-bit `imm` trong các lệnh `andi, ori, xori` của RISC-V luôn được **mở rộng có dấu (Sign-extended)** thành 32-bit trước khi thực hiện phép toán.

- **Các phép dịch bit (Shift):** Với `uimm` là hằng số không dấu 5 bit (từ 0 đến 31), hoặc dịch theo 5 bit thấp của thanh ghi `rs2`:
  - *Dịch trái logic (`sll, slli`):* Đẩy các bit sang trái k vị trí, chèn k bit `0` vào bên phải. Tương đương **nhân với** 2^k.
  - *Dịch phải logic (`srl, srli`):* Đẩy các bit sang phải k vị trí, luôn chèn k bit `0` vào bên trái. Tương đương **chia số không dấu cho** 2^k lấy phần nguyên.
  - *Dịch phải số học (`sra, srai`):* Đẩy các bit sang phải k vị trí, nhưng **giữ nguyên bit dấu** (nếu bit 31 đang là `1` thì chèn toàn bit `1` vào bên trái; nếu bit 31 là `0` thì chèn bit `0`). Tương đương **chia số có dấu cho** 2^k lấy phần nguyên.
  - *Quy đổi nhanh khi dịch theo bội số của 4 bit:* Vì 1 chữ số Hexa = 4 bit, nên dịch trái/phải 4 bit là dịch 1 chữ số Hexa; dịch 8 bit là dịch 2 chữ số Hexa; dịch 20 bit là dịch 5 chữ số Hexa; dịch 24 bit là dịch 6 chữ số Hexa.

4. Nhóm lệnh So sánh, Rẽ nhánh, Nhảy và Quản lý Ngăn xếp (Stack)

- **Lệnh thiết lập nếu nhỏ hơn (Set Less Than):**

- `slt rd, rs1, rs2` và `slti rd, rs1, imm`: So sánh hai số nguyên **có dấu**. Nếu `rs1` < `rs2` (hoặc SignExt(`imm`)) thì `rd` = 1, ngược lại `rd` = 0.
- `sltu rd, rs1, rs2` và `sltiu rd, rs1, imm`: So sánh hai số nguyên **không dấu**.

- **Các lệnh rẽ nhánh có điều kiện (Branch):**
  - `beq rs1, rs2, L1` (rẽ nhánh nếu `rs1` == `rs2`); `bne rs1, rs2, L1` (rẽ nhánh nếu `rs1` $\neq$ `rs2`).
  - `blt rs1, rs2, L1` (rẽ nhánh nếu `rs1` < `rs2` theo số có dấu); `bge rs1, rs2, L1` (nếu `rs1` $\geq$ `rs2` có dấu).
  - `bltu rs1, rs2, L1` (rẽ nhánh nếu `rs1` < `rs2` không dấu); `bgeu rs1, rs2, L1` (nếu `rs1` $\geq$ `rs2` không dấu).
  - *Lưu ý khi dịch câu lệnh `if` / `while` từ C sang hợp ngữ:* Điều kiện kiểm tra trong lệnh rẽ nhánh nhảy tới nhãn thoát (`done` hoặc `else`) thường là **điều kiện ngược** với điều kiện viết trong câu lệnh C (ví dụ: trong C viết `if (i == j)` thì hợp ngữ dùng `bne` để nhảy qua khối lệnh nếu $i \neq j$).

- **Các lệnh gọi và trở về từ chương trình con (Thủ tục):**
  - `jal rd, Label` (Jump and Link): Thực hiện đồng thời 2 việc: (1) Lưu địa chỉ của lệnh kế tiếp vào thanh ghi đích `rd` = $PC + 4$, và (2) Nhảy tới nhãn đích $PC = PC + \text{imm}$.
    - Khi gọi chương trình con: Dùng `jal ra, ProcLabel` (lưu địa chỉ trở về vào `ra`).
    - Khi nhảy không điều kiện (tương đương lệnh giả `j Label`): Dùng `jal x0, Label` (hủy địa chỉ trở về vào `x0`).
  - `jalr rd, rs1, imm` (Jump and Link Register): Lưu `rd` = $PC + 4$ và nhảy tới địa chỉ PC = `rs1` + SignExt(`imm`).
    - Khi trở về từ chương trình con (tương đương lệnh giả `ret` hoặc `jr ra`): Dùng `jalr x0, ra, 0` (hoặc viết dạng `jalr x0, 0(ra)`).

- **Thao tác với Ngăn xếp (Stack):**
  - Ngăn xếp là vùng nhớ hoạt động theo cơ chế **LIFO (Last In – First Out: Vào sau – Ra trước)** nằm trong bộ nhớ chính (RAM). Thanh ghi `sp` (`x2`) luôn chứa địa chỉ của **đỉnh ngăn xếp**.
  - Trong RISC-V, ngăn xếp phát triển theo chiều **giảm địa chỉ bộ nhớ**:
    1. **Khi cất (Push) k thanh ghi (mỗi thanh ghi 4 byte) vào ngăn xếp:** Phải giảm `sp` trước bằng lệnh `addi sp, sp, -4*k`, sau đó dùng lệnh `sw` lưu các thanh ghi vào `0(sp)`, `4(sp)`, ...
    2. **Khi lấy (Pop / Khôi phục) k thanh ghi ra khỏi ngăn xếp:** Phải dùng lệnh `lw` đọc các thanh ghi từ `0(sp)`, `4(sp)`, ... ra đúng vị trí cũ trước, sau đó mới tăng `sp` trả lại bằng lệnh `addi sp, sp, 4*k`.

3.1.4 Mã máy 32-bit của RISC-V, Lệnh nén 16-bit và 4 Phương thức định địa chỉ

1. Sáu khuôn dạng lệnh máy 32-bit của RISC-V (RV32I)

Mọi câu lệnh chuẩn 32-bit của RISC-V đều thuộc 1 trong 6 khuôn dạng: **R, I, S, B, U, J**. Trường `opcode` luôn cố định ở 7 bit cuối cùng (`bit 6:0`).

Bảng cấu trúc 6 khuôn dạng lệnh máy 32-bit của RISC-V

Kiểu	Bit 31:25 (7b)	Bit 24:20 (5b)	Bit 19:15 (5b)	Bit 14:12 (3b)	Bit 11:7 (5b)	Bit 6:0 (7b)
R-type	`funct7`	`rs2`	`rs1`	`funct3`	`rd`	`opcode`
I-type	`imm[11:0]` (12 bit)		`rs1`	`funct3`	`rd`	`opcode`
S-type	`imm[11:5]`	`rs2`	`rs1`	`funct3`	`imm[4:0]`	`opcode`
B-type	`imm[12,10:5]`	`rs2`	`rs1`	`funct3`	`imm[4:1,11]`	`opcode`
U-type	`imm[31:12]` (20 bit)				`rd`	`opcode`
J-type	`imm[20,10:1,11,19:12]` (20 bit)				`rd`	`opcode`

Các đặc điểm nhận dạng nhanh trong đề thi:

- Khuôn dạng duy nhất **không chứa hằng số (`imm`)** là kiểu **R-type** (chỉ dùng cho các phép toán thanh ghi – thanh ghi).
- Khuôn dạng bắt buộc sử dụng đầy đủ cả hai trường **`funct3` và `funct7`** là kiểu **R-type** (ngoài ra các lệnh dịch bit tức thì `slli, srli, srai` thuộc kiểu I nhưng dùng 7 bit đầu `imm[11:5]` đóng vai trò như `funct7` và 5 bit `imm[4:0]` làm số lượng bit cần dịch `uimm`).
- Hai khuôn dạng có trường hằng số dài **20 bit** là kiểu **U-type** (`lui, auipc`) và kiểu **J-type** (`jal`). Lưu ý: Lệnh `jalr` thuộc kiểu **I-type** (12-bit `imm`), không phải J-type.
- Hai khuôn dạng **không có thanh ghi đích `rd`** là kiểu **S-type** (Store) và kiểu **B-type** (Branch).

Bảng tra cứu nhanh Opcode, Funct3 và Funct7 của tập lệnh RV32I

Nhóm / Kiểu	Opcode (6:0)	funct3	funct7	Tên lệnh hợp ngữ tương ứng
R-type (Số học / Logic)	0110011 (51)	000	0000000	`add rd, rs1, rs2`
		000	0100000	`sub rd, rs1, rs2`
		001 / 010	0000000	`sll` (001), `slt` (010), `sltu` (011)
		100 / 101	000/010..	`xor` (100), `srl/sra` (101), `or` (110), `and` (111)
I-type (Tính toán)	0010011 (19)	000 / 010	–	`addi` (000), `slti` (010), `sltiu` (011)
		100 / 110	–	`xori` (100), `ori` (110), `andi` (111), `slli` (001), `srli/srai` (101)
I-type (Load)	0000011 (3)	000..101	–	`lb` (000), `lh` (001), `lw` (010), `lbu` (100), `lhu` (101)
S-type (Store)	0100011 (35)	000..010	–	`sb` (000), `sh` (001), `sw` (010)
B-type (Branch)	1100011 (99)	000..111	–	`beq` (000), `bne` (001), `blt` (100), `bge` (101), `bltu` (110), `bgeu` (111)
U-type	0110111 (55)	–	–	`lui rd, upimm` (`auipc` có opcode 0010111)
J-type & **I-jalr**	1101111 (111)	–	–	`jal rd, label` (Riêng `jalr` là kiểu I, opcode 1100111, `funct3=000`)

2. Tập lệnh nén 16-bit (Compressed Instructions – RVC):

- Để giảm dung lượng bộ nhớ lưu trữ mã chương trình, RISC-V (cũng như ARM Thumb) cung cấp tập lệnh nén có độ dài **16 bit** (2 byte) cho các thao tác thường gặp, ký hiệu bằng tiền tố `c.` (ví dụ: `c.add, c.addi, c.lw`).
- **Các hạn chế của lệnh nén 16-bit so với lệnh 32-bit:**
  1. Giảm số lượng thanh ghi có thể truy cập trực tiếp (nhiều lệnh nén chỉ dùng 3 bit để định danh 8 thanh ghi phổ biến từ `x8` đến `x15`).
  2. Giảm số lượng toán hạng trong lệnh (thường gộp thanh ghi đích trùng với thanh ghi nguồn thứ nhất: `rd` = `rd` + `rs2`).
  3. Phạm vi biểu diễn của hằng số tức thời (`imm`) và khoảng cách nhảy rẽ nhánh bị thu

hẹp đáng kể.

3. Bốn phương thức định địa chỉ của RISC-V

1. **Định địa chỉ thanh ghi (Register Addressing):** Toán hạng nằm hoàn toàn trong các thanh ghi (áp dụng cho các lệnh kiểu R).
2. **Định địa chỉ tức thì (Immediate Addressing):** Toán hạng là hằng số nằm ngay bên trong lệnh máy (áp dụng cho toán hạng hằng số của `addi, andi, ori, lui...`).
3. **Định địa chỉ cơ sở (Base Addressing):** Toán hạng nằm trong bộ nhớ, địa chỉ được tính bằng tổng nội dung thanh ghi cơ sở và hằng số độ lệch: Address = `rs1`+SignExt(`imm`) (áp dụng cho các lệnh Load và Store).
4. **Định địa chỉ tương đối với PC (PC-relative Addressing):** Địa chỉ của lệnh đích cần nhảy tới bằng tổng giá trị thanh ghi `PC` (địa chỉ của chính lệnh rẽ nhánh/nhảy hiện tại) cộng với hằng số độ lệch:

   $$\text{Địa chỉ đích (BTA hoặc JTA)} = PC_{\text{hiện tại}} + \text{imm} \iff \text{imm} = \text{Địa chỉ đích} - PC_{\text{hiện tại}}$$

3.2 Phần 2: Toàn bộ bài tập Chương 3 và Hướng dẫn giải

3.2.1 Nhóm 1: Thanh ghi, Khuôn dạng lệnh, Phương thức định địa chỉ và Kiểm tra hợp lệ

Bài tập 3.1 (Đề Cuối kỳ 2023.1 – #08de08, 2023.2 & Ôn tập Cuối kỳ 2025.1, 2025.2 – Câu 53)
Phát biểu nào sau đây là đúng về các thanh ghi tạm thời (`t0 - t6`) và các thanh ghi lưu trữ (`s0 - s11`) trong RISC-V?

A. Giá trị của các thanh ghi lưu trữ luôn phải được lưu bởi bên được gọi (callee).

B. Giá trị của các thanh ghi tạm thời luôn phải được lưu bởi bên gọi (caller).

C. Trong một số quy ước gọi hàm, thanh ghi tạm thời là "caller-saved", còn thanh ghi lưu trữ là "callee-saved".

D. Trong một số quy ước gọi hàm, thanh ghi tạm thời là "callee-saved", còn thanh ghi lưu trữ là "caller-saved".

Hướng dẫn giải:

- **Kiến thức áp dụng:** Theo quy ước gọi hàm của RISC-V, các thanh ghi tạm thời (`t0 - t6`) thuộc nhóm **caller-saved** (bên gọi chỉ cất vào ngăn xếp nếu cần dùng lại sau lời gọi hàm, không phải "luôn luôn" phải cất), còn các thanh ghi lưu trữ (`s0 - s11`) thuộc nhóm **callee-saved** (bên được gọi chỉ cất vào ngăn xếp nếu nó làm thay đổi các thanh ghi này).
- **Kết luận:** Đáp án đúng là **C**.

Bài tập 3.2 (Đề Giữa kỳ 2022.2 – Đề 0008 Câu 10 & Đề 0006 Câu 5)

a) Trong 32 thanh ghi được đánh địa chỉ của bộ xử lý 32-bit, tên của 1 thanh ghi chỉ đọc (read-only) luôn chứa hằng số 0 là gì?

b) Trong khuôn dạng lệnh R của bộ xử lý 32-bit, tại sao độ rộng của vùng thông tin thanh ghi đích `rd` phải là 5 bit?

Hướng dẫn giải:

- **Giải câu a:** Thanh ghi chỉ đọc luôn có giá trị bằng 0 là thanh ghi `zero` (hoặc `x0`).
- **Giải câu b:** Vì bộ xử lý có tất cả 32 thanh ghi đa dụng. Để mã hóa phân biệt 32 thanh ghi khác nhau cần số bit là $\log_2(32) = \mathbf{5}$ bit (vì $2^5 = 32$).

Bài tập 3.3 (Đề Giữa kỳ 2022.2 – Đề 0006, Câu 8)
Một phần mềm viết cho bộ xử lý RISC-V 32-bit có 150 lệnh hợp ngữ chuẩn (không dùng lệnh giả và không dùng lệnh nén) thì chiếm bao nhiêu byte lệnh mã máy trong bộ nhớ?

Hướng dẫn giải:

- Mỗi lệnh mã máy chuẩn của RISC-V 32-bit có độ dài cố định là 32 bit = 4 byte.
- Vậy 150 lệnh chiếm dung lượng là: 150×4 byte = **600** byte.

Bài tập 3.4 (Đề Giữa kỳ 2019.2 – Mã đề 243 Câu 17 & Mã đề 244 Câu 10)

a) **(Mã đề 243):** Một chương trình hợp ngữ có 8 lệnh `addi`, 5 lệnh `sub`, 1 lệnh `lui`, 3 lệnh `xori`. Hỏi mã máy của phần mềm trên chiếm bao nhiêu byte dung lượng?
A. 18 B. 72 C. 17 D. 68

b) **(Mã đề 244):** Một chương trình hợp ngữ có 5 lệnh `addi`, 2 lệnh `sub`, 3 lệnh `lui`, 8 lệnh `xori`. Hỏi mã máy của phần mềm trên chiếm bao nhiêu byte dung lượng?
A. 72 B. 68 C. 18 D. 17

Hướng dẫn giải:

- **Giải câu a:** Tổng số lệnh máy = $8 + 5 + 1 + 3 = 17$ lệnh. Mỗi lệnh dài 4 byte nên dung lượng mã máy là: $17 \times 4 = 68$ byte → Chọn **D (68)**.
- **Giải câu b:** Tổng số lệnh máy = $5 + 2 + 3 + 8 = 18$ lệnh. Dung lượng mã máy là: $18 \times 4 = 72$ byte → Chọn **A (72)**.

Bài tập 3.5 (Đề Giữa kỳ 2019.2 – Mã đề 243 Câu 20 & Mã đề 244 Câu 2)
Đoạn hợp ngữ gồm 3 lệnh chuẩn liên tiếp (`lui s0, ...` → `addi s0, s0, ...` → `lw t0, 4(s0)`).

a) **(Mã đề 243):** Nếu lệnh đầu tiên ở địa chỉ `0xBCDE8004`, hỏi lệnh cuối cùng (lệnh thứ 3) ở địa chỉ nào?
A. Không xác định B. `0xBCDE8006` C. `0x20201238` D. `0xBCDE800C`

b) **(Mã đề 244):** Nếu lệnh đầu tiên ở địa chỉ `0x8004FEDC`, hỏi lệnh cuối cùng (lệnh thứ 3) ở địa chỉ nào?
A. Không xác định B. `0x8004FEE4` C. `0x8004FEDA` D. `0x80004000`

Hướng dẫn giải:

- Mỗi lệnh dài 4 byte. Lệnh 1 ở địa chỉ A, lệnh 2 ở $A + 4$, lệnh 3 ở địa chỉ $A + 8$:
- **Câu a:** `0xBCDE8004` + 8 = `0xBCDE800C` → Chọn **D**.
- **Câu b:** `0x8004FEDC` + 8 = `0x8004FEE4` (vì `0xDC` + 8 = 220 + 8 = 228 = `0xE4`) → Chọn **B**.

Bài tập 3.6 (Đề Cuối kỳ 2022.2 – #17e1a7, 2024.1 – #366b03, 2024.2 & Ôn tập

2025.2 – Câu 34)
Phương pháp định địa chỉ nào được sử dụng mã hóa cho toán hạng hằng số của lệnh hợp ngữ RISC-V sau: `addi s0, s3, -5` (hoặc `addi x18, x19, -5`)?

A. Định địa chỉ cơ sở

B. Định địa chỉ tương đối với PC

C. Định địa chỉ tức thì

D. Định địa chỉ thanh ghi

Hướng dẫn giải:

- Toán hạng hằng số `-5` nằm trực tiếp ngay trong mã máy của câu lệnh `addi`, do đó sử dụng phương pháp **Định địa chỉ tức thì (Immediate Addressing)**.
- **Kết luận:** Đáp án đúng là **C**.

Bài tập 3.7 (Đề Giữa kỳ 2018.2 – Mã đề 198 Câu 19, Mã đề 199 Câu 13 & Mã đề 204 Câu 15)
Trong các phương pháp định địa chỉ toán hạng dưới đây, phương pháp nào có tốc độ thực thi lệnh chậm hơn?

A. Định địa chỉ thanh ghi

B. Định địa chỉ trực tiếp / Định địa chỉ cơ sở (truy cập bộ nhớ)

C. Định địa chỉ tức thì

D. Định địa chỉ trực tiếp và định địa chỉ tức thì

Hướng dẫn giải:

- Định địa chỉ tức thì (lấy hằng số có sẵn trong lệnh tại IR) và định địa chỉ thanh ghi (đọc thanh ghi trong CPU) không cần truy cập ra ngoài CPU nên tốc độ rất nhanh.
- Định địa chỉ trực tiếp / cơ sở phải truy xuất toán hạng nằm trong bộ nhớ chính (RAM) qua Bus hệ thống nên có tốc độ thực thi chậm hơn.
- **Kết luận:** Đáp án đúng là **B**.

Bài tập 3.8 (Đề Cuối kỳ 2020.2 – Trang 5 & Cuối kỳ 2022.2, 2024.2 – #3ace1f)

a) **(Đề CK 2020.2):** Trong các giá trị địa chỉ bộ nhớ hệ thống dưới đây, giá trị nào là địa chỉ hợp lệ và căn chỉnh đúng biên (aligned address) của một từ nhớ (word) dữ liệu 32-bit?
A. `0x1001A00C` B. `0x10800016` C. `0x100020CE` D. `0x1050009A`

b) **(Đề CK 2022.2 & 2024.2 – #3ace1f):** Trong các giá trị địa chỉ dưới đây, giá trị nào có thể là địa chỉ của word nhớ trong bộ nhớ chính của máy tính RISC-V?

A. `0x00404508` B. `0x0050601A` C. `0x00601002` D. `0x00402346`

c) **(Đề GK 2018.2 – Mã đề 198 Câu 9):** Địa chỉ nào sau đây **không phải** là địa chỉ hợp lệ của một lệnh máy 32-bit?
A. `0x188A` B. `0xFB019AC` C. `0x18D88` D. `0x198C124`

Hướng dẫn giải:

- **Quy tắc:** Địa chỉ của một word 32-bit hoặc một lệnh máy 32-bit bắt buộc phải **chia hết cho 4**, nghĩa là chữ số Hexa cuối cùng bên phải chỉ được phép là `0, 4, 8` **hoặc** `C` **(= 12)**. Các số tận cùng bằng `2, 6, A` (= 10), `E` (= 14) là số chẵn nhưng không chia hết cho 4.
- **Câu a:** Chỉ có `0x1001A00C` tận cùng bằng `C` chia hết cho 4 → Chọn **A**.
- **Câu b:** Chỉ có `0x00404508` tận cùng bằng `8` chia hết cho 4 → Chọn **A**.
- **Câu c:** Chỉ có `0x188A` tận cùng bằng `A` (10) không chia hết cho 4 → Chọn **A**.

Bài tập 3.9 (Đề Cuối kỳ 2023.1 – Trang 6 & Ôn tập Cuối kỳ 2025.2 – P1, Câu 28)
Toán hạng địa chỉ đích trong câu lệnh lưu trữ từ nhớ `sw` dưới đây của hệ RISC-V có hợp lệ để thực thi hay không?

```
lui  s0, 0x10010
addi t0, zero, 100
sw   t0, 2(s0)
```

A. Có, hoàn toàn hợp lệ.

B. Có, nhưng đoạn mã này yêu cầu quyền đặc biệt để thực thi.

C. Không, nó không có một địa chỉ hợp lệ (do vi phạm căn lề ranh giới word nhớ – alignment error).

D. Không, nó không nằm trong phân đoạn dữ liệu hợp lệ.

Hướng dẫn giải:

- Lệnh `lui s0, 0x10010` nạp địa chỉ `0x10010000` vào `s0`.
- Lệnh `sw t0, 2(s0)` ghi một word 32-bit (4 byte) vào địa chỉ `0x10010000`+2 = `0x10010002`.
- Địa chỉ `0x10010002` không chia hết cho 4, vi phạm quy tắc căn chỉnh biên word nhớ.
- **Kết luận:** Đáp án đúng là **C**.

Bài tập 3.10 (Đề Cuối kỳ 2020.2 – Trang 2, Trang 8 & Đề Giữa kỳ 2018.2 – Mã đề 204 Câu 7)

Hãy xác định tổng số lần CPU trực tiếp truy nhập bộ nhớ dữ liệu (Data Memory) trong mỗi đoạn chương trình sau:

a) **(Đề CK 2020.2 – Trang 2):** `lw t0, 0(s0)` → `addi t1, t0, 1` → `sw t1, 4(s0)` → `add t2, t1, t0`.

b) **(Đề CK 2020.2 – Trang 8):** `lw a1, 0(a2)` → `addi a1, a1, -4` → `slli a1, a1, 0` → `sw zero, 8(a1)`.

c) **(Đề GK 2018.2 – Mã đề 204, Câu 7):** `lui t0, 0x2019` → `addi t0, t0, 0x104` → `slli t1, t0, 3` → `sub t2, t1, t3`.

Hướng dẫn giải:

- Chỉ có các lệnh Load (`lw, lh, lb...`) và Store (`sw, sh, sb`) mới truy nhập vào bộ nhớ dữ liệu (Data Memory).
- **Câu a:** Có 1 lệnh `lw` và 1 lệnh `sw` $\implies$ Truy nhập bộ nhớ dữ liệu **2 lần**.
- **Câu b:** Có 1 lệnh `lw` và 1 lệnh `sw` $\implies$ Truy nhập bộ nhớ dữ liệu **2 lần**.
- **Câu c:** Không có lệnh Load/Store nào $\implies$ Truy nhập bộ nhớ dữ liệu **0 lần**.

Bài tập 3.11 (Ôn tập Cuối kỳ 2025.2 – P1, Câu 18)
Phát biểu nào sau đây là đúng về thứ tự lưu trữ các byte trong bộ nhớ chính?

A. Intel x86 dùng big-endian

B. SunSPARC dùng middle-endian

C. ARM dùng little-endian hoặc big-endian tùy phiên bản

D. RISC-V dùng big-endian

Hướng dẫn giải:

- Theo Slide Chương 3 (trang 16): Intel x86 và RISC-V dùng little-endian; Motorola 680x0 và SunSPARC dùng big-endian; còn kiến trúc **ARM** dùng little-endian hoặc big-endian tùy phiên bản.
- **Kết luận:** Đáp án đúng là **C**.

Bài tập 3.12 (Đề Cuối kỳ 2025.1 – #9853fd & Ôn tập Cuối kỳ 2025.2 – P1, Câu 42)

a) **(Đề CK 2025.1 – #9853fd):** Phạm vi giá trị của toán hạng hằng số (immediate) trong lệnh `addi` của RISC-V là gì?
A. $[-4095, +4095]$ B. $[-2^{31}, +2^{31}-1]$ C. $[-2048, +2048]$ D. $[-2048, +2047]$

b) **(Ôn tập 2025.2 – P1, Câu 42):** Những câu lệnh nào sau đây bị lỗi biên dịch trong phần mềm RARS?

1. `lui x1, 0x1` → `addi x1, x0, -2048` 2. `lui x1, 0x1` → `addi x1, x0, 0x800`
3. `lui x1, 0x1` → `addi x1, x0, 0xFFFF800` 4. `addi x1, x0, 0x801`
5. `li x1, 0x800` 6. `lui x1, 0x1` → `addi x1, x0, 2048`
7. `lui x1, 0x1` → `addi x1, x0, 0xFFFFF800`

Hướng dẫn giải:

- **Giải câu a:** Trường immediate của lệnh `addi` là số nguyên có dấu 12 bit, có dải biểu diễn từ -2^{11} đến $+2^{11}-1$, tức là $[-\mathbf{2048}, +\mathbf{2047}]$ → Chọn **D**.
- **Giải câu b:** Mọi giá trị truyền vào `addi` nằm ngoài đoạn $[-2048, +2047]$ (tức ngoài đoạn `0xFFFFF800` đến `0x7FF`) đều báo lỗi biên dịch:
  - Mục **2** (`0x800` = 2048), mục **3** (`0xFFFF800` chỉ có 4 chữ `F` nên là số dương lớn), mục **4** (`0x801` = 2049) và mục **6** (2048) đều vượt quá +2047 $\implies$ **Bị lỗi biên dịch (Chọn 2, 3, 4, 6)**.

Bài tập 3.13 (Đề Giữa kỳ 2021.1 – IT3030E, Problem 6)
Các lệnh chuẩn của RISC-V/MIPS dài 32 bit. Một cách để tiết kiệm bộ nhớ chương trình là giảm độ dài lệnh xuống 16 bit như tập lệnh nén (RISC-V Compressed RVC hoặc ARM Thumb). Hãy nêu ngắn gọn những hạn chế (limitations) tất yếu của các lệnh 16-bit này?

Hướng dẫn giải:

- Khi thu gọn độ dài mã lệnh từ 32 bit xuống còn 16 bit, số bit dành cho các trường bị cắt giảm mạnh dẫn đến 4 hạn chế chính:
  1. **Hạn chế số lượng thanh ghi truy cập được:** Thay vì dùng 5 bit để truy cập cả 32 thanh ghi, nhiều lệnh 16-bit chỉ dùng 3 bit nên chỉ truy cập được 8 thanh ghi thông dụng (`x8 - x15`).
  2. **Giảm số địa chỉ toán hạng trong lệnh:** Chuyển từ cấu trúc 3 toán hạng (`add rd, rs1, rs2`) sang 2 toán hạng (`c.add rd, rs2`, trong đó `rd` vừa là nguồn vừa là đích), làm tăng số lệnh cần viết nếu muốn giữ nguyên giá trị thanh ghi nguồn.
  3. **Thu hẹp phạm vi hằng số tức thời (Immediate):** Hằng số chỉ còn từ 5 đến 6 bit thay vì 12 bit hay 20 bit.
  4. **Thu hẹp khoảng cách nhảy và rẽ nhánh:** Độ lệch nhảy tương đối với PC ngắn hơn nhiều so với lệnh 32-bit.

3.2.2 Nhóm 2: Thực thi lệnh Số học, Logic, Dịch bit và Khởi tạo thanh ghi

Bài tập 3.14 (Đề Cuối kỳ 2020.2 – Trang 4 & Đề Cuối kỳ 2021.1 – Trang 3)
Xác định nội dung giá trị số Hexa lưu trữ tại thanh ghi `t1` sau khi thực hiện đoạn lệnh hợp

ngữ tính toán cơ bản dưới đây:

```
addi s5, zero, 199    # 0xC7 tuong ung gia tri thap phan 199
addi s6, zero, 229
add  t1, s6, s5
```

Hướng dẫn giải:

- Thực hiện phép cộng trong hệ thập phân: $\texttt{t1} = 229 + 199 = 428_{(10)}$.
- Đổi $428_{(10)}$ sang hệ Hexa:

$$428 = 256 + 172 = 1 \times 16^2 + 10 \times 16^1 + 12 \times 16^0 = \texttt{0x1AC}$$

- Viết đầy đủ 8 chữ số Hexa (32-bit): `0x000001AC` (hoặc `0x1AC`).

Bài tập 3.15 (Đề Cuối kỳ 2022.2 & Cuối kỳ 2024.2 – #014753)
Cho đoạn chương trình hợp ngữ RISC-V sau (giả sử ban đầu $\texttt{s0} = 5$ và $\texttt{s1} = 4$, hoặc $\texttt{s0} = 7, \texttt{s1} = 2$):

```
add  t0, s0, s1
sub  t1, s0, s1
addi t2, t0, 10
addi t3, t1, -5
```

Hãy xác định giá trị của `t2` và `t3` sau khi thực hiện đoạn chương trình:

A. $\texttt{t2} = 19, \texttt{t3} = -3$ (khi $\texttt{s0} = 5.5$? Không, ứng với $\texttt{s0} + \texttt{s1} = 9$ và $\texttt{s0} - \texttt{s1} = 2$: không nguyên) $\rightarrow$ Kiểm tra với $\texttt{s0} = 7, \texttt{s1} = 2$: $\texttt{t0} = 9 \implies \texttt{t2} = 19$; $\texttt{t1} = 5 \implies \texttt{t3} = 0$.

B. $\texttt{t2} = 19, \texttt{t3} = 0$

C. $\texttt{t2} = 15, \texttt{t3} = -5$

D. $\texttt{t2} = 24, \texttt{t3} = -8$

Hướng dẫn giải:

- Ta có hệ phương trình giữa `t2` và `t3`:

$$\begin{cases} \texttt{t2} = (\texttt{s0} + \texttt{s1}) + 10 \\ \texttt{t3} = (\texttt{s0} - \texttt{s1}) - 5 \end{cases} \implies \texttt{t2} + \texttt{t3} = 2 \times \texttt{s0} + 5$$

- Vì `s0` là số nguyên nên $2 \times \texttt{s0} + 5$ bắt buộc phải là một **số lẻ** (tức tổng $\texttt{t2} + \texttt{t3}$ phải là số lẻ).
- Kiểm tra 4 đáp án:

  - Đáp án A: $19 + (-3) = 16$ (chẵn – loại).
  - Đáp án C: $15 + (-5) = 10$ (chẵn – loại).
  - Đáp án D: $24 + (-8) = 16$ (chẵn – loại).
  - Đáp án B: $19 + 0 = 19$ (lẻ, ứng với $\texttt{s0} = 7, \texttt{s1} = 2$: $\texttt{t0} = 7 + 2 = 9 \implies \texttt{t2} = 19$; $\texttt{t1} = 7 - 2 = 5 \implies \texttt{t3} = 5 - 5 = 0$).

- **Kết luận:** Đáp án đúng là **B** ($\texttt{t2} = 19, \texttt{t3} = 0$).

Bài tập 3.16 (Đề Giữa kỳ 2024.1 – #4610c2, Ôn tập 2025.2 – Câu 75 & Đề Giữa kỳ 2018.2 – Mã đề 198 Câu 18)

a) **(Đề GK 2024.1 – #4610c2 & Ôn tập 2025.2 Câu 75):** Những lệnh nào sau đây có thể dùng để xóa nội dung của thanh ghi `s0` về 0?
1. `and s0, s0, zero` 2. `xor s0, s0, s0` 3. `xor s0, s0, zero` 4. `and s0, s0, s0`

b) **(Đề GK 2018.2 – Mã đề 198, Câu 18):** Trong hai lệnh `add s4, s4, s4` và `slli s4, s4, 1`, chúng cùng thực hiện phép toán nào? Hai lệnh `mul s4, s4, s4` và `srl s4, s4, s4` có tương đương với hai lệnh trên không?

Hướng dẫn giải:

- **Giải câu a:**
  - `and s0, s0, zero`: `s0` & $0 = 0$ (Đúng).
  - `xor s0, s0, s0`: Hai số giống nhau XOR với nhau luôn bằng 0 (Đúng).
  - `xor s0, s0, zero` và `and s0, s0, s0` đều giữ nguyên giá trị `s0`. Vậy chọn **1 và 2**.
- **Giải câu b:** Lệnh `add s4, s4, s4` ($\texttt{s4} + \texttt{s4} = 2 \times \texttt{s4}$) và lệnh `slli s4, s4, 1` (dịch trái 1 bit $= 2 \times \texttt{s4}$) tương đương nhau; còn lệnh nhân bình phương `mul s4, s4, s4` và dịch phải `srl` không tương đương.

Bài tập 3.17 (Đề Giữa kỳ 2024.1 – #99ef6, #e10492 & Ôn tập 2025.2 – Câu 6)

a) **(Câu #99ef6 & Câu 6):** Đoạn chương trình nào tương đương với lệnh giả `li t0, 0x20241120`?
A. `lui t0, 0x202411` → `ori t0, t0, 0x20` B. `lui t0, 0x2024` → `addi t0, t0, 0x1120`
C. `lui t0, 0x2024` → `ori t0, t0, 0x1120` D. `lui t0, 0x20241` → `addi t0, t0, 0x120`

b) **(Câu #e10492):** Chọn đoạn lệnh RISC-V thực hiện câu lệnh C: `a += 0x12345678;` (với biến `a` nằm trong thanh ghi `s0`).

Hướng dẫn giải:

- **Giải câu a:** Số 32-bit `0x20241120` có 20 bit cao (5 chữ Hexa đầu) là `0x20241` và 12 bit thấp (3 chữ Hexa cuối) là `0x120` (dương). Do đó dùng: `lui t0, 0x20241` → `addi t0, t0, 0x120` → Chọn **D**.
- **Giải câu b:** Đầu tiên nạp hằng số `0x12345678` vào thanh ghi tạm `t0` bằng 2 lệnh `lui t0, 0x12345` và `addi t0, t0, 0x678`, sau đó cộng dồn vào `s0` bằng lệnh `add s0, s0, t0`.

Bài tập 3.18 (Đề Giữa kỳ 2022.2 – Đề 0009, 0008, 0007, 0006 Câu 9)
Hãy lập trình hợp ngữ chuẩn RISC-V (không dùng lệnh giả `li`) để thực hiện các phép gán hằng số vào thanh ghi dưới đây:

a) **(Đề 0008):** `t1` = `0x234` và `t2` = `0x56789`.

b) **(Đề 0009):** `t1` = `0xE7F` và `t2` = `0x11062023`.

c) **(Đề 0006):** `s5` = `0xCA03CA06` và `s6` = `0x2B1A`.

Hướng dẫn giải:

- **Giải câu a:**
  - Với `t1` = `0x234` (nhỏ hơn `0x800`, nằm gọn trong 12-bit dương): `ori t1, x0, 0x234` (hoặc `addi t1, x0, 0x234`).
  - Với `t2` = `0x00056789` (5 chữ đầu là `0x56`, 3 chữ cuối là `0x789` < `0x800`): `lui t2, 0x56` → `addi t2, t2, 0x789`.
- **Giải câu b:**
  - Với `t1` = `0x00000E7F`: Vì chữ Hexa thứ 3 là `E` ≥ 8 (nếu dùng trực tiếp `addi/ori 0xE7F` sẽ bị mở rộng dấu âm thành `0xFFFFFE7F`), nên ta phải nạp `lui t1, 1` (`0x00001000`) rồi cộng số âm `0xE7F` − `0x1000` = −385: `lui t1, 1` → `addi t1, t1, -385`.
  - Với `t2` = `0x11062023` (3 chữ cuối là `0x023` < `0x800`): `lui t2, 0x11062` → `addi t2, t2, 0x023`.
- **Giải câu c:**
  - Với `s5` = `0xCA03CA06`: Chữ số thứ 3 từ phải sang là `A` ≥ 8 (`0xA06` = 2566 − 4096 = −1530). Phải tăng 5 chữ đầu từ `0xCA03C` lên `0xCA03D`: `lui s5, 0xCA03D` → `addi s5, s5, -1530`.
  - Với `s6` = `0x00002B1A`: Chữ thứ 3 từ phải sang là `B` ≥ 8 (`0xB1A` = 2842 − 4096 = −1254). Tăng 5 chữ đầu từ `0x2` lên `0x3`: `lui s6, 0x3` → `addi s6, s6, -1254`.

Bài tập 3.19 (Đề Giữa kỳ 2024.1 – #1b55cf & Ôn tập Cuối kỳ 2025.2 – P1, Câu 24)
Cho đoạn mã hợp ngữ RISC-V dưới đây. Hãy xác định giá trị của thanh ghi `s2` ở dạng số Hexa sau khi thực thi:

```
li   s0, 1
lui  s1, 0xFFFFF
ori  s1, s1, 2047
add  s2, s0, s1
```

Hướng dẫn giải:

- Lệnh 1: `s0` = `0x00000001`.
- Lệnh 2: `s1` = `0xFFFFF000`.
- Lệnh 3: Vì 2047 = `0x7FF` (bit 11 bằng 0, mở rộng 32-bit là `0x000007FF`), nên sau phép OR ta có `s1` = `0xFFFFF7FF`.
- Lệnh 4: `s2` = `0xFFFFF7FF` + 1 = `0xFFFFF800` (Nếu đề bài viết `lui s1, 0xFFFF` gồm 4 chữ `F` thì kết quả là `0x0FFFF800`).

Bài tập 3.20 (Đề Giữa kỳ 2024.1 – #5e7591, Cuối kỳ 2025.1 – #48b64c & Ôn tập 2025.2 – Câu 64)

a) **(Đề GK 2024.1 – #5e7591 & Ôn tập 2025.2 Câu 64):** Tính giá trị thập phân của `s4` sau đoạn lệnh: `addi s1, zero, 87` → `addi s2, zero, 53` → `xor s3, s2, zero` → `addi s3, s3, 1` → `add s4, s1, s3`.

b) **(Đề CK 2025.1 – #48b64c):** Tính giá trị thập phân của `s4` sau đoạn lệnh: `addi s1, zero, 80` → `addi s2, zero, 56` → `xor s3, s2, zero` → `addi s3, s3, 1` → `add s4, s1, s3`.

Hướng dẫn giải:

- Phép toán `xor s3, s2, zero` giữ nguyên giá trị của `s2` (`s3` = `s2`).
- **Câu a:** `s3` = 53 + 1 = 54 $\implies$ `s4` = 87 + 54 = **141**.
- **Câu b:** `s3` = 56 + 1 = 57 $\implies$ `s4` = 80 + 57 = **137**.

Bài tập 3.21 (Đề Giữa kỳ 2018.2 – Mã đề 204, Câu 9)
Cho biết giá trị của thanh ghi `s1` sau khi thực hiện xong đoạn lệnh sau:

```
addi t1, x0, -1
addi t2, zero, 5
or   s1, t1, t2
```

A. 5

B. `0xFFFFFFFF`

C. -5

D. 1

Hướng dẫn giải:

- Số nguyên âm -1 trong biểu diễn bù 2 32-bit gồm toàn bộ 32 bit bằng 1: `t1` = `0xFFFFFFFF`.
- Khi thực hiện phép OR với `0xFFFFFFFF`, tất cả các bit của kết quả đều bằng 1: `s1` = `0xFFFFFFFF`.
- **Kết luận:** Đáp án đúng là **B** (`0xFFFFFFFF`).

Bài tập 3.22 (Đề Cuối kỳ 2022.2 – #585834, 2024.1 – #d9f3db, 2024.2 & Ôn tập 2025.2 – Câu 54)

a) **(Đề CK 2024.1 – #d9f3db & Ôn tập 2025.2 Câu 54):** Cho biết nội dung thanh ghi `s1` theo số Hexa sau đoạn lệnh: `lui s0, 0x11223` → `ori s0, s0, 0x4AB` → `ori s1, s0, 0xFF`.

b) **(Đề CK 2022.2 & 2024.2 – #585834):** Nếu sau hai lệnh đầu thanh ghi `s0` = `0x223389CD`, thực hiện lệnh thứ ba là `andi s1, s0, 0xFF` thì nội dung thanh ghi `s1` bằng bao nhiêu?

Hướng dẫn giải:

- **Giải câu a:**
  - `lui s0, 0x11223` $\Longrightarrow$ `s0` = `0x11223000`.
  - `ori s0, s0, 0x4AB` $\Longrightarrow$ `s0` = `0x112234AB`.
  - `ori s1, s0, 0xFF`: Phép OR với `0x000000FF` giữ nguyên 6 chữ Hexa đầu và bật 2 chữ cuối thành `FF` $\Longrightarrow$ `s1 = 0x112234FF`.
- **Giải câu b:** Lệnh `andi s1, s0, 0xFF` thực hiện phép AND với `0x000000FF`, xóa 6 chữ số Hexa đầu về `0` và giữ lại 2 chữ số cuối: `s1 = 0x000000CD`.

Bài tập 3.23 (Đề Cuối kỳ 2020.2, 2022.2 – #5070f9, 2024.2, Giữa kỳ 2023.2 & Giữa kỳ 2025.2 – Đề 0012 Câu 108)

Hãy xác định kết quả dạng Hexa cho các bài tập phép toán logic dưới đây:

a) **(Đề CK 2020.2 – Trang 3):** Cho `s0` = `0x1234ABCD` và `s1` = `0xFFFF0000`. Tìm `s2` sau lệnh `xor s2, s0, s1`.

b) **(Đề CK 2020.2 – Trang 7):** Cho `t0` = `0xAAAAAAAA` và `t1` = `0x0000FFC0`. Tìm `t3` sau lệnh `and t3, t0, t1`.

c) **(Đề CK 2022.2, 2024.2 – #5070f9 & Ôn tập 2025.2 Câu 64):** Cho `t0` = `0x5A6B7C8D` và `t1` = `0x000FFF00`. Tìm `t2, t3` sau 2 lệnh: `xor t2, t0, t1` và `xor t3, t2, t1`.

d) **(Đề GK 2025.2 – Đề 0012, Câu 108):** Cho `t0` = `0x35B9D2E4` và `t1` = `0x0F0F0F0F`. Tìm `t2` sau lệnh `xor t2, t0, t1`.

e) **(Đề GK 2023.2 – Đề 2, Câu 4):** Cho `t0` = `0xA1B2C3D4`. Viết một lệnh logic để biến đổi `t0` thành `0x286CB0F5` (giả sử thanh ghi mặt nạ `t1` đã được nạp sẵn hằng số cần thiết).

Hướng dẫn giải:

- **Giải câu a:** XOR với `0xFFFF0000` sẽ đảo 4 chữ số đầu `1234` (lấy 15 trừ từng chữ số: $15-1=$ `E`, $15-2=$ `D`, $15-3=$ `C`, $15-4=$ `B`) và giữ nguyên 4 chữ cuối `ABCD` $\Longrightarrow$ `s2 = 0xEDCBABCD`.
- **Giải câu b:** Trong `t0` = `0xAAAAAAAA` (`A` = 1010_2) và `t1` = `0x0000FFC0` (`C` = 1100_2), lấy từng chữ số AND với nhau: 1010_2 & $1100_2 = 1000_2 =$ `8` $\Longrightarrow$ `t3 = 0x0000AA80`.
- **Giải câu c:** Đảo 3 chữ giữa `B7C` của `0x5A6B7C8D` được `t2 = 0x5A64838D`. Khi lấy `t2` XOR tiếp với `t1` sẽ hoàn nguyên về giá trị của `t0`: `t3 = 0x5A6B7C8D`.
- **Giải câu d:** Đảo các chữ số ở vị trí chẵn (`5, 9, 2, 4` $\rightarrow$ `A, 6, D, B`) và giữ nguyên các chữ số ở vị trí lẻ (`3, B, D, E`) $\Longrightarrow$ `t2 = 0x3AB6DDEB`.
- **Giải câu e:** Lấy `0xA1B2C3D4` $\oplus$ `0x286CB0F5` = `0x89DE7321`. Với `t1` = `0x89DE7321`, lệnh cần viết là: `xor t0, t0, t1`.

Bài tập 3.24 (Đề Giữa kỳ 2022.2 – Đề 0008 Câu 5 & Đề 0007 Câu 5)
Thực hiện các phép toán logic 32-bit sau và biểu diễn kết quả ở hệ 16 (Hexa):

a) `0xB8C32AC9 and 0x00FFFF00` và `0xCF072D76 or 0xF0F0F0F0`.

b) `0x0000FFFF and 0x079CC0D1` và `0xAC4F0130 or 0xFFFF0000`.

Hướng dẫn giải:

- **Giải câu a:**
  - `0xB8C32AC9 and 0x00FFFF00` = `0x00C32A00`.
  - `0xCF072D76 or 0xF0F0F0F0` = `0xFFF7FDF6`.
- **Giải câu b:**
  - `0x0000FFFF and 0x079CC0D1` = `0x0000C0D1`.
  - `0xAC4F0130 or 0xFFFF0000` = `0xFFFF0130`.

Bài tập 3.25 (Đề Cuối kỳ 2020.2 – Trang 6 & Đề Cuối kỳ 2021.1 – Trang 5)
Cho đoạn mã nguồn C thực hiện thao tác xử lý trên bit: `X = (Y & 0x0F) << 4;`. Giả thiết hai biến số nguyên `X` và `Y` lần lượt được phân bổ tại các thanh ghi `s2` và `s1`. Hãy chỉ ra chuỗi lệnh hợp ngữ RISC-V dịch chính xác câu lệnh C trên?

A. `and t1, s1, 0xF` → `slli s2, t1, 16`

B. `andi t1, s1, 15` → `slli s2, t1, 4`

C. `andi t1, s1, 0xFFFF` → `slli s2, t1, 2`

D. `slli t1, s1, 15` → `andi s2, t1, 4`

Hướng dẫn giải:

- Bước 1: Tính biểu thức trong ngoặc `Y & 0x0F`. Vì `0x0F` = $15_{(10)}$ là hằng số tức thời nên phải dùng lệnh `andi`: `andi t1, s1, 15`.
- Bước 2: Dịch trái kết quả đi 4 bit (`« 4`) và lưu vào `s2` (`X`) bằng lệnh `slli`: `slli s2, t1, 4`.
- **Kết luận:** Đáp án đúng là **B**.

Bài tập 3.26 (Đề Giữa kỳ 2023.2 – Đề 1 Câu 4 & Đề Cuối kỳ 2021.2 – Mã đề B8 Câu 2.2)

a) **(Đề GK 2023.2):** Cho `t0` = `0xAAAAAAAA`, `t1` = `0x00FF0000`. Tìm giá trị Hexa của `t2` và `t3` sau hai lệnh: `srli t2, t0, 3` và `or t3, t2, t1`.

b) **(Đề CK 2021.2 – Mã đề B8):** Cho `t0` = `0xBEADFEED`, `t1` = `0xDEADFADE`. Tìm giá trị Hexa của `t2` sau hai lệnh: `slli t2, t0, 4` và `or t2, t2, t1`.

Hướng dẫn giải:

- **Giải câu a:**
  - Đổi `t0` = `0xAAAAAAAA` = $1010\,1010\,\ldots\,1010_{(2)}$.
  - Khi dịch phải logic 3 bit (`srli t2, t0, 3`), 3 bit `000` chèn vào bên trái ghép với bit `1` đầu tiên thành `0001` (`1`), các nhóm 4 bit tiếp theo đều trở thành `0101` (`5`) $\Longrightarrow$ `t2 = 0x15555555`.
  - Thực hiện `or t3, t2, t1` với `t1` = `0x00FF0000`: `t3 = 0x15FF5555`.
- **Giải câu b:**
  - `slli t2, t0, 4` (dịch trái 1 chữ số Hexa): `t2` = `0xEADFEED0`.
  - `or t2, t2, t1`: Lấy `0xEADFEED0` OR từng chữ số với `0xDEADFADE` được: `t2 = 0xFEFFFEFE`.

3.2.3 Nhóm 3: Lệnh So sánh (`slt, slti, sltu, blt, bltu`) và Rẽ nhánh

Bài tập 3.27 (Đề Cuối kỳ 2021.2 – Mã đề B8, Câu 2.1)
Cho câu lệnh hợp ngữ: `slti t2, t0, X`. Giả sử thanh ghi `t0` có giá trị `0xAD100002`. Với

những giá trị nào của hằng số `X` (viết dưới dạng thập phân) thì thanh ghi `t2` sẽ có giá trị bằng 1?

Hướng dẫn giải:

- Lệnh `slti` thực hiện so sánh số nguyên **có dấu** giữa thanh ghi `t0` (32-bit) và hằng số tức thời `X` (12-bit có dấu trong RISC-V, có dải giá trị hợp lệ là $[-2048, +2047]$).
- Xét `t0` = `0xAD100002`: Chữ số Hexa đầu tiên là `A` $= 1010_{(2)}$ có bit dấu bằng 1, nên `t0` là một số nguyên âm rất nhỏ ($\approx -1.39 \times 10^9$), luôn nhỏ hơn -2048.
- Do đó, với **mọi giá trị hợp lệ của hằng số `X` thuộc đoạn** $[-2048, +2047]$ (hoặc $[-32768, +32767]$ nếu xét chuẩn 16-bit), điều kiện `t0` $<$ `X` luôn luôn đúng và làm cho `t2` $= 1$.

Bài tập 3.28 (Đề Cuối kỳ 2025.1 – #a91b41, Giữa kỳ 2024.1 – #8d33e2, #484325 & Ôn tập 2025.2 – Câu 23, 61)

a) **(Đề CK 2025.1 – #a91b41):** Tìm giá trị thập phân của `t1` sau khi chạy đoạn mã:

```
    li   s1, -15
    li   s2, 17
    slt  t0, s1, s2
    bne  t0, zero, else
    addi t1, s1, 14
    j    endif
else:
    addi t1, s2, -1
endif: nop
```

b) **(Ôn tập 2025.2 – Câu 23):** Cho `x22` $= 10$, `x23` $= 5$, `x24` $= 0$. Chạy: `slt x24, x22, x23` → `bne x24, x0, L1` → `addi x24, x24, 10` → `L1: addi x24, x24, 1`. Tìm giá trị cuối của `x24`?

c) **(Ôn tập 2025.2 – Câu 61):** Cho `x5` = `0x0000000A` (10) và `x6` = `0x0000000A`. Chạy: `addi x7, x0, 100` → `bne x5, x6, Skip` → `addi x7, x7, 50` → `Skip: addi x7, x7, 1`. Tìm `x7`?

d) **(Đề GK 2024.1 – #8d33e2 & #484325):**

- *Bài 1:* `t0` $= -2$, `t1` $= 2$. Chạy `blt t0, t1, else`. Nhánh không nhảy tính `a0` $= 1+5$; nhánh `else` tính `a0` $= 2 + 7$. Tìm `a0`?
- *Bài 2:* `t0` $= 10$, `t1` $= 10$. Chạy `bltu t0, t1, else`. Nhánh không nhảy tính `a0` $= 10 + 5$; nhánh `else` tính `a0` $= 20 + 7$. Tìm `a0`?

Hướng dẫn giải:

- **Giải câu a:** Vì $-15 < 17$ là Đúng nên `t0` $= 1$. Lệnh `bne t0, zero, else` thỏa mãn (`t0` $\neq 0$) nên nhảy tới nhãn `else` thực hiện: `t1` $=$ `s2` $- 1 = 17 - 1 =$ **16**.
- **Giải câu b:** Vì $10 < 5$ là Sai nên `x24` $= 0$. Lệnh `bne` không nhảy, thực hiện cộng 10 (`x24` $= 10$) rồi trôi tiếp qua nhãn `L1` cộng thêm 1 $\implies$ `x24 = 11`.
- **Giải câu c:** Vì `x5` $==$ `x6` nên lệnh `bne` không nhảy. Thanh ghi `x7` nhận giá trị: $100 + 50 + 1 =$ **151**.
- **Giải câu d:**
  - *Bài 1:* $-2 < 2$ là Đúng $\implies$ Nhảy tới `else`: `a0` $= 2 + 7 =$ **9**.
  - *Bài 2:* $10 < 10$ là Sai $\implies$ Không nhảy: `a0` $= 10 + 5 =$ **15**.

3.2.4 Nhóm 4: Truy cập bộ nhớ (`lw, lh, lb, lbu, sw`) và Thứ tự Little-Endian

Bài tập 3.29 (Đề Cuối kỳ 2022.2 & Cuối kỳ 2024.2 – #56e53f)
Giả sử thanh ghi `s0` đang chứa giá trị địa chỉ `0x10021000`. Lựa chọn chức năng đúng của câu lệnh hợp ngữ `lw t0, 8(s0)` (hoặc `sw t0, 8(s0)`):

A. Ghi giá trị hằng số `0x10021008` vào thanh ghi `t0`.

B. Nạp nội dung word nhớ ở địa chỉ `0x10021008` đưa vào thanh ghi `t0`.

C. Lấy nội dung thanh ghi `t0` cộng thêm 8 rồi lưu ra word nhớ ở địa chỉ `0x10021000`.

D. Lưu nội dung thanh ghi `t0` ra word nhớ ở địa chỉ `0x10021008`.

Hướng dẫn giải:

- Địa chỉ hiệu dụng của toán hạng bộ nhớ là `s0` $+ 8 =$ `0x10021000` $+ 8 =$ `0x10021008`.
- Nếu đề bài hỏi lệnh `lw t0, 8(s0)` (load word): Chức năng là **B (Nạp nội dung word nhớ ở địa chỉ `0x10021008` đưa vào thanh ghi `t0`)**.
- Nếu đề bài hỏi lệnh `sw t0, 8(s0)` (store word): Chức năng là **D (Lưu nội dung thanh ghi `t0` ra word nhớ ở địa chỉ `0x10021008`)**.

Bài tập 3.30 (Đề Cuối kỳ 2025.1 – #1f833e)
Đoạn lệnh nào dưới đây dùng để nạp một word từ địa chỉ bộ nhớ `0x10010004` vào thanh ghi `t1` (mà không dùng lệnh giả)?

A. `li t0, 0x10010004` → `lw t1, zero(t0)`

B. `lui t0, 0x1001` → `addi t0, t0, 0x0004` → `lw t1, 0(t0)`

C. `lw t1, 0x10010004(zero)`

D. `lui t0, 0x10010` → `addi t0, t0, 0x004` → `lw t1, 0(t0)`

Hướng dẫn giải:

- Địa chỉ 32-bit `0x10010004` có 20 bit cao (5 chữ số Hexa đầu) là `0x10010` và 12 bit thấp (3 chữ số Hexa cuối) là `0x004`.
- Lệnh `lui t0, 0x10010` tạo ra `0x10010000`; lệnh `addi t0, t0, 0x004` cộng thêm 4 tạo thành địa chỉ `0x10010004`; sau đó dùng `lw t1, 0(t0)` đọc word nhớ lên `t1`. (Phương án B sai vì `0x1001` thiếu một chữ số 0).
- **Kết luận:** Đáp án đúng là **D**.

Bài tập 3.31 (Đề Giữa kỳ 2018.2 – Mã đề 198 Câu 20, Mã đề 199 Câu 6 & Mã đề 204 Câu 1)

Biến `X` thuộc kiểu word 32-bit nằm ở địa chỉ `0x0FB028C4` (hoặc `0xAFB028C4`). Đoạn chương trình nào sau đây tương đương với câu lệnh trong ngôn ngữ C: `X = X + 1;`?

A. `addi t3, x0, 0x0FB028C4` → `lw s3, 0(t3)` → `addi s3, s3, 1` → `sw s3, 0(t3)`

B. `lui t3, 0x0FB03` → `addi t3, t3, -1852` (*hoặc* ***lui*** *+* ***ori***) → `lw s3, 0(t3)` → `addi s3, s3, 1` → `sw s3, 0(t3)`

C. `lui t3, 0x28C4` → `ori t3, t3, 0x0FB0` → `lw s3, 0(t3)` → `addi s3, s3, 1` → `sw s3, 0(t3)`

D. `lui t3, 0x0FB03` → `addi t3, t3, -1852` → `sw s3, 0(t3)` → `addi s3, s3, 1` → `lw s3, 0(t3)`

Hướng dẫn giải:

- Quy trình chuẩn để tăng biến `X` trong bộ nhớ lên 1 đơn vị gồm 3 bước:
  1. Nạp phần cao của địa chỉ bằng `lui`, ghép phần thấp để tạo địa chỉ `0x0FB028C4` trong thanh ghi con trỏ `t3` (Loại A vì `addi` không nạp được hằng số 32-bit; loại C vì nạp ngược phần thấp lên phần cao).
  2. Đọc (`lw`) giá trị của `X` từ bộ nhớ vào thanh ghi `s3`, cộng thêm 1 (`addi s3, s3, 1`).
  3. Ghi (`sw`) kết quả từ `s3` trả lại bộ nhớ (Loại D vì thực hiện `sw` trước khi `lw`).
- **Kết luận:** Đáp án đúng là **B**.

Bài tập 3.32 (Đề Cuối kỳ 2022.2 – #757298, 2024.1 – #c99f87, 2024.2 & Ôn tập 2025.2 – Câu 20)

Biến `x` thuộc kiểu word 32-bit nằm ở địa chỉ `0x10010200`. Đoạn chương trình RISC-V nào dưới đây tương đương với câu lệnh ngôn ngữ C: `x = x & 15;`?

A. `andi t3, t3, 15`

B. `lui t3, 0x10200` → `addi t3, t3, 0x100` → `lw s3, 0(t3)` → `andi s3, s3, 15` → `sw s3, 0(t3)`

C. `lui t3, 0x10010` → `addi t3, t3, 0x200` → `lw s3, 0(t3)` → `andi s3, s3, 15` → `sw s3, 0(t3)`

D. `lui t3, 0x10010` → `addi t3, t3, 0x200` → `sw s3, 0(t3)` → `andi s3, s3, 15` → `lw s3, 0(t3)`

Hướng dẫn giải:

- Tạo địa chỉ `0x10010200` trong `t3` bằng cặp lệnh `lui t3, 0x10010` và `addi t3, t3, 0x200`.
- Đọc `x` từ bộ nhớ lên `s3` (`lw s3, 0(t3)`), thực hiện AND với 15 (`andi s3, s3, 15`), rồi ghi trả lại bộ nhớ (`sw s3, 0(t3)`).
- **Kết luận:** Đáp án đúng là **C**.

Bài tập 3.33 (Đề Giữa kỳ 2018.2 – Mã đề 198 Câu 11, Mã đề 204 Câu 12 & Đề Giữa kỳ 2019.2 – Mã đề 243 Câu 19, Mã đề 244 Câu 3)

a) **(Mã đề 243 – Câu 19):** Trong bộ nhớ chính có 4 byte theo thứ tự địa chỉ tăng dần là `1A, 2B, 3C, 4D` được lưu theo thứ tự đầu nhỏ (Little-Endian). Hỏi số nguyên 32-bit tương ứng bằng bao nhiêu?
A. `0x4321DCBA` B. `0x4D3C2B1A` C. `0x1A2B3C4D` D. `0xDCBA4321`

b) **(Mã đề 244 – Câu 3):** Trong bộ nhớ chính có 4 byte theo thứ tự địa chỉ tăng dần là `4D, 3C, 2B, 1A` được lưu theo thứ tự đầu nhỏ. Hỏi số nguyên 32-bit tương ứng bằng bao nhiêu?
A. `0x4D3C2B1A` B. `0x1A2B3C4D` C. `0x4321DCBA` D. `0xDCBA4321`

c) **(Mã đề 204 – Câu 12):** Số 32-bit `0xC81A56` (tức `0x00C81A56`) khi được lưu trữ vào bộ nhớ chính theo thứ tự đầu nhỏ thì trình tự các byte theo địa chỉ tăng dần như thế nào?
A. `56, 1A, C8` B. `56, 1A, C8, 00` C. `00, C8, 1A, 56` D. `C8, 1A, 56`

d) **(Mã đề 198 – Câu 11):** Một số 32-bit được lưu ở 4 ngăn nhớ liên tiếp từ địa chỉ 612 đến 615 lần lượt chứa các giá trị Hexa: `E` (tức `0E`), `48`, `21`, `A` (tức `0A`). Giá trị của số đó ở hệ 16 bằng bao nhiêu?
A. `E4821A` nếu đầu to B. `A4821E` nếu đầu to C. `0A48210E` nếu đầu nhỏ D. `0E48210A` nếu đầu to

Hướng dẫn giải:

- **Giải câu a:** Theo Little-Endian, ghép ngược từ byte ở địa chỉ lớn nhất về địa chỉ nhỏ nhất: `0x4D3C2B1A` → Chọn **B**.
- **Giải câu b:** Ghép ngược từ phải sang trái: `0x1A2B3C4D` → Chọn **B**.

- **Giải câu c:** Số 32-bit `0xC81A56` viết đủ 8 chữ số Hexa (4 byte) là `0x00C81A56`. Lưu theo Little-Endian từ byte thấp nhất đến byte cao nhất là: `56, 1A, C8, 00` → Chọn **B**.
- **Giải câu d:** Mỗi ngăn nhớ 1 byte phải biểu diễn bằng 2 chữ số Hexa: tại 612 là `0E`, 613 là `48`, 614 là `21`, 615 là `0A`. Nếu lưu theo thứ tự đầu to (Big-Endian) thì ghép xuôi từ 612 đến 615 được `0E48210A` (nếu đầu nhỏ thì ghép ngược là `0A21480E`) → Chọn **D**.

Bài tập 3.34 (Đề Giữa kỳ 2023.2 – Đề 1 Câu 3, Đề 2 Câu 1 & Giữa kỳ 2022.2 – Đề 0008, 0007 Câu 2)

a) **(Đề GK 2023.2 – Đề 1, Câu 3):** Bộ nhớ chính lưu theo kiểu đầu nhỏ (Little-Endian). Thanh ghi `s6` chứa địa chỉ một ngăn nhớ dữ liệu. Sau khi thực hiện: `lui s0, 0x1B32C` → `addi s0, s0, 0x2E5` (để `s0` = `0x1B32C2E5`) → `sw s0, 0(s6)` → `lb s1, 1(s6)`. Hãy chỉ ra nội dung của `s0` và `s1` theo số Hexa.

b) **(Đề GK 2023.2 – Đề 2, Câu 1):** Thực hiện đoạn lệnh: `lui s0, 0x4567C` → `addi s0, s0, -802` (để `s0` = `0x4567BCDE`) → `addi s1, zero, -16` → `and s2, s0, s1`. Hãy chỉ ra nội dung của `s1` và `s2` theo số Hexa.

c) **(Đề GK 2022.2 – Đề 0008 & 0007, Câu 2):**

- *Đề 0008:* Từ địa chỉ `0x100110F8` đến `0x100110FB` lưu 4 byte theo Little-Endian là: `0x5C, 0x52, 0x0E, 0xEF`. Giá trị biến 32-bit này bằng bao nhiêu? Viết lệnh nạp biến này vào `s1`.
- *Đề 0007:* Biến 32-bit có giá trị `0x4CDC001A` lưu ở địa chỉ `0x1001272C`. Hãy điền các byte vào 4 địa chỉ từ `0x1001272C` đến `0x1001272F`.

Hướng dẫn giải:

- **Giải câu a:**
  - Thanh ghi `s0 = 0x1B32C2E5`.
  - Khi `sw s0, 0(s6)` ra bộ nhớ Little-Endian: tại `0(s6)` là `0xE5`, tại `1(s6)` là `0xC2`, tại `2(s6)` là `0x32`, tại `3(s6)` là `0x1B`.
  - Lệnh `lb s1, 1(s6)` đọc byte `0xC2`. Vì `C` ≥ 8 (số âm), mở rộng dấu thành: `s1 = 0xFFFFFFC2`.
- **Giải câu b:**
  - Số -16 đổi sang mã bù 2 32-bit là: `s1 = 0xFFFFFFF0`.
  - Lệnh `and s2, s0, s1` xóa chữ số Hexa cuối cùng của `0x4567BCDE` về 0: `s2 = 0x4567BCD0`.
- **Giải câu c:**
  - *Đề 0008:* Ghép ngược từ `0x100110FB` về `0x100110F8` được `0xEF0E525C`. Lệnh nạp vào `s1`: `lui t0, 0x10011` → `lw s1, 0x0F8(t0)`.

– *Đề 0007:* Tại `0x1001272C` là `0x1A`; tại `0x1001272D` là `0x00`; tại `0x1001272E` là `0xDC`; tại `0x1001272F` là `0x4C`.

Bài tập 3.35 (Đề Giữa kỳ 2024.1 – #6a94a2 & Ôn tập Cuối kỳ 2025.2 – P1, Câu 49)
Địa chỉ của toán hạng đích trong lệnh `sw` bên dưới là gì (viết ở dạng số Hexa)?

```
li   s0, 0x1001BCDE
addi s1, s0, 0x321
sw   s0, 1(s1)
```

Hướng dẫn giải:

- Lệnh 1: `s0` = `0x1001BCDE`.
- Lệnh 2: `s1` = `0x1001BCDE` + `0x321` = `0x1001BFFF` (vì `0xCDE` + `0x321` = `0xFFF`).
- Lệnh 3 (`sw s0, 1(s1)`): Địa chỉ của toán hạng đích là:

$$\text{Địa chỉ} = \texttt{s1} + 1 = \texttt{0x1001BFFF} + 1 = \texttt{0x1001C000}$$

(Nếu đề bài cắt ngắn hằng số thành `li s0, 0x1001BC` và `addi s1, s0, 0x32` như ở ảnh chụp bị che của Câu 49 Ôn tập 2025.2 thì địa chỉ là `0x1001BC` + `0x32` + 1 = `0x1001EF`).

3.2.5 Nhóm 5: Chuyển đổi giữa Ngôn ngữ C và Hợp ngữ RISC-V (Biểu thức, Mảng, Con trỏ, Rẽ nhánh)

Bài tập 3.36 (Đề Giữa kỳ 2018.2 – Mã đề 198 Câu 13, Mã đề 199 Câu 5 & Mã đề 204 Câu 2)
Biến `X, Y, Z` nằm ở các thanh ghi `s1, s2, s3` tương ứng. Hỏi mỗi đoạn mã hợp ngữ dưới đây thực hiện phép toán nào?

a) **(Mã đề 198 & 199):** `addi t1, s1, 2` → `slli s2, s2, 2` → `sub s3, t1, s2`.
A. $Z = 2 - Y \times 2$ B. $Z = X + 2 - Y \times 4$
C. $Z = X \times 2 + Y/2$ D. $Z = X - Y \times 4$

b) **(Mã đề 204):** `add t1, s1, s2` → `srli t2, s2, 1` → `add s3, t1, t2`.
A. $Z = X + Y \times 2$ B. $Z = X + Y/2$
C. $Z = X/2 + Y$ D. $Z = X + Y + Y/2$

Hướng dẫn giải:

- **Giải câu a:**
  – `addi t1, s1, 2`: `t1` = $X + 2$.

- `slli s2, s2, 2`: Dịch trái 2 bit tương đương nhân với $2^2 = 4 \implies$ `s2` $= Y \times 4$.
- `sub s3, t1, s2`: $Z =$ `t1` $-$ `s2` $= X + 2 - Y \times 4 \rightarrow$ Chọn **B**.

- **Giải câu b:**
  - `add t1, s1, s2`: `t1` $= X + Y$.
  - `srli t2, s2, 1`: Dịch phải 1 bit tương đương chia nguyên cho $2^1 = 2 \implies$ `t2` $= Y/2$.
  - `add s3, t1, t2`: $Z = X + Y + Y/2 \rightarrow$ Chọn **D**.

Bài tập 3.37 (Đề Cuối kỳ 2020.2 – Trang 9, 2022.2 – Trang 14, 2024.1 – #f4, 2024.2 & Ôn tập 2025.2 – Câu 68)
Đoạn lệnh hợp ngữ RISC-V dưới đây tương ứng với câu lệnh C nào (biết rằng các thanh ghi `s0, s1, s2, s3` được sử dụng tương ứng cho các biến `f, g, h, i`)?

```
slli t0, s1, 2
sub  s3, s0, s2
add  s3, s3, t0
addi s3, s3, 5
```

A. $i = f + h + 4 * g - 5$

B. $i = f - h + 4 * g + 5$

C. $i = f - h + 4 * g - 5$

D. $i = f + h - 4 * g - 5$

Hướng dẫn giải:

- Lệnh 1 (`slli t0, s1, 2`): Dịch trái `s1` (g) đi 2 bit, tương đương `t0` $= 4 * g$.
- Lệnh 2 (`sub s3, s0, s2`): `s3` $= f - h$.
- Lệnh 3 (`add s3, s3, t0`): `s3` $= f - h + 4 * g$.
- Lệnh 4 (`addi s3, s3, 5`): $i =$ `s3` $= f - h + 4 * g + 5$.
- **Kết luận:** Đáp án đúng là **B** ($i = f - h + 4 * g + 5$).

Bài tập 3.38 (Đề Giữa kỳ 2021.1 – IT3030E, Problem 1)
Dịch câu lệnh C sau đây sang hợp ngữ RISC-V mà không sử dụng lệnh giả (pseudo instructions):

```
x = (4*x - y) & (x + 5*z);
```

Biết rằng các biến `x, y, z` được lưu lần lượt trong các thanh ghi `s1, s2, s3`.

Hướng dẫn giải:

- **Phân tích:** Vì bộ xử lý cơ bản thực hiện phép nhân với hằng số qua phép dịch bit và phép cộng, ta tách $4 * x = (x \ll 2)$ và $5 * z = 4 * z + z = (z \ll 2) + z$:

```
slli t0, s1, 2      # t0 = 4 * x
sub  t0, t0, s2     # t0 = 4 * x - y
slli t1, s3, 2      # t1 = 4 * z
add  t1, t1, s3     # t1 = 4 * z + z = 5 * z
add  t1, s1, t1     # t1 = x + 5 * z
and  s1, t0, t1     # x = (4 * x - y) & (x + 5 * z)
```

Bài tập 3.39 (Đề Giữa kỳ 2022.2 – Đề 0009, 0008, 0007, 0006 & Giữa kỳ 2025.2 – Đề 0012 Câu 107)

a) **(Đề 0009, 0008, 0012):** Viết 1 lệnh hợp ngữ còn thiếu vào chỗ trống giữa 2 lệnh `add t2, s3, s4` và `sub s3, t3, s4` để thực hiện biểu thức (với `s3` là biến ở vế trái):

- TH1 (Đề 0009): $y = (x + y) \times 16 - x$ (với y ở `s3`, x ở `s4`).
- TH2 (Đề 0008): $x = (x + y) \times 4 - y$ (với x ở `s3`, y ở `s4`).
- TH3 (Đề 0012): $a = (a + b) \times 64 - a$ (với a ở `s3` ở vế trái, `s4` $= a$ bị trừ).

b) **(Đề 0008 – Câu 6):** Thay thế chuỗi 3 lệnh liên tiếp: `add s0, t2, t2` → `slli s0, s0, 1` → `slli s0, s0, 2` bằng 1 lệnh hợp ngữ duy nhất.

c) **(Đề 0008 Câu 7, Đề 0007 Câu 6 & Đề 0006 Câu 10):** Lập trình hợp ngữ RISC-V cho 3 biểu thức:

- Biểu thức 1: `t0` $= 4 *$ (`t1` + `t2`) $+ 5$.
- Biểu thức 2: `t0` $=$ `t1` $+ 8 *$ (`t2` $- 7$).
- Biểu thức 3: `s2` $= 8 *$ (`t3` + `t7`) $+ 6$.

Hướng dẫn giải:

- **Giải câu a:** Thanh ghi `t2` đã giữ tổng $(x + y)$, lệnh thứ 3 lấy `t3` $-$ `s4`, nên lệnh ở giữa cần dịch trái `t2` và cất vào `t3`:
  - TH1 (nhân $16 = 2^4$): `slli t3, t2, 4`.
  - TH2 (nhân $4 = 2^2$): `slli t3, t2, 2`.
  - TH3 (nhân $64 = 2^6$): `slli t3, t2, 6`.
- **Giải câu b:** Lệnh `add s0, t2, t2` nhân 2, `slli 1` nhân 2, `slli 2` nhân 4. Tổng cộng nhân $2 \times 2 \times 4 = 16 = 2^4$. Thay bằng 1 lệnh duy nhất là: `slli s0, t2, 4`.
- **Giải câu c:**
  - Biểu thức 1: `add t0, t1, t2` → `slli t0, t0, 2` → `addi t0, t0, 5`.

– Biểu thức 2: `addi t0, t2, -7` → `slli t0, t0, 3` → `add t0, t1, t0`.

– Biểu thức 3: `add s2, t3, t7` → `slli s2, s2, 3` → `addi s2, s2, 6`.

Bài tập 3.40 (Đề Giữa kỳ 2024.1 – #f96f43, #12416e & Ôn tập Cuối kỳ 2025.2 – P1, Câu 32)

a) **(Câu #f96f43):** Chọn đoạn lệnh RISC-V thực hiện câu lệnh C: `*a = b + c;` (giả sử `a` là con trỏ trong thanh ghi `a1`, còn `b, c` là các biến nguyên trong `a2, a3`).
A. `lw t0, 0(a1)` → `add t0, a2, a3` B. `add t0, a2, a3` → `lw t0, 0(a1)`
C. `sw t0, 0(a1)` → `add t0, a2, a3` D. `add t0, a2, a3` → `sw t0, 0(a1)`

b) **(Câu #12416e & Câu 32):** Chọn đoạn lệnh RISC-V thực hiện câu lệnh C: `a = b + *c;` (giả sử `a, b` nằm trong `a1, a2`, còn `c` là một con trỏ nằm trong `a3`).
A. `li t0, 0(a3)` → `add a1, a2, t0` B. `sw t0, 0(a3)` → `add a1, a2, t0`
C. `la t0, 0(a3)` → `add a1, a2, t0` D. `lw t0, 0(a3)` → `add a1, a2, t0`

Hướng dẫn giải:

- **Giải câu a:** Toán tử `*a` ở vế trái nghĩa là lưu kết quả vào ô nhớ được trỏ bởi `a1`. Ta tính tổng `add t0, a2, a3` trước, rồi ghi ra bộ nhớ bằng `sw t0, 0(a1)` → Chọn **D**.
- **Giải câu b:** Toán tử `*c` ở vế phải nghĩa là đọc giá trị từ ô nhớ được trỏ bởi `a3`. Ta đọc bằng `lw t0, 0(a3)` trước, rồi cộng với `a2` bằng `add a1, a2, t0` → Chọn **D**.

Bài tập 3.41 (Đề Cuối kỳ 2022.2 – #13504d, 2024.2 & Đề Giữa kỳ 2024.1 – #d0125d)

a) **(Đề CK 2022.2 & 2024.2 – #13504d):** Biến `k` ở thanh ghi `s1`; `P, Q` là hai mảng số nguyên 32-bit có địa chỉ cơ sở ở `s6` và `s7`. Đoạn lệnh: `lw t0, 160(s7)` → `add t0, s1, t0` → `sw t0, 80(s6)` tương ứng với câu lệnh C nào?
A. `P[80] = k + Q[160]` B. `Q[40] = k + P[20]`
C. `P[20] = k + Q[40]` D. `Q[160] = k + P[80]`

b) **(Đề GK 2024.1 – #d0125d):** Cho `s1, s2` giữ địa chỉ cơ sở của mảng số nguyên 32-bit `A` và `B`. Đoạn lệnh: `lw t0, 24(s1)` → `addi s0, t0, 1` → `sw s0, 20(s2)` tương ứng với câu lệnh C nào?
A. `A[24] = B[20] + 1` B. `B[20] = A[24] + 1`
C. `A[5] = B[6] + 1` D. `B[5] = A[6] + 1`

Hướng dẫn giải:

- Mỗi phần tử số nguyên 32-bit chiếm 4 byte, do đó Chỉ số mảng = Độ lệch byte (Offset) $\div$ 4:
- **Câu a:** `160(s7)` → $Q[160 \div 4] = Q[40]$; `80(s6)` → $P[80 \div 4] = P[20]$. Vậy biểu thức là `P[20] = k + Q[40]` → Chọn **C**.

- **Câu b:** `24(s1)` $\rightarrow A[24 \div 4] = A[6]$; `20(s2)` $\rightarrow B[20 \div 4] = B[5]$. Vậy biểu thức là `B[5] = A[6] + 1` $\rightarrow$ Chọn **D**.

Bài tập 3.42 (Đề Giữa kỳ 2023.2 – Đề 1 Câu 5 & Đề 2 Câu 3)
Giả thiết `f, g, h, i, j` là các biến số nguyên 32-bit ở `s0, s1, s2, s3, s4`; `A` và `B` là hai mảng số nguyên 32-bit có địa chỉ cơ sở ở `s6` và `s7`.

a) **(Đề 1 – Câu 5):** Viết đoạn hợp ngữ RISC-V (có giải thích) cho câu lệnh C: `f = g + h - A[20];` (với địa chỉ cơ sở của `A` ở `s7`).

b) **(Đề 2 – Câu 3):** Viết câu lệnh C tương ứng với 2 đoạn hợp ngữ sau:

- Đoạn 1: `lw t1, 20(s7)` $\rightarrow$ `sub s2, s3, t1`.
- Đoạn 2: `add t0, s1, s0` $\rightarrow$ `slli t1, s4, 2` $\rightarrow$ `add t1, s6, t1` $\rightarrow$ `sw t0, 0(t1)`.

Hướng dẫn giải:

- **Giải câu a:** Phần tử `A[20]` có độ lệch là $20 \times 4 = 80$ byte:

```
lw   t0, 80(s7)     # t0 = A[20] (do lech 20 * 4 = 80 byte)
add  t1, s1, s2     # t1 = g + h
sub  s0, t1, t0     # f = (g + h) - A[20]
```

- **Giải câu b:**
  - Đoạn 1: `20(s7)` là phần tử `B[20/4]` = `B[5]`, `s2` là h, `s3` là i $\implies$ Câu lệnh C: `h = i - B[5];`.
  - Đoạn 2: `t0` $= g + f$; `slli t1, s4, 2` tính $4 \times j$, cộng với `s6` ra địa chỉ của `A[j]`, rồi lưu `t0` vào `A[j]` $\implies$ Câu lệnh C: `A[j] = g + f;`.

Bài tập 3.43 (Đề Cuối kỳ 2021.2 – Mã đề B2 Câu 2 & Mã đề B3 Câu 2)
Viết đoạn chương trình hợp ngữ tối thiểu của RISC-V (không dùng lệnh giả, có lời giải thích) để thực hiện các câu lệnh C sau (biết biến `a` nằm trong thanh ghi `s2`):

a) **(Mã đề B2):** `x[2] = x[7] + a;` với `x` là mảng 32-bit có địa chỉ cơ sở là `0x00283A00`.

b) **(Mã đề B3):** `x[3] = x[9] + a;` với `x` là mảng 32-bit có địa chỉ cơ sở là `0x00D28900`.

Hướng dẫn giải:

- **Giải câu a (Địa chỉ cơ sở `0x00283A00`):**
  Chữ số Hexa thứ 3 từ phải sang là `A` ≥ 8 (`0xA00` $= 2560 - 4096 = -1536$), nên khi dùng `lui` phải tăng `0x00283` lên `0x00284`:

```
lui  t0, 0x00284      # t0 = 0x00284000
addi t0, t0, -1536    # t0 = 0x00283A00 (dia chi co so mang x)
lw   t1, 28(t0)       # t1 = x[7] (do lech 7 * 4 = 28 byte)
add  t1, t1, s2       # t1 = x[7] + a
sw   t1, 8(t0)        # x[2] = x[7] + a (do lech 2 * 4 = 8 byte)
```

- **Giải câu b (Địa chỉ cơ sở `0x00D28900`):**
  Chữ số thứ 3 từ phải sang là $9 \geq 8$ (`0x900` $= 2304 - 4096 = -1792$), tăng `0x00D28` lên `0x00D29`:

```
lui  t0, 0x00D29      # t0 = 0x00D29000
addi t0, t0, -1792    # t0 = 0x00D28900 (dia chi co so mang x)
lw   t1, 36(t0)       # t1 = x[9] (do lech 9 * 4 = 36 byte)
add  t1, t1, s2       # t1 = x[9] + a
sw   t1, 12(t0)       # x[3] = x[9] + a (do lech 3 * 4 = 12 byte)
```

Bài tập 3.44 (Đề Giữa kỳ 2019.2 – Mã đề 243 Câu 7, 8, Mã đề 244 Câu 16, 18, Giữa kỳ 2022.2 – Đề 0008 Câu 8, Đề 0007 Câu 7, Giữa kỳ 2024.1 – #3602ec & Cuối kỳ 2025.1 – #da417b)
Hãy xác định địa chỉ hoặc chỉ số phần tử mảng được truy cập trong các bài toán sau:

a) **(Mã đề 243 Câu 7 & Mã đề 244 Câu 18):** Mảng 32-bit `A` có `A[6]` ở `0x20200508`, hỏi `A[9]` ở đâu? Nếu `A[2]` ở `0x20200008`, hỏi `A[5]` ở đâu?

b) **(Mã đề 243 Câu 8 & Mã đề 244 Câu 16):** Mảng 32-bit `A` có địa chỉ cơ sở `0x20201234`.

- TH1 (Mã đề 243): Chạy `lui s0, 0x20201` → `addi s0, s0, 0x230` → `lw t0, 8(s0)`. Hỏi `t0` nhận phần tử nào?
- TH2 (Mã đề 244): Chạy `lui s0, 0x20201` → `addi s0, s0, 0x238` → `lw t0, 4(s0)`. Hỏi `t0` nhận phần tử nào?

c) **(Đề GK 2022.2 – Đề 0008 Câu 8 & Đề 0007 Câu 7):** Mảng 32-bit `A` ở địa chỉ cơ sở `0x1000CDC0`.

- TH1 (Đề 0008): Khi `s1` = `0x1000CDC4`, thực hiện `lw t1, 4(s1)`. Hỏi các địa chỉ byte nào được nạp vào `t1` và đó là phần tử nào của `A`?
- TH2 (Đề 0007): Khi `s1` = `0x1000CDC8`, thực hiện `lw t1, -4(s1)`. Hỏi đó là phần tử nào của `A`?

d) **(Đề GK 2024.1 – #3602ec, Ôn tập 2025.2 Câu 72 & CK 2025.1 – #da417b):**

- TH1: Mảng `A[]` ở địa chỉ cơ sở `0x20230200`. Chạy `lui s0, 0x20230` → `addi s0, s0, 0x20C` → `lw t0, 4(s0)`. Hỏi `t0` giữ phần tử nào?

- TH2: Mảng `A[]` ở địa chỉ cơ sở `0x20240214`. Chạy `lui t1, 0x20240` → `addi t1, t1, 0x218` → `lw t0, 4(t1)`. Hỏi `t0` giữ phần tử nào?

Hướng dẫn giải:

- **Giải câu a:** Cách nhau 3 phần tử = $3 \times 4 = 12$ byte = `0x0C`. Vậy `A[9]` = `0x20200508` + `0x0C` = `0x20200514`; và `A[5]` = `0x20200008` + `0x0C` = `0x20200014`.
- **Giải câu b:**
  - TH1: Địa chỉ đọc = `0x20201230` + 8 = `0x20201238`. Trừ địa chỉ cơ sở `0x20201234` được 4 byte $\implies 4 \div 4 = 1 \rightarrow$ `A[1]`.
  - TH2: Địa chỉ đọc = `0x20201238` + 4 = `0x2020123C`. Trừ `0x20201234` được 8 byte $\implies 8 \div 4 = 2 \rightarrow$ `A[2]`.
- **Giải câu c:**
  - TH1: Địa chỉ đọc = `0x1000CDC4` + 4 = `0x1000CDC8` (gồm 4 byte từ `0x1000CDC8` đến `0x1000CDCB`). Độ lệch so với `0x1000CDC0` là 8 byte $\implies$ `A[2]`.
  - TH2: Địa chỉ đọc = `0x1000CDC8` − 4 = `0x1000CDC4`. Độ lệch là 4 byte $\implies$ `A[1]`.
- **Giải câu d:**
  - TH1: Địa chỉ đọc = `0x2023020C` + 4 = `0x20230210`. Trừ `0x20230200` được `0x10` = 16 byte $\implies 16 \div 4 = 4 \rightarrow$ `A[4]`.
  - TH2: Địa chỉ đọc = `0x20240218` + 4 = `0x2024021C`. Trừ `0x20240214` được 8 byte $\implies 8 \div 4 = 2 \rightarrow$ `A[2]`.

Bài tập 3.45 (Đề Cuối kỳ 2022.2 & Cuối kỳ 2024.2 – #0542f & #70e201)

a) **(Câu #0542f):** Mảng `M[]` gồm 100 phần tử số nguyên 32-bit bắt đầu từ địa chỉ `0x10020400`. Đoạn lệnh hoán đổi `M[i]` và `M[j]` dùng `lw t0, 0(s6)` và `lw t1, 0(s7)`. Nếu $i = 9, j = 6$, nội dung các thanh ghi `s6` và `s7` cần có giá trị bao nhiêu?

b) **(Câu #70e201):** Mảng 2 chiều `M` kích thước (4×5) gồm các phần tử 32-bit, lưu trữ theo hàng bắt đầu tại địa chỉ `0x10024000`. Để lệnh `lw t0, 0(s6)` đọc phần tử `M[2, 2]` (với $i = 0..3, j = 0..4$), nội dung thanh ghi `s6` phải bằng bao nhiêu?

Hướng dẫn giải:

- **Giải câu a:**
  - Với $i = 9$: Độ lệch = $9 \times 4 = 36$ byte = `0x24` $\implies$ `s6` = `0x10020400` + `0x24` = `0x10020424`.
  - Với $j = 6$: Độ lệch = $6 \times 4 = 24$ byte = `0x18` $\implies$ `s7` = `0x10020400` + `0x18` = `0x10020418`.

- **Giải câu b:** Mảng có 5 cột. Với $i = 2, j = 2$, số phần tử đứng trước `M[2, 2]` là $2 \times 5 + 2 = 12$ phần tử. Độ lệch byte là $12 \times 4 = 48$ byte = `0x30`. Vậy `s6` = `0x10024000` + `0x30` = `0x10024030`.

Bài tập 3.46 (Đề Cuối kỳ 2021.2 – Mã đề B1 Câu 4.1 & Ôn tập Cuối kỳ 2025.2 – P2, Câu 67)

Trên máy tính có các thanh ghi 32-bit, giả sử có một mảng dữ liệu hai chiều `B` kích thước 5×7 (5 hàng, 7 cột) của các phần tử 32-bit. Mảng `B` được lưu trữ theo hàng trong bộ nhớ chính, cấp phát ban đầu từ địa chỉ `0x000A5020`. Để trỏ vào phần tử `B[i, j]`, yêu cầu kết hợp thanh ghi `R1` để trỏ vào đầu hàng thứ i, thanh ghi `R2` để trỏ tương đối vào phần tử thứ j trong hàng i tính từ đầu hàng.

a) **(Đề CK 2021.2 – B1):** Với chỉ số tính từ 1 ($i = 1..5, j = 1..7$), các thanh ghi `R1` và `R2` phải được gán giá trị bằng bao nhiêu (dạng Hexa) khi trỏ vào phần tử `B[2, 5]`?

b) **(Ôn tập 2025.2 – P2, Câu 67):** Với chỉ số tính từ 0 ($i = 0..4, j = 0..6$), cho biết địa chỉ của phần tử `B[1, 4]`?

Hướng dẫn giải:

- Mỗi hàng có 7 phần tử 32-bit (4 byte), nên kích thước 1 hàng là $7 \times 4 = 28$ byte = `0x1C` byte.
- **Giải câu a:** Phần tử `B[2, 5]` (khi đếm từ 1) nằm ở hàng thứ 2, cột thứ 5:
  - Thanh ghi `R1` trỏ vào đầu hàng 2 (cách đầu mảng 1 hàng = `0x1C`): `R1` = `0x000A5020` + `0x1C` = `0x000A503C`.
  - Thanh ghi `R2` trỏ tương đối vào phần tử thứ 5 trong hàng (cách đầu hàng $5 - 1 = 4$ phần tử = 16 byte = `0x10`): `R2 = 0x00000010`.
- **Giải câu b:** Phần tử `B[1, 4]` (khi đếm từ 0) cũng chính là phần tử ở hàng thứ 2, cột thứ 5 nói trên. Địa chỉ của nó là: `R1` + `R2` = `0x000A503C` + `0x10` = `0x000A504C`.

Bài tập 3.47 (Đề Cuối kỳ 2022.2 – #bcfd00, 2024.2 & Đề Cuối kỳ 2025.1 – #31d5bf)

a) **(Đề CK 2022.2 & 2024.2 – #bcfd00):** Cho các biến `a, b, c` được gán tương ứng trong `s0, s1, s2`. Hãy chỉ ra câu lệnh C tương ứng với đoạn hợp ngữ sau:

```
    slt  t0, s2, s1
    bne  t0, zero, L1
    add  s0, s0, s1
    j    L2
L1: add  s0, s0, s2
L2:
```

A. `if (b > c) a = a + b; else a = a + c;`
B. `if (b <= c) a = a + b; else a = a + c;`
C. `if (b < c) a = a + b; else a = a + c;`
D. `if (b >= c) a = a + b; else a = a + c;`

b) **(Đề CK 2025.1 – #31d5bf & Ôn tập 2025.2 – P2, Câu 46):** Cho các biến `f, g, h, i, j` lần lượt nằm trong `s0, s1, s2, s3, s4`. Đoạn mã hợp ngữ sau tương đương với đoạn mã C nào?

```
    blt  s2, s1, L1
    beq  s3, s4, L1
    add  s0, s0, s1
    jal  x0, L2
L1: sub  s0, s0, s1
L2:
```

A. `if ((g > h) && (i == j)) f = f + g; else f = f - g;`
B. `if ((h >= g) && (i == j)) f = f + g; else f = f - g;`
C. `if ((g > h) && (i != j)) f = f + g; else f = f - g;`
D. `if ((h >= g) && (i != j)) f = f + g; else f = f - g;`

Hướng dẫn giải:

- **Giải câu a:** Lệnh `slt t0, s2, s1` kiểm tra `s2` < `s1` ($c < b$, tức $b > c$). Nếu $b > c$ (`t0` $= 1 \neq 0$) thì nhảy xuống `L1` thực hiện $a = a + c$. Nếu không nhảy (tức $b \leq c$) thì thực hiện lệnh ngay bên dưới là $a = a + b \rightarrow$ Chọn **B**.
- **Giải câu b:** Để thực hiện lệnh $f = f + g$ (`add s0, s0, s1`), chương trình phải **không nhảy** ở cả hai lệnh `blt` và `beq`:
  - Không nhảy ở `blt s2, s1, L1` (nhảy khi $h < g$) $\implies$ Điều kiện là $h \geq g$.
  - Không nhảy ở `beq s3, s4, L1` (nhảy khi $i == j$) $\implies$ Điều kiện là $i \neq j$.
  - Kết hợp lại: `if ((h >= g) && (i != j)) f = f + g; else f = f - g;` $\rightarrow$ Chọn **D**.

Bài tập 3.48 (Đề Giữa kỳ 2023.2 – Đề 2 Câu 5, Giữa kỳ 2022.2 – Đề 0007 Câu 8 & Cuối kỳ 2021.2 – Mã đề B5 Câu 3, Mã đề B8 Câu 3)
Viết đoạn chương trình hợp ngữ tối thiểu của RISC-V (không dùng lệnh giả, có lời giải thích) cho 4 câu lệnh rẽ nhánh `if/else` sau:

a) **(Đề GK 2023.2 – Đề 2, Câu 5):** `if (i <= j) f = f + g; else f = f + h;` (với `f, g, h, i, j` ở `s0, s1, s2, s3, s4`).

b) **(Đề GK 2022.2 – Đề 0007, Câu 8):** `if (s1 == 0) t2 = 1; else t2 = 2;`.

c) **(Đề CK 2021.2 – Mã đề B5, Câu 3):** `if (a == b) i += 1; else i += 3;` (với `a, b, i` ở `s0, s1, s2`).

d) **(Đề CK 2021.2 – Mã đề B8, Câu 3):** `if (a == 1) m[3] = b - a; else m[4] = b + a*2;` (với `a, b` ở `s0, s1`; `m` là mảng 32-bit có địa chỉ cơ sở `0x13090134`).

Hướng dẫn giải:

- **Giải câu a:** Điều kiện ngược của $i \leq j$ là $j < i$ (`blt s4, s3, Else`):

```
    blt  s4, s3, Else     # Neu j < i (tuc i > j) thi nhay xuong Else
    add  s0, s0, s1       # f = f + g
    jal  x0, Exit         # Nhay khong dieu kien toi Exit
Else:
    add  s0, s0, s2       # f = f + h
Exit:
```

- **Giải câu b:**

```
    bne  s1, x0, Else     # Neu s1 != 0 thi nhay xuong Else
    addi t2, x0, 1        # t2 = 1
    jal  x0, Exit         # Nhay qua nhanh Else
Else:
    addi t2, x0, 2        # t2 = 2
Exit:
```

- **Giải câu c:**

```
    bne  s0, s1, Else     # Neu a != b thi nhay xuong Else
    addi s2, s2, 1        # i = i + 1
    jal  x0, Exit         # Nhay qua nhanh Else
Else:
    addi s2, s2, 3        # i = i + 3
Exit:
```

- **Giải câu d:**

```
    lui  t0, 0x13090      # Nap 20 bit cao cua dia chi mang m
    addi t0, t0, 0x134    # t0 = 0x13090134 (dia chi co so mang m)
    addi t1, x0, 1        # t1 = 1
    bne  s0, t1, Else     # Neu a != 1 thi nhay xuong Else
    sub  t2, s1, s0       # t2 = b - a
    sw   t2, 12(t0)       # m[3] = b - a (do lech 3 * 4 = 12 byte)
    jal  x0, Exit         # Nhay toi Exit
Else:
    slli t2, s0, 1        # t2 = a * 2
    add  t2, s1, t2       # t2 = b + a * 2
    sw   t2, 16(t0)       # m[4] = b + a * 2 (do lech 4 * 4 = 16B)
Exit:
```

Bài tập 3.49 (Đề Giữa kỳ 2021.1 – IT3030E, Problem 5)
Cho đoạn mã C sau:

```
if (i < j) f = A[2*i] + B[4*j];
```

Giả sử các biến `f, i, j` được gán cho các thanh ghi `s0, s1, s2`; địa chỉ cơ sở của mảng 32-bit `A` và `B` nằm trong `s6` và `s7`. Hãy dịch câu lệnh trên sang hợp ngữ RISC-V và viết mã máy cho từng lệnh.

Hướng dẫn giải:

- **Phân tích độ lệch byte:** Mỗi phần tử rộng 4 byte. Phần tử `A[2*i]` có độ lệch byte là $4 \times (2*i) = 8*i = (i \ll 3)$; phần tử `B[4*j]` có độ lệch byte là $4 \times (4*j) = 16*j = (j \ll 4)$:

```
    bge  s1, s2, Exit     # Ma may: 0x0124D063 (offset = +32)
    slli t0, s1, 3        # Ma may: 0x00349293 (t0 = 8 * i)
    add  t0, s6, t0       # Ma may: 0x005B02B3 (dia chi A[2*i])
    lw   t0, 0(t0)        # Ma may: 0x0002A283 (t0 = A[2*i])
    slli t1, s2, 4        # Ma may: 0x00491313 (t1 = 16 * j)
    add  t1, s7, t1       # Ma may: 0x006B8333 (dia chi B[4*j])
    lw   t1, 0(t1)        # Ma may: 0x00032303 (t1 = B[4*j])
    add  s0, t0, t1       # Ma may: 0x00628433 (f = A[2*i] + B[4*j])
Exit:
```

Bài tập 3.50 (Tổng hợp 4 bài tự luận viết hợp ngữ vòng lặp `for` trong Đề Cuối kỳ 2021.2 – Mã đề B1, B4, B6, B7)
Viết đoạn chương trình hợp ngữ tối thiểu của RISC-V (không dùng lệnh giả, có lời giải thích) để thực hiện các vòng lặp ngôn ngữ C sau đây:

a) **(Mã đề B1, Câu 3):** `for (i = 0; i < 50; i += 2) a = a + m[5];` (với `m` là mảng

các phần tử 32-bit có địa chỉ cơ sở `0x07E1100A`; giả sử `i, a` ở `s0, s1`).

b) **(Mã đề B4, Câu 2):** `for (i = 1; i != 20; i += 3) a[i] = a[5] + 1;` (với `a` là mảng 32-bit có địa chỉ cơ sở `0x0FE3B128`; `i` ở `s0`).

c) **(Mã đề B6, Câu 3):** `for (i = 0; i < 100; i++) N[i] = N[i] * 16;` (với `N` là mảng 32-bit có địa chỉ cơ sở `0x100A4023`; `i` ở `s0`).

d) **(Mã đề B7, Câu 3):** `for (i = 0; i < 100; i++) M[i] = M[i] * 8;` (với `M` là mảng 32-bit có địa chỉ cơ sở `0x07AD1309`; `i` ở `s0`).

Hướng dẫn giải:

- **Giải câu a (Mã đề B1):** Phần tử `m[5]` cố định nên đọc ra ngoài vòng lặp để tối ưu:

```
    lui  t0, 0x07E11      # Nap 20 bit cao cua dia chi co so m
    addi t0, t0, 0x00A    # t0 = 0x07E1100A (dia chi co so mang m)
    lw   t1, 20(t0)       # t1 = m[5] (do lech 5 * 4 = 20 byte)
    addi s0, x0, 0        # i = 0
    addi t2, x0, 50       # t2 = 50 (gia tri gioi han)
Loop:
    bge  s0, t2, Done     # Neu i >= 50 thi thoat vong lap
    add  s1, s1, t1       # a = a + m[5]
    addi s0, s0, 2        # i = i + 2
    jal  x0, Loop         # Quay lai Loop
Done:
```

- **Giải câu b (Mã đề B4):**

```
    lui  t0, 0x0FE3B      # Nap 20 bit cao cua dia chi co so a
    addi t0, t0, 0x128    # t0 = 0x0FE3B128 (dia chi co so mang a)
    lw   t1, 20(t0)       # t1 = a[5]
    addi t1, t1, 1        # t1 = a[5] + 1
    addi s0, x0, 1        # i = 1
    addi t2, x0, 20       # t2 = 20
Loop:
    beq  s0, t2, Done     # Neu i == 20 thi thoat
    slli t3, s0, 2        # t3 = i * 4 (do lech byte cua a[i])
    add  t3, t0, t3       # t3 = dia chi cua a[i]
    sw   t1, 0(t3)        # a[i] = a[5] + 1
    addi s0, s0, 3        # i = i + 3
    jal  x0, Loop         # Quay lai Loop
Done:
```

- **Giải câu c và câu d (Mã đề B6 & B7):** Nhân với $16 = 2^4$ dùng `slli ..., 4`; nhân với $8 = 2^3$ dùng `slli ..., 3`. (Đoạn mã dưới viết cho Đề B6, với Đề B7 chỉ cần thay địa chỉ `0x07AD1` / `0x309` và dịch trái `3`):

```
    lui  t0, 0x100A4      # Nap 20 bit cao dia chi mang N
    addi t0, t0, 0x023    # t0 = 0x100A4023 (dia chi co so N[0])
    addi s0, x0, 0        # i = 0
    addi t1, x0, 100      # t1 = 100
Loop:
    bge  s0, t1, Done     # Neu i >= 100 thi thoat
    lw   t2, 0(t0)        # t2 = N[i]
    slli t2, t2, 4        # t2 = N[i] * 16 (De B7 thay 4 bang 3)
    sw   t2, 0(t0)        # N[i] = N[i] * 16
    addi t0, t0, 4        # Tang con tro t0 sang N[i+1]
    addi s0, s0, 1        # i++
    jal  x0, Loop         # Lap lai
Done:
```

3.2.6 Nhóm 6: Phân tích Vòng lặp và Đếm tổng số lệnh thực thi

Bài tập 3.51 (Đề Cuối kỳ 2020.2 – Trang 4 & Đề Cuối kỳ 2021.1 – Trang 3)
Hãy phân tích đoạn mã nguồn chương trình vòng lặp hợp ngữ RISC-V dưới đây và xác định giá trị hằng số hệ thập phân thu được tại thanh ghi kết quả `a0` sau khi thực thi hoàn tất chương trình:

```
    addi a1, zero, 14
    addi s0, zero, 0
    add  s1, zero, zero
loop:
    slt  t0, a1, s1
    bne  t0, zero, finish
    add  s0, s0, s1
    addi s1, s1, 2
    j    loop
finish:
    add  a0, s0, zero
```

A. 54

B. 14

C. 56

D. 28

Hướng dẫn giải:

- Khởi tạo: `a1` = 14, biến tổng `s0` = 0, biến chạy `s1` = 0.
- Trong mỗi vòng lặp, lệnh `slt t0, a1, s1` kiểm tra 14 < `s1`. Chừng nào `s1` ≤ 14 thì `t0` = 0 (không nhảy tới `finish`), chương trình cộng `s1` vào `s0` và tăng `s1` thêm 2. Khi `s1` = 16 (14 < 16 đúng) thì thoát vòng lặp.
- Vậy chương trình tính tổng các số chẵn từ 0 đến 14:

$$\texttt{a0} = \texttt{s0} = 0 + 2 + 4 + 6 + 8 + 10 + 12 + 14 = 56$$

- **Kết luận:** Đáp án đúng là **C (56)**.

Bài tập 3.52 (Đề Giữa kỳ 2024.1 – #f4fd9f & Ôn tập Cuối kỳ 2025.2 – P1, Câu 35)

Sau khi chạy chương trình dưới đây, giá trị thập phân của thanh ghi `s5` là bao nhiêu?

```
.data
A: .word 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15
.text
    li   s1, 0
    li   s3, 10
    la   s2, A
    li   s5, 0
loop:
    add  t1, s1, s1
    add  t1, t1, t1
    add  t1, t1, s2
    lw   t0, 0(t1)
    add  s5, s5, t0
    addi s1, s1, 2
    bne  s1, s3, loop
```

Hướng dẫn giải:

- Hai lệnh `add t1, s1, s1` (`t1` = 2 × `s1`) và `add t1, t1, t1` (`t1` = 4 × `s1`) tính độ lệch byte của phần tử `A[s1]`.
- Biến chỉ số `s1` bắt đầu từ 0, mỗi vòng tăng thêm 2, và dừng khi `s1` == 10. Vậy `s1` lần lượt nhận các giá trị: 0, 2, 4, 6, 8.

- Tổng cộng dồn vào `s5` là (chú ý `A[0]` = 1):

$$\texttt{s5} = \texttt{A[0]} + \texttt{A[2]} + \texttt{A[4]} + \texttt{A[6]} + \texttt{A[8]} = 1 + 3 + 5 + 7 + 9 = \mathbf{25}$$

Bài tập 3.53 (Ôn tập Cuối kỳ 2025.2 – P1, Câu 33)
Sau khi thực thi đoạn mã sau, giá trị cuối cùng của thanh ghi `x15` là bao nhiêu (thập phân)? Giả sử ban đầu `x14` chứa 4 và `x15` chứa 0:

```
Loop:
    addi x15, x15, 1
    addi x14, x14, -1
    bge  x14, x0, Loop
```

A. 4

B. 0

C. 5

D. 6

Hướng dẫn giải:

- Lệnh `bge x14, x0, Loop` quay lại vòng lặp chừng nào `x14` ≥ 0.
- Lần 1: `x15` $= 1, $`x14` $= 3 \geq 0$ (lặp lại).
- Lần 2: `x15` $= 2, $`x14` $= 2 \geq 0$ (lặp lại).
- Lần 3: `x15` $= 3, $`x14` $= 1 \geq 0$ (lặp lại).
- Lần 4: `x15` $= 4, $`x14` $= 0 \geq 0$ (vẫn thỏa mãn ≥ 0 nên lặp lại lần nữa).
- Lần 5: `x15` $= 5, $`x14` $= -1 < 0$ (thoát vòng lặp).
- **Kết luận:** Đáp án đúng là **C (5)**.

Bài tập 3.54 (Đề Cuối kỳ 2021.2 – Mã đề B1 Câu 2 & Mã đề B5 Câu 2)

a) **(Mã đề B1, Câu 2):** Cho đoạn chương trình hợp ngữ:

```
    addi t1, zero, 100
LOOP:
    lw   s1, 0(s0)
    add  s2, s2, s1
    addi s0, s0, 4
    addi t1, t1, -1
    bne  t1, x0, LOOP
```

2.1. Khi thực hiện đoạn chương trình trên, có bao nhiêu lệnh được thực hiện?

2.2. Biết `t1` chứa biến `i`, `s2` chứa biến `a`, `s0` chứa địa chỉ cơ sở mảng `A`. Hãy chuyển sang ngôn ngữ C.

b) **(Mã đề B5, Câu 2):** Cho đoạn chương trình hợp ngữ (giả sử ban đầu `s0` giữ địa chỉ cơ sở mảng `B`, và lệnh đầu gán `t1` = `s0` + 600):

```
    addi t1, s0, 600
LOOP:
    lw   s1, 0(s0)
    add  s2, s2, s1
    lw   s1, 4(s0)
    add  s2, s2, s1
    addi s0, s0, 8
    bne  t1, s0, LOOP
```

2.1. Có bao nhiêu lệnh được thực hiện? 2.2. Chuyển đoạn lệnh trên sang ngôn ngữ C.

Hướng dẫn giải:

- **Giải câu a (Mã đề B1):**
  - *Ý 2.1:* Có 1 lệnh khởi tạo bên ngoài (`addi t1, zero, 100`) và 100 lần lặp, mỗi lần lặp thực hiện 5 lệnh trong thân `LOOP`. Tổng số lệnh được thực hiện là: $1 + 100 \times 5 =$ **501** lệnh.
  - *Ý 2.2:* Đoạn mã C tương ứng: `for (i = 100; i != 0; i-) { a = a + *A; A++; }` (hoặc viết theo chỉ số mảng: `for (i = 0; i < 100; i++) a = a + A[i];`).
- **Giải câu b (Mã đề B5):**
  - *Ý 2.1:* Mỗi vòng lặp địa chỉ trong `s0` tăng thêm 8 byte (tương ứng 2 phần tử 32-bit). Để tăng hết 600 byte, số lần thực hiện vòng lặp `LOOP` là $600 \div 8 = 75$ vòng. Mỗi vòng có 6 lệnh, cộng 1 lệnh khởi tạo ở ngoài là: $1 + 75 \times 6 =$ **451** lệnh.
  - *Ý 2.2:* Đoạn mã C tương ứng (duyệt 150 phần tử của mảng `B`, mỗi bước cộng 2 phần tử liên tiếp vào `b`): `for (i = 0; i < 150; i += 2) b = b + B[i] + B[i+1];`.

Bài tập 3.55 (Đề Cuối kỳ 2021.2 – Mã đề B2 Câu 3, Mã đề B3 Câu 3 & Mã đề B4 Câu 3)

a) **(Mã đề B2 & B3, Câu 3):** Cho đoạn chương trình vòng lặp sau:

```
LOOP: slt  t2, zero, t1
      bne  t2, zero, ELSE
      jal  x0, DONE
ELSE: addi s2, s2, 2
      addi t1, t1, -1
      jal  x0, LOOP
DONE:
```

3.1. Xác định giá trị thanh ghi `s2` sau khi chạy xong nếu khởi tạo: $\texttt{t1} = 20, \texttt{s2} = 0$ (Đề B2) hoặc $\texttt{t1} = 18, \texttt{s2} = 0$ (Đề B3).

3.2. Nếu `t1` khởi tạo bằng N (N nguyên dương), có bao nhiêu lệnh được thực hiện?

b) **(Mã đề B4, Câu 3):** Cho đoạn chương trình 2 vòng lặp lồng nhau:

```
LOOP:  addi t2, zero, 0xA
LOOP2: addi s2, s2, 14
       addi t2, t2, -1
       bne  t2, x0, LOOP2
       addi t1, t1, -1
       bne  t1, x0, LOOP
DONE:
```

3.1. Giả sử ban đầu $\texttt{t1} = 18, \texttt{s2} = 0$. Xác định giá trị của `s2` sau khi thực hiện xong.

3.2. Nếu `t1` khởi tạo bằng N (N nguyên dương), có bao nhiêu lệnh được thực hiện?

Hướng dẫn giải:

- **Giải câu a (Mã đề B2 & B3):**
  - *Ý 3.1:* Chừng nào $0 < \texttt{t1}$ ($\texttt{t1} > 0$), chương trình nhảy vào `ELSE` cộng thêm 2 vào `s2` và giảm `t1` đi 1.
    - Với $\texttt{t1} = 20$ (Đề B2): $\texttt{s2} = 20 \times 2 = \mathbf{40}$.
    - Với $\texttt{t1} = 18$ (Đề B3): $\texttt{s2} = 18 \times 2 = \mathbf{36}$.
  - *Ý 3.2:* Trong N lần lặp đầu tiên (khi $\texttt{t1} = N, N-1, \ldots, 1$), mỗi lần thực hiện 5 lệnh (`slt, bne, addi s2, addi t1, jal LOOP`). Ở lần cuối cùng khi $\texttt{t1} = 0$, chương trình thực hiện thêm 3 lệnh (`slt, bne, jal DONE`) để thoát. Vậy tổng số lệnh thực hiện là: $\mathbf{5N + 3}$ lệnh.
- **Giải câu b (Mã đề B4):**
  - *Ý 3.1:* Vòng lặp trong (`LOOP2`) lặp `0xA` $= 10$ lần, mỗi lần cộng 14 vào `s2` (tức mỗi vòng ngoài tăng $10 \times 14 = 140$). Với $\texttt{t1} = 18$ vòng ngoài, giá trị cuối của `s2` là: $18 \times 140 = \mathbf{2520}$.

– *Ý 3.2:* Trong mỗi vòng lặp ngoài (`LOOP`), số lệnh thực hiện gồm: 1 lệnh (`addi t2`) $+10 \times 3$ lệnh (của `LOOP2`) $+2$ lệnh cuối (`addi t1, bne t1`) $= 33$ lệnh. Lặp N vòng ngoài sẽ thực hiện tổng cộng: **33N** lệnh.

Bài tập 3.56 (Đề Cuối kỳ 2021.2 – Mã đề B6 Câu 2 & Mã đề B7 Câu 2)
Cho đoạn chương trình hợp ngữ dưới đây:

```
    addi t1, zero, K
    add  s2, x0, x0
LOOP:
    slt  t2, x0, t1
    beq  t2, x0, DONE
    addi s2, s2, M
    slli s2, s2, L
    addi t1, t1, -1
    jal  x0, LOOP
DONE:
```

Hãy xác định (2.1) Tổng số lệnh được thực hiện, và (2.2) Giá trị của thanh ghi `s2` sau khi kết thúc đoạn lệnh trên trong 2 mã đề:

a) **(Mã đề B6):** $K = 5, M = 3, L = 2$.

b) **(Mã đề B7):** $K = 7, M = 1, L = 1$.

Hướng dẫn giải:

- **Công thức đếm số lệnh tổng quát với K vòng lặp:** Có 2 lệnh khởi tạo ở ngoài $+K \times 6$ lệnh trong thân `LOOP` $+2$ lệnh kiểm tra thoát (`slt, beq`) khi `t1` $= 0 \implies$ Tổng số lệnh $= 6K + 4$.
- **Giải câu a (Mã đề B6: $K = 5, M = 3, L = 2$):**
  - *Ý 2.1:* Tổng số lệnh thực hiện $= 6 \times 5 + 4 =$ **34** lệnh.
  - *Ý 2.2:* Trong mỗi vòng lặp, `s2` được cộng thêm 3 rồi dịch trái 2 bit (nhân 4): $\texttt{s2}_{\text{mới}} = (\texttt{s2}_{\text{cũ}} + 3) \times 4$:
    - Vòng 1 (`t1` $= 5$): `s2` $= (0 + 3) \times 4 = 12$.
    - Vòng 2 (`t1` $= 4$): `s2` $= (12 + 3) \times 4 = 60$.
    - Vòng 3 (`t1` $= 3$): `s2` $= (60 + 3) \times 4 = 252$.
    - Vòng 4 (`t1` $= 2$): `s2` $= (252 + 3) \times 4 = 1020$.
    - Vòng 5 (`t1` $= 1$): `s2` $= (1020 + 3) \times 4 =$ **4092**.
- **Giải câu b (Mã đề B7: $K = 7, M = 1, L = 1$):**

- *Ý 2.1:* Tổng số lệnh thực hiện $= 6 \times 7 + 4 =$ **46** lệnh.
- *Ý 2.2:* Trong mỗi vòng lặp, $\texttt{s2}_{\text{mới}} = (\texttt{s2}_{\text{cũ}} + 1) \times 2$:
  - Vòng 1: 2; Vòng 2: 6; Vòng 3: 14; Vòng 4: 30; Vòng 5: 62; Vòng 6: 126; Vòng 7: $(126 + 1) \times 2 =$ **254**.

3.2.7 Nhóm 7: Chương trình con (Thủ tục), Ngăn xếp (Stack) và Lệnh `jal / jalr`

Bài tập 3.57 (Đề Giữa kỳ 2024.1 – #5ebf59 & Ôn tập Cuối kỳ 2025.2 – P1, Câu 50)
Những lệnh nào lưu địa chỉ của $PC + 4$ vào một thanh ghi trước khi thực hiện rẽ nhánh / nhảy trong RISC-V?

A. `jal`

B. `beq`

C. `j`

D. `jr`

E. `jalr`

Hướng dẫn giải:

- Hai lệnh nhảy và liên kết (Jump and Link) trong RISC-V thực hiện lưu địa chỉ trở về $PC + 4$ vào thanh ghi đích trước khi nhảy là `jal` và `jalr`.
- **Kết luận:** Chọn cả **A** (`jal`) và **E** (`jalr`).

Bài tập 3.58 (Đề Cuối kỳ 2022.2 – #a3c257, 2024.1 – #e79a4d, 2024.2 & Ôn tập 2025.2 – P2, Câu 31)
Cho thân chương trình con hợp ngữ RISC-V gồm có 3 đoạn lệnh như dưới đây:

```
# Doan lenh cat noi dung s2 (x18) va s3 (x19) ra ngan xep:
    addi sp, sp, -8
    sw   s3, 4(sp)
    sw   s2, 0(sp)
# Doan lenh khong lien quan den sp va ngan xep:
    ...
# Doan lenh khoi phuc lai noi dung s2, s3 va sp:
```

Hãy chỉ ra đoạn lệnh phù hợp để khôi phục lại nội dung `s2, s3` và `sp`?

A. `lw s2, 0(sp)` → `lw s3, 4(sp)` → `addi sp, sp, 8`

B. `addi sp, sp, 8` → `lw s3, 4(sp)` → `lw s2, 0(sp)`

C. `lw s2, 4(sp)` → `lw s3, 0(sp)` → `addi sp, sp, 8`

D. `addi sp, sp, 8` → `lw s3, 0(sp)` → `lw s2, 4(sp)`

Hướng dẫn giải:

- Lúc cất vào ngăn xếp, `s2` được lưu tại `0(sp)` và `s3` được lưu tại `4(sp)`.
- Khi khôi phục, phải đọc (`lw`) `s2` từ `0(sp)` và `s3` từ `4(sp)` ra trước, sau đó mới cộng trả lại 8 byte cho con trỏ `sp` (`addi sp, sp, 8`).
- **Kết luận:** Đáp án đúng là **A (Đoạn 1)**.

Bài tập 3.59 (Đề Cuối kỳ 2022.2 – #9aec2f, 2024.1 – #f023de, 2024.2 & Ôn tập 2025.2 – P2, Câu 65)
Cho đoạn lệnh như dưới đây, bên trái là địa chỉ của lệnh tương ứng:

```
0x00402034        jal  ra, XYZ
0x00402038        ...
0x00402060  XYZ:  addi sp, sp, -8
    ...           ...
0x00402090        ret               # (tuc jalr x0, 0(ra))
```

Hãy cho biết các giá trị sau (viết theo số Hexa 8 chữ số, ví dụ `0x01234567`):

a) Nội dung các thanh ghi `PC` và `ra` (`x1`) ngay sau khi thực hiện lệnh `jal ra, XYZ`.

b) Nội dung thanh ghi `PC` ngay sau khi thực hiện lệnh `ret` (`jalr x0, 0(ra)`).

Hướng dẫn giải:

- **Giải câu a:** Khi thực hiện lệnh `jal ra, XYZ` tại địa chỉ `0x00402034`:
  - Thanh ghi `PC` nhảy tới địa chỉ của nhãn `XYZ`: `PC = 0x00402060`.
  - Thanh ghi `ra` (`x1`) lưu địa chỉ của lệnh kế tiếp ($PC + 4$): `ra = 0x00402038`.
- **Giải câu b:** Khi thực hiện lệnh `ret` (`jalr x0, 0(ra)`), giá trị trong thanh ghi `ra` được nạp trả lại cho `PC`: `PC = 0x00402038`.

Bài tập 3.60 (Đề Cuối kỳ 2022.2 – #526d03, 2024.1 – #115355 & 2024.2)
Cho chương trình con `count` sau:

```
count:  addi s0, zero, 0
L1:     beq  a0, zero, done
        andi t0, a0, 1
        beq  t0, zero, L2
        addi s0, s0, 1
L2:     srli a0, a0, 1
        j    L1
done:   add  a0, s0, zero
        ret
```

a) Giả sử chương trình con được dịch ra mã máy và nạp vào bộ nhớ. Khi thực hiện lệnh `beq t0, zero, L2` mà nội dung thanh ghi `t0` = 0 thì sau khi hoàn thành lệnh đó, nội dung thanh ghi `PC` nhận giá trị bao nhiêu? (Xét cả 2 trường hợp đề bài: TH1 đặt lệnh đầu tiên `count` ở `0x00400100`; TH2 cắt từ dòng `beq t0, zero, L2` làm dòng đầu tiên ở `0x00400100`).

b) Giả thiết trước khi thực hiện chương trình con `count`, thanh ghi `a0` = `0x5678ABCD`. Hãy cho biết nội dung kết quả trả về tại `a0` (hoặc `$v0`) sau khi thực hiện chương trình con đó (viết theo số thập phân)?

Hướng dẫn giải:

- **Giải câu a:** Khi `t0` = 0, điều kiện của lệnh `beq t0, zero, L2` thỏa mãn nên chương trình rẽ nhánh đến nhãn `L2` (bỏ qua lệnh `addi s0, s0, 1`), tức là `PC` tăng thêm 2 lệnh = 8 byte so với địa chỉ của lệnh `beq t0, zero, L2`:
  - Nếu đề bài cắt từ dòng `beq t0, zero, L2` là dòng đầu tiên ở `0x00400100` (như bản in trang 10 đề 2024.2): Nhãn `L2` nằm ở `0x00400100` + 8 = `0x00400108`.
  - Nếu đề bài in đầy đủ từ dòng `count: addi s0, zero, 0` ở `0x00400100`: Lệnh `beq t0, zero, L2` là lệnh thứ 4 (ở `0x0040010C`), nên nhãn `L2` (lệnh thứ 6) nằm ở `0x00400114`.
- **Giải câu b:** Thuật toán dùng `andi t0, a0, 1` để kiểm tra từng bit của `a0` từ phải sang trái; nếu bit bằng 1 thì tăng bộ đếm `s0` lên 1. Vậy hàm trả về **tổng số bit 1** trong biểu diễn nhị phân của `a0` = `0x5678ABCD`:
  - `5`(0101_2) → 2 bit 1; `6`(0110_2) → 2 bit 1; `7`(0111_2) → 3 bit 1; `8`(1000_2) → 1 bit 1.
  - `A`(1010_2) → 2 bit 1; `B`(1011_2) → 3 bit 1; `C`(1100_2) → 2 bit 1; `D`(1101_2) → 3 bit 1.
  - Tổng cộng: $2+2+3+1+2+3+2+3 = \mathbf{18}$.

3.2.8 Nhóm 8: Dịch Mã máy – Hợp ngữ, Tính địa chỉ đích và Giải mã Bản đồ bộ nhớ (Memory Map)

Bài tập 3.61 (Ôn tập Cuối kỳ 2025.2 – P1, Câu 65, 19, 78, 22, 11, 14, 5)
Hãy dịch 7 câu lệnh hợp ngữ RISC-V dưới đây sang mã máy biểu diễn ở hệ Hexa (8 chữ số, bắt buộc có `0x` ở đầu):

a) **(Câu 65 – Kiểu R):** `sub t0, t2, t1`

b) **(Câu 19 – Kiểu I tính toán):** `addi s2, s1, -14`

c) **(Câu 78 – Kiểu I đọc bộ nhớ):** `lh t5, -14(s1)` (và biến thể `lh t1, -13(s2)`)

d) **(Câu 22 – Kiểu S ghi bộ nhớ):** `sw s11, -16(s4)`

e) **(Câu 11 – Kiểu B rẽ nhánh xuôi):** `beq s0, t5, L1` (biết địa chỉ lệnh là `0x00401050`, địa chỉ `L1` là `0x00401060`).

f) **(Câu 14 – Kiểu B rẽ nhánh ngược):** `bne t0, s0, L2` (biết địa chỉ lệnh là `0x0040008C`, địa chỉ `L2` là `0x00400070`).

g) **(Câu 5 – Kiểu J nhảy):** `jal x0, L1` (biết địa chỉ lệnh là `0x004030AC`, địa chỉ `L1` là `0x00403074`).

Hướng dẫn giải:

- **Giải câu a (`sub t0, t2, t1` – Kiểu R):**
  - Số hiệu thanh ghi: `rd` = `t0` = `x5` (`00101`); `rs1` = `t2` = `x7` (`00111`); `rs2` = `t1` = `x6` (`00110`).
  - Lệnh `sub` có `funct7` = `0100000`, `funct3` = `000`, `opcode` = `0110011`.
  - Ghép 32 bit (`funct7 | rs2 | rs1 | funct3 | rd | op`):

$$\texttt{0100000_00110_00111_000_00101_0110011}_{(2)} = \texttt{0x406382B3}$$

- **Giải câu b (`addi s2, s1, -14` – Kiểu I):**
  - Số hiệu: `rd` = `s2` = `x18` (`10010`); `rs1` = `s1` = `x9` (`01001`); `funct3` = `000`; `op` = `0010011`.
  - Đổi `imm` $= -14$ sang bù 2 12-bit: $+14 = \texttt{0000 0000 1110}_2 \implies -14 = \texttt{1111 1111 0010}_2 = \texttt{0xFF2}$.
  - Ghép 32 bit (`imm[11:0] | rs1 | funct3 | rd | op`):

$$\texttt{1111 1111 0010_01001_000_10010_0010011}_{(2)} = \texttt{0xFF248913}$$

- **Giải câu c (`lh t5, -14(s1)` – Kiểu I Load):**

- Số hiệu: `rd` = `t5` = `x30` (11110); `rs1` = `s1` = `x9` (01001); `funct3` = `001` (`lh`); `op` = `0000011`; `imm` = $-14 = 1111\,1111\,0010_2$.
- Ghép 32 bit: $1111\,1111\,0010_01001_001_11110_0000011_{(2)} =$ `0xFF249F03`.
- (Với biến thể `lh t1, -13(s2)`: `imm` = $-13 = 1111\,1111\,0011_2$, `rs1` = $18 = 10010_2$, `rd` = $6 = 00110_2 \implies$ `0xFF391303`).

- **Giải câu d (`sw s11, -16(s4)` – Kiểu S Store):**
  - Số hiệu: `rs2` = `s11` = `x27` (11011); `rs1` = `s4` = `x20` (10100); `funct3` = `010`; `op` = `0100011`.
  - Đổi `imm` = -16 sang bù 2 12-bit: $1111\,1111\,0000_2 \implies$ `imm[11:5]` = `1111111`, `imm[4:0]` = `10000`.
  - Ghép 32 bit (`imm[11:5] | rs2 | rs1 | funct3 | imm[4:0] | op`):

$$1111111_11011_10100_010_10000_0100011_{(2)} = \texttt{0xFFBA2823}$$

- **Giải câu e (`beq s0, t5, L1` – Kiểu B):**
  - Độ lệch `imm` = `0x00401060` − `0x00401050` = `0x10` = $+16 = 0_0_000000_1000_0_{(2)}$ (từ bit 12 đến bit 0).
  - Số hiệu: `rs1` = `s0` = 8 (01000); `rs2` = `t5` = 30 (11110); `funct3` = `000`; `op` = `1100011`.
  - Ghép theo B-type (`imm[12,10:5] | rs2 | rs1 | funct3 | imm[4:1,11] | op`):

$$0_000000_11110_01000_000_1000_0_1100011_{(2)} = \texttt{0x01E40863}$$

- **Giải câu f (`bne t0, s0, L2` – Kiểu B):**
  - Độ lệch `imm` = `0x00400070` − `0x0040008C` = $-$`0x1C` $= -28 = 1_1_111111_0010_0_{(2)}$ (từ bit 12 đến bit 0).
  - Số hiệu: `rs1` = `t0` = 5 (00101); `rs2` = `s0` = 8 (01000); `funct3` = `001`; `op` = `1100011`.
  - Ghép theo B-type: $1_111111_01000_00101_001_0010_1_1100011_{(2)} =$ `0xFE8292E3`.

- **Giải câu g (`jal x0, L1` – Kiểu J):**
  - Độ lệch `imm` = `0x00403074`−`0x004030AC` = $-$`0x38` $= -56 = 1_11111111_1_1111100100_0_{(2)}$ (từ bit 20 đến bit 0).
  - Ghép theo J-type (`imm[20,10:1,11,19:12] | rd | op`):

$$1_1111100100_1_11111111_00000_1101111_{(2)} = \texttt{0xFC9FF06F}$$

Bài tập 3.62 (Đề Cuối kỳ 2024.1 – #b9e098 & Ôn tập Cuối kỳ 2025.2 – P1, Câu 8)

Cho lệnh máy RISC-V được biểu diễn theo số Hexa: `0x012B2423`. Hãy chỉ ra lệnh hợp ngữ tương ứng:

A. `add t0, s4, s5`

B. `lw t2, 4(s6)`

C. `sw s2, 8(s6)`

D. `addi t0, s3, -4`

Hướng dẫn giải:

- Đổi 2 chữ số Hexa cuối $23_{16} = 0010\,0011_2 \implies$ 7 bit opcode $= 0100011_2 = 35_{10}$ (Nhóm lệnh Store kiểu S).
- Tách 32 bit theo khuôn dạng S-type (`imm[11:5] | rs2 | rs1 | funct3 | imm[4:0] | op`):

$$\texttt{0x012B2423} = \underbrace{0000000}_{\texttt{imm[11:5]=0}} \mid \underbrace{10010}_{\texttt{rs2=18(s2)}} \mid \underbrace{10110}_{\texttt{rs1=22(s6)}} \mid \underbrace{010}_{\texttt{funct3=2(sw)}} \mid \underbrace{01000}_{\texttt{imm[4:0]=8}} \mid \underbrace{0100011}_{\text{opcode}}$$

- Vậy câu lệnh hợp ngữ là: `sw s2, 8(s6)` → Chọn **C**.

Bài tập 3.63 (Đề Cuối kỳ 2022.2 & Cuối kỳ 2024.2 – #b7c77a)
Cho lệnh máy được biểu diễn theo số Hexa: `0xFFC98293` (trong đề cũ chuyển từ MIPS là `0x2268FFFC`). Hãy chỉ ra lệnh hợp ngữ tương ứng:

A. `lw t2, 4(s6)`

B. `add t0, s4, s5`

C. `sw s2, 8(s6)`

D. `addi t0, s3, -4`

Hướng dẫn giải:

- Trong mã máy `0xFFC98293` (hoặc `0x2268FFFC`): hằng số ở đầu/cuối là `0xFFC` $= -4$, thanh ghi nguồn là `s3` (`x19` $= 10011_2$), thanh ghi đích là `t0` (`x5` $= 00101_2$), phép toán là cộng hằng số `addi`.
- **Kết luận:** Đáp án đúng là **D** (`addi t0, s3, -4`).

Bài tập 3.64 (Đề Cuối kỳ 2024.1 – #3ad10f & Ôn tập Cuối kỳ 2025.2 – P1, Câu 62)
Giả thiết lệnh `bge s0, s1, L1` của RISC-V có mã máy là `0x02945063`, được đặt tại word nhớ có địa chỉ `0x00404014`. Hãy chỉ ra địa chỉ đích của lệnh đó (nếu điều kiện thỏa mãn). Biết BTA $= PC +$ SignExt($\{\texttt{imm}_{12:1}, \texttt{1'b0}\}$).

A. `0x0040401F`

B. `0x00404034`

C. `0x00404030`

D. `0x00404018`

Hướng dẫn giải:

- Đổi mã máy `0x02945063` sang nhị phân và tách theo B-type:

$$\underbrace{0_000001}_{\texttt{imm[12,10:5]}} \mid \underbrace{01001}_{\texttt{rs2=9}} \mid \underbrace{01000}_{\texttt{rs1=8}} \mid \underbrace{101}_{\texttt{funct3}} \mid \underbrace{0000_0}_{\texttt{imm[4:1,11]}} \mid \underbrace{1100011}_{\text{opcode}}$$

- Ghép lại hằng số 13-bit $\texttt{imm}_{12:0} = \{\texttt{imm}_{12}, \texttt{imm}_{11}, \texttt{imm}_{10:5}, \texttt{imm}_{4:1}, 0\}$:

$$\texttt{imm} = \texttt{0_0_000001_0000_0}_{(2)} = 0000\,0010\,0000_{(2)} = \texttt{0x20} = 32_{(10)}$$

- Địa chỉ đích BTA $= PC + \texttt{imm} = \texttt{0x00404014} + \texttt{0x20} = \texttt{0x00404034}$.
- **Kết luận:** Đáp án đúng là **B** (`0x00404034`).

Bài tập 3.65 (Đề Cuối kỳ 2020.2 – Trang 4, Giữa kỳ 2024.1 – #a2a36c & Ôn tập 2025.2 – Câu 47, 57)

a) **(Đề CK 2020.2 – Trang 4):** Lệnh `beq t3, s4, L1` trong RISC-V được đặt tại địa chỉ `0x00401000`, lệnh kế tiếp tại `0x00401004`. Biết nhãn đích `L1` nằm tại địa chỉ `0x0040104C`. Hãy xác định giá trị độ lệch byte (offset) và giá trị 12-bit `imm[12:1]` mã hóa trong lệnh.

b) **(Đề GK 2024.1 – #a2a36c & Ôn tập 2025.2 Câu 47):** Cho chương trình RISC-V bắt đầu từ địa chỉ `0x00400000`: (1) `addi s0, zero, 100`; (2) `slti t0, s0, -100`; (3) `beq t0, zero, else`; (4) `addi t2, t2, 1`; (5) `j endif`; (6) `else: addi t2, t2, -1`; (7) `endif: nop`. Giá trị thập phân của trường 12-bit immediate trong lệnh `beq t0, zero, else` là bao nhiêu?
A. 8 B. 12 C. 3 D. 6

c) **(Ôn tập 2025.2 – Câu 57):** Giả sử `PC` hiện tại (địa chỉ của lệnh `beq`) là `0x2000`. Với câu lệnh `beq x1, x2, -12`, nếu `x1` == `x2` thì địa chỉ lệnh tiếp theo (Target PC) là bao nhiêu?
A. `0x1FE8` B. `0x1FF8` C. `0x1FF4` D. `0x200C`

Hướng dẫn giải:

- **Giải câu a:** Độ lệch tính theo byte là: Offset = `0x0040104C` − `0x00401000` = `0x4C` = **76** byte. Nếu tính trường 12-bit `imm[12:1]` lưu trong mã máy (bỏ bit 0, tức chia 2) thì giá trị là $76 \div 2 =$ **38** (`0x26`); nếu tính số lệnh nhảy qua (mỗi lệnh 4 byte) là $76 \div 4 = 19$

(hoặc 18 nếu trừ $PC + 4$ theo đề cũ).

- **Giải câu b:** Lệnh (3) `beq` nằm ở địa chỉ `0x00400008`, nhãn `else` ở lệnh (6) nằm ở địa chỉ `0x00400014`. Độ lệch byte là `0x14` − `0x08` = 12 byte. Trường 12-bit `imm[12:1]` được mã hóa trong lệnh có giá trị thập phân là: $12 \div 2 = \mathbf{6} \rightarrow$ Chọn **D (6)**.
- **Giải câu c:** Target PC = `0x2000` − 12 = `0x2000` − `0x0C` = `0x1FF4` → Chọn **C**.

Bài tập 3.66 (Đề Cuối kỳ 2023.1 – #5a6d48, #e08504, 2025.1 – #794d71, #0c9d, Giữa kỳ 2024.1 – #1c2f2d, #65dce1 & Ôn tập 2025.2 – Câu 16, 60, 77)
Hãy xác định khuôn dạng lệnh, thanh ghi đích (`rd`) và các thanh ghi nguồn (`rs1, rs2`) cho các mã máy RISC-V dưới đây:

a) **(CK 2023.1 – #e08504):** Lệnh mã máy `0x00628863` (`bne t0, t1, 16`) thuộc định dạng nào (`R, I, B, J`)?

b) **(CK 2023.1 – #5a6d48):** Lệnh mã máy `0x00008067` (`jalr zero, ra, 0`) có toán hạng đích lưu tại đâu?

c) **(Ôn tập 2025.2 – P2, Câu 16):** Cho mã máy `0000000 00111 00000 000 11111 0110011`. Toán hạng đích của nó là gì?

d) **(GK 2024.1 – #1c2f2d & CK 2025.1 – #794d71):** Tìm toán hạng đích (`rd`) của lệnh R-format có mã máy: (1) `0x007302b3`; và (2) `0x01c303b3`.

e) **(GK 2024.1 – #65dce1 & CK 2025.1 – #0c9d):** Tìm các thanh ghi nguồn (`rs1, rs2`) của lệnh `beq` có mã máy: (1) `0x01ee8463`; và (2) `0x01de0463`.

Hướng dẫn giải:

- **Câu a:** Lệnh `bne` có opcode `0x63` = 1100011_2, thuộc định dạng **B-format (B-type)**.
- **Câu b:** Trong `jalr zero, ra, 0`, toán hạng đích là **thanh ghi** `zero` (`x0`).
- **Câu c:** Trường `rd` (5 bit đứng ngay trước 7 bit opcode) là $11111_2 = 31 \implies$ Thanh ghi `x31`, tức là **Thanh ghi** `t6`.
- **Câu d:** Đổi 3 chữ số Hexa cuối sang 12 bit nhị phân (`rd` 5 bit + `op` 7 bit):
  - Với `0x007302b3`: $2b3_{16} = \underbrace{00101}_{\texttt{rd=5}}\ \underbrace{0110011}_{\text{op}} \implies$ `x5` (`t0`).
  - Với `0x01c303b3`: $3b3_{16} = \underbrace{00111}_{\texttt{rd=7}}\ \underbrace{0110011}_{\text{op}} \implies$ `x7` (`t2`).
- **Câu e:** Trong lệnh `beq rs1, rs2, label`, bit 24:20 là `rs2` và bit 19:15 là `rs1`:
  - Với `0x01ee8463`: $01ee8_{16} = 0000000_\underbrace{11110}_{\texttt{rs2=30}}_\underbrace{11101}_{\texttt{rs1=29}}_000 \implies$ Nguồn `rs1, rs2` là `x29, x30`.

– Với `0x01de0463`: $\texttt{01de0}_{16} = \texttt{0000000_}\underbrace{\texttt{11101}}_{\texttt{rs2=29}}\texttt{_}\underbrace{\texttt{11100}}_{\texttt{rs1=28}}\texttt{_000} \implies$ Nguồn `rs1, rs2` là `x28, x29`.

Bài tập 3.67 (Đề Cuối kỳ 2025.1 – #15fd1c & Ôn tập Cuối kỳ 2025.2 – P2, Câu 48)

Cho chương trình hợp ngữ RISC-V dưới đây đã được nạp vào bộ nhớ chính. Giả sử lệnh đầu tiên của chương trình nằm tại địa chỉ `0x00400000`:

```
    lui  s0, 0xFFFF0          # 0x00400000
    addi s0, zero, 0          # 0x00400004
    slti t1, s0, 100          # 0x00400008
    beq  t1, zero, else       # 0x0040000C
    addi t2, zero, 1          # 0x00400010
    addi t3, zero, 20         # 0x00400014
    j    endif                # 0x00400018
else:
    addi t2, zero, -1         # 0x0040001C
    addi t3, zero, -20        # 0x00400020
endif: nop                    # 0x00400024
```

Giá trị hệ thập lục phân (Hexa) của thanh ghi `PC` ngay sau khi thực hiện lệnh `beq` là bao nhiêu?

Hướng dẫn giải:

- Lệnh thứ 2 (`addi s0, zero, 0`) gán lại `s0` $= 0 + 0 = 0$ (ghi đè giá trị của lệnh `lui` đầu tiên).
- Lệnh thứ 3 (`slti t1, s0, 100`) kiểm tra $0 < 100 \rightarrow$ Đúng, nên `t1` $= 1$.
- Lệnh thứ 4 (`beq t1, zero, else` tại địa chỉ `0x0040000C`) kiểm tra `t1` $== 0$ ($1 == 0 \rightarrow$ Sai), nên **không rẽ nhánh**.
- Do không rẽ nhánh, thanh ghi `PC` tăng thêm 4 để trỏ sang lệnh kế tiếp thứ 5:

$$PC = \texttt{0x0040000C} + 4 = \texttt{0x00400010}$$

Bài tập 3.68 (Giải chi tiết Đề Giữa kỳ 2025.2 – Đề 0013, Bản 1: Câu 67 đến Câu 77)

Cho bản đồ bộ nhớ (Memory Map) của hệ thống RISC-V 32-bit lưu trữ theo thứ tự đầu nhỏ (Little-Endian) từ địa chỉ `0x400000` đến `0x40001B` như bảng dưới đây:

Địa chỉ	4 byte (+0..+3)	Mã máy 32-bit	Giải mã sang lệnh hợp ngữ RISC-V
0x400000	B3, 02, 73, 00	0x007302B3	add t0, t1, t2 (rd = t0, rs1 = t1, rs2 = t2)
0x400004	13, 0E, 54, 00	0x00540E13	addi t3, s0, 5 (rd = t3, rs1 = s0, imm = 5)
0x400008	93, DE, 74, 00	0x0074DE93	srli t4, s1, 7 (dịch phải logic 7 bit)
0x40000C	23, 24, E9, 01	0x01E92423	sw t5, 8(s2) (rs2 = t5, rs1 = s2, imm = 8)
0x400010	E3, 88, 0F, FE	0xFE0F88E3	beq t6, x0, -16 (rs1 = t6, rs2 = x0)
0x400014	B7, 59, 34, 12	0x123459B7	lui s3, 0x12345 (rd = x19 = s3)
0x400018	EF, F0, 9F, FE	0xFE9FF0EF	jal ra, -24 (rd = x1 = ra)

Hãy trả lời các câu hỏi trong Đề 0013 (Bản 1):

Câu 67. Hãy cho biết giá trị số 32-bit ở địa chỉ 0x400000 ở hệ 16.

Câu 68. Sau đoạn lệnh: lui s0, 0x400 → addi s0, s0, 0x18 → lh t1, 0(s0), hai thanh ghi s0 và t1 bằng bao nhiêu?

Câu 69. Lệnh ở địa chỉ 0x400014 có thanh ghi đích rd là gì?

Câu 70. Lệnh thứ 3 trong bản đồ (tại 0x400008) là lệnh tương đương với phép chia. Đó là phép chia với hằng số bằng mấy?

Câu 71. Hằng số của lệnh ở địa chỉ 0x400008 bằng mấy (viết ở hệ 16)?

Câu 72. Hãy cho biết thao tác của lệnh ở địa chỉ 0x400010 là gì (ví dụ: add, bne, lui)?

Hướng dẫn giải:

- **Câu 67:** Ghép ngược 4 byte từ 0x400003 về 0x400000 theo Little-Endian: 0x007302B3.
- **Câu 68:** Sau 2 lệnh đầu, s0 = 0x00400018. Lệnh lh t1, 0(s0) đọc 2 byte tại 0x400018 (0xEF) và 0x400019 (0xF0), ghép lại là 0xF0EF, mở rộng dấu âm 32-bit thành: t1 = 0xFFFFF0EF.
- **Câu 69:** Lệnh ở 0x400014 là 0x123459B7. Đổi 3 chữ cuối $\texttt{9B7}_{16} = \underbrace{10011}_{\texttt{rd=19}}\ \underbrace{0110111}_{\text{lui}}$. Thanh ghi x19 có tên là s3.
- **Câu 70:** Lệnh thứ 3 ở 0x400008 là 0x0074DE93 (srli t4, s1, 7). Dịch phải 7 bit tương đương chia cho $2^7 = \mathbf{128}$.
- **Câu 71:** Hằng số dịch của lệnh 0x0074DE93 là 12 bit đầu 0x007 $\implies$ Trả lời: 0x7 (hoặc 0x007).
- **Câu 72:** Lệnh ở 0x400010 (0xFE0F88E3) có opcode 1100011_2 (Branch) và funct3 = 000_2 $\implies$ Thao tác là beq.

Bài tập 3.69 (Giải chi tiết Đề Giữa kỳ 2025.2 – Đề 0013, Bản 2: Câu 936 đến Câu 946)

Sử dụng cùng bản đồ bộ nhớ của Đề 0013 ở Bài tập 3.68 (nhưng tại địa chỉ 0x40000A đổi byte 0x74 thành 0x34, tức mã máy tại 0x400008 là 0x0034DE93), hãy trả lời các câu hỏi sau:

Câu 936. Giá trị số 32-bit ở địa chỉ `0x400018` ở hệ 16 là bao nhiêu?

Câu 937. Sau đoạn lệnh: `lui s0, 0x400` → `addi s0, s0, 0x16` → `lh t1, 0(s0)`, hai thanh ghi `s0` và `t1` bằng bao nhiêu?

Câu 938. Lệnh ở địa chỉ `0x400000` có thanh ghi nguồn `rs1` là gì?

Câu 939. Lệnh thứ 3 (tại `0x400008`: `0x0034DE93`) tương đương với phép chia cho hằng số bằng mấy?

Câu 940. Trong bản đồ, lệnh `beq` duy nhất nằm ở địa chỉ nào?

Câu 941. Hãy cho biết thao tác của lệnh ở địa chỉ `0x400008` là gì?

Hướng dẫn giải:

- **Câu 936:** Ghép ngược 4 byte từ `0x40001B` về `0x400018` được: `0xFE9FF0EF`.
- **Câu 937:** Sau 2 lệnh đầu, `s0 = 0x00400016`. Lệnh `lh t1, 0(s0)` đọc 2 byte tại `0x400016` (`0x34`) và `0x400017` (`0x12`), ghép lại thành `0x1234`, mở rộng dấu dương được: `t1 = 0x00001234`.
- **Câu 938:** Lệnh ở `0x400000` (`0x007302B3`) có `rs1` ở bit 19:15 là $00110_2 = 6$, tức là thanh ghi `t1` (`x6`).
- **Câu 939:** Mã máy `0x0034DE93` là lệnh `srli t4, s1, 3`. Dịch phải 3 bit tương đương chia cho $2^3 = \mathbf{8}$.
- **Câu 940:** Lệnh `beq` duy nhất nằm ở địa chỉ `0x400010`.
- **Câu 941:** Lệnh ở `0x400008` (`0x0034DE93`) có opcode 0010011_2, `funct3` $= 101_2$, 7 bit cao bằng 0 $\implies$ Thao tác là `srli`.

Bài tập 3.70 (Giải chi tiết Đề Giữa kỳ 2025.2 – Đề 0012: Câu 100 đến Câu 109)
Cho bản đồ bộ nhớ của Đề 0012 từ địa chỉ `0x400000` đến `0x40001B` như bảng dưới đây:

Địa chỉ	4 byte (+0..+3)	Mã máy 32-bit	Giải mã sang lệnh hợp ngữ RISC-V
`0x400000`	`33, 0E, 73, 40`	`0x40730E33`	`sub t3, t1, t2` (`rd` = `t3`, `rs1` = `t1`, `rs2` = `t2`)
`0x400004`	`37, 0B, 04, 00`	`0x00040B37`	`lui s6, 0x40` (U-type: `rd` = `x22` = `s6`)
`0x400008`	`13, 0B, 8B, 03`	`0x038B0B13`	`addi s6, s6, 56` (I-type: `imm` = 56 = `0x38`)
`0x40000C`	`E3, 0C, CB, FF`	`0xFFCB0CE3`	`beq s6, s11, -8` (B-type)
`0x400010`	`33, 83, C3, 01`	`0x01C38333`	`add t1, t2, t3` (R-type)
`0x400014`	`6F, F0, 1F, FF`	`0xFF1FF06F`	`jal x0, -16` (J-type)
`0x400018`	`13, 1B, 2E, 00`	`0x002E1B13`	`slli s6, t3, 2` (dịch trái 2 bit, nhân 4)

Hãy trả lời các câu hỏi trong Đề 0012:

Câu 100. Hãy cho biết giá trị số 32-bit ở địa chỉ `0x400008` ở hệ 16.

Câu 101. Sau đoạn lệnh: `lui s0, 0x400` → `addi s0, s0, 0x16` → `lb t1, 0(s0)`, hai thanh ghi `s0` và `t1` bằng bao nhiêu?

Câu 102. Lệnh ở địa chỉ `0x400000` (`0x40730E33`) có thanh ghi nguồn `rs2` là gì?

Câu 103. Lệnh cuối cùng trong bản đồ (tại `0x400018`: `0x002E1B13`) tương đương với phép nhân với hằng số bằng mấy?

Câu 104. Lệnh dạng U ở địa chỉ `0x400004` (`0x00040B37`) có toán hạng hằng số bằng mấy (viết ở hệ 16)?

Câu 105. Hãy cho biết thao tác của lệnh ở địa chỉ `0x400004` là gì?

Câu 106. Giải thích nào về lệnh `bne t1, t2, -16` là đúng?

A. Nếu `t1` $\neq$ `t2` rẽ nhánh về 4 lệnh sau B. Nếu `t1` $\neq$ `t2` rẽ nhánh về 4 lệnh trước

C. Nếu `t1` == `t2` rẽ nhánh về 4 lệnh sau D. Nếu `t1` == `t2` rẽ nhánh về 4 lệnh trước

Hướng dẫn giải:

- **Câu 100:** Tại `0x400008` có 4 byte `0x13, 0x0B, 0x8B, 0x03`. Ghép ngược theo Little-Endian được: `0x038B0B13`.
- **Câu 101:** `s0 = 0x00400016`. Tại địa chỉ `0x400016` của Đề 0012 chứa byte `0x1F` (số dương), nên sau lệnh `lb t1, 0(s0)` ta có: `t1 = 0x0000001F`.
- **Câu 102:** Trong `0x40730E33`, 12 bit đầu là $407_{16} = \underbrace{0100000}_{\texttt{funct7}}\ \underbrace{00111}_{\texttt{rs2=7}}$. Thanh ghi `x7` là `t2`.
- **Câu 103:** Lệnh ở `0x400018` (`0x002E1B13`) là lệnh `slli s6, t3, 2`. Dịch trái 2 bit tương đương nhân với $2^2 = \mathbf{4}$.
- **Câu 104:** Lệnh dạng U ở `0x400004` (`0x00040B37`) có 20 bit cao (5 chữ số Hexa đầu) là `0x40` (hoặc `0x00040`).
- **Câu 105:** Opcode của `0x00040B37` là $0110111_2 = 55_{10} \implies$ Thao tác là `lui`.
- **Câu 109:** Lệnh `bne` kiểm tra khác nhau (`t1` $\neq$ `t2`), và độ lệch -16 byte $= -4 \times 4$ byte tức là lùi về **4 lệnh trước lệnh hiện thời** → Chọn **B**.

CHƯƠNG 4

SỐ HỌC MÁY TÍNH

4.1 Phần 1: Lý thuyết trọng tâm

4.1.1 Hệ đếm và Chuyển đổi giữa các hệ đếm

1. Ba hệ đếm cơ bản trong máy tính

- **Hệ thập phân (Decimal – Cơ số 10):** Gồm 10 chữ số từ 0 đến 9. Dùng n chữ số thập phân biểu diễn được 10^n giá trị khác nhau (từ 0 đến $10^n - 1$).
- **Hệ nhị phân (Binary – Cơ số 2):** Gồm 2 chữ số là 0 và 1 (mỗi chữ số nhị phân gọi là 1 bit – đơn vị thông tin nhỏ nhất). Một nhóm 8 bit tạo thành 1 byte ($2^8 = 256$ giá trị). Dùng n bit có thể biểu diễn được 2^n **giá trị** khác nhau.
- **Hệ mười sáu (Hexadecimal – Cơ số 16):** Gồm 16 ký tự: `0-9` và `A` (10), `B` (11), `C` (12), `D` (13), `E` (14), `F` (15). Hệ Hexa dùng để viết gọn cho số nhị phân: cứ **1 nhóm 4-bit nhị phân** (tính từ dấu phẩy sang hai bên) được thay bằng đúng **1 chữ số Hexa**.

Bảng quy đổi chuẩn giữa 4-bit Nhị phân, Hệ Hexa và Hệ Thập phân

Hệ 2	Hex	Hệ 10	Hệ 2	Hex	Hệ 10	Hệ 2	Hex	Hệ 10	Hệ 2	Hex	Hệ 10
0000	0	0	0100	4	4	1000	8	8	1100	C	12
0001	1	1	0101	5	5	1001	9	9	1101	D	13
0010	2	2	0110	6	6	1010	A	10	1110	E	14
0011	3	3	0111	7	7	1011	B	11	1111	F	15

2. Công thức giá trị và Quy tắc chuyển đổi giữa Hệ 10 và Hệ 2

- **Dạng tổng quát của số nhị phân có cả phần nguyên và phần lẻ:**

$$A = a_n a_{n-1} \dots a_1 a_0 . a_{-1} a_{-2} \dots a_{-m(2)} = \sum_{i=-m}^{n} a_i \times 2^i$$

Trong đó các bit phần nguyên có trọng số là $2^0 = 1, 2^1 = 2, 2^2 = 4, 2^3 = 8, 2^4 = 16, 2^5 =$

$32, 2^6 = 64, 2^7 = 128 \ldots$; các bit phần lẻ sau dấu phẩy có trọng số là:

$$2^{-1} = 0.5 \quad ; \quad 2^{-2} = 0.25 \quad ; \quad 2^{-3} = 0.125 \quad ; \quad 2^{-4} = 0.0625 \quad ; \quad 2^{-5} = 0.03125$$

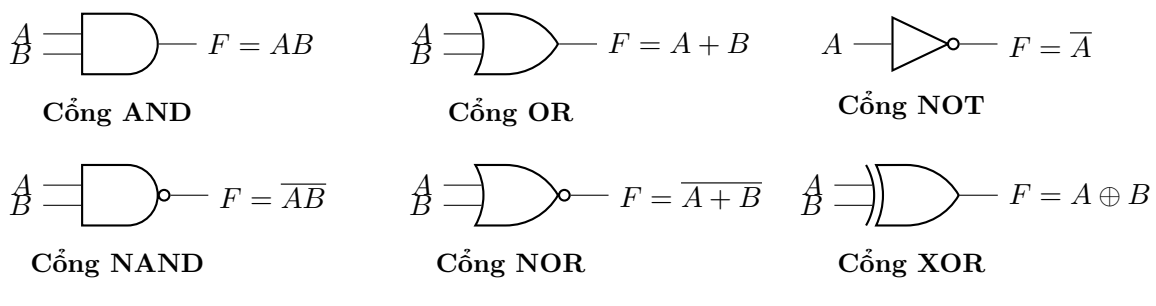
- **Chuyển đổi số thập phân sang nhị phân:**
  - *Đối với phần nguyên:* Chia liên tiếp cho 2 rồi viết các số dư theo thứ tự ngược từ dưới lên (hoặc phân tích thành tổng các lũy thừa 2^i giảm dần: $128, 64, 32, 16, 8, 4, 2, 1$).
  - *Đối với phần lẻ thập phân:* Nhân liên tiếp phần lẻ với 2, lấy phần nguyên (0 hoặc 1) sau mỗi lần nhân viết theo thứ tự từ trên xuống dưới.
  - *Lưu ý về số thực không thể biểu diễn chính xác tuyệt đối trong hệ nhị phân:* Một số lẻ thập phân chỉ biểu diễn chính xác hữu hạn trong hệ nhị phân nếu phần lẻ của nó viết được dưới dạng phân số tối giản có mẫu số là lũy thừa của 2 ($\frac{k}{2^m}$, như $0.5 = \frac{1}{2}, 0.25 = \frac{1}{4}, 0.75 = \frac{3}{4}, 0.125 = \frac{1}{8}, 0.375 = \frac{3}{8}, 0.625 = \frac{5}{8}$). Các số như $0.1, 0.2, 0.3, 0.4, 0.6, 0.7, 0.8, 0.9$ khi nhân đôi liên tiếp sẽ lặp vô hạn tuần hoàn nên **không thể biểu diễn chính xác tuyệt đối** bằng số dấu phẩy động nhị phân.

4.1.2 Đại số Boolean, Cổng logic và Mạch số cơ bản

1. Các phép toán Đại số Boolean và Cổng logic cơ bản

- **Ba phép toán cơ bản:** NOT ($\overline{A}$ hoặc A'), AND ($A \cdot B$ hoặc AB), OR ($A + B$). Thứ tự ưu tiên: NOT > AND > OR.
- **Ba phép toán mở rộng:**
  - NAND (NOT-AND): $F = \overline{A \cdot B}$ (Chỉ bằng 0 khi cả $A = 1$ và $B = 1$).
  - NOR (NOT-OR): $F = \overline{A + B}$ (Chỉ bằng 1 khi cả $A = 0$ và $B = 0$).
  - XOR (Exclusive-OR): $F = A \oplus B = \overline{A}B + A\overline{B}$ (Bằng 1 khi hai đầu vào khác nhau; bằng 0 khi hai đầu vào giống nhau).
- **Định lý De Morgan:** $\overline{A \cdot B} = \overline{A} + \overline{B}$ và $\overline{A + B} = \overline{A} \cdot \overline{B}$.

Ký hiệu hình vẽ chuẩn của 6 cổng logic cơ bản



2. Các mạch tổ hợp và mạch dãy cơ bản trong máy tính

- **Mạch tổ hợp (Combinational Circuits – mạch không nhớ):** Giá trị đầu ra chỉ phụ thuộc vào tổ hợp giá trị đầu vào tại thời điểm hiện tại. Các bộ phận điển hình:
  - *Bộ ghép kênh (MUX – Multiplexer):* Có 2^n đầu vào dữ liệu, n đầu vào chọn (S) và 1 đầu ra. Với MUX 2-ra-1: $F = D_0 \cdot \overline{S} + D_1 \cdot S$ (khi $S = 0$ thì nối D_0 ra F; khi $S = 1$ thì nối D_1 ra F).
  - *Bộ giải mã (Decoder):* Có n đầu vào và 2^n đầu ra. Với mỗi tổ hợp của n đầu vào, chỉ có duy nhất 1 đầu ra tích cực.
  - *Bộ cộng bán phần 1-bit (Half Adder):* Cộng 2 bit A, B tạo ra bit tổng $S = A \oplus B$ (dùng cổng **XOR**) và bit nhớ ra $C_{out} = A \cdot B$ (dùng cổng **AND**).
  - *Bộ cộng toàn phần 1-bit (Full Adder):* Cộng 3 bit (A, B và bit nhớ vào C_{in}, hoặc ký hiệu là C) tạo ra bit tổng S và bit nhớ ra C_{out}:

$$S = A \oplus B \oplus C = \overline{A}\,\overline{B}C + \overline{A}B\overline{C} + A\overline{B}\,\overline{C} + ABC \quad ; \quad C_{out} = AB + AC + BC$$

    (*Quy tắc nhớ nhanh Full Adder:* Đếm số lượng bit 1 trong 3 đầu vào (A, B, C): nếu có **lẻ** bit 1 tức 1 hoặc 3 bit 1 thì $S = 1$, ngược lại $S = 0$; nếu có **từ 2 bit** 1 **trở lên** thì $C_{out} = 1$, ngược lại $C_{out} = 0$).

- **Mạch dãy (Sequential Circuits – mạch có nhớ):** Đầu ra phụ thuộc cả đầu vào hiện tại và trạng thái quá khứ. Được xây dựng từ các phần tử nhớ **Flip-Flop** (S-R, J-K, D) kết hợp xung nhịp (Clock) để tạo thành các **Thanh ghi song song** và **Thanh ghi dịch**.

4.1.3 Biểu diễn Số nguyên không dấu và Số nguyên có dấu (Mã bù 2)

1. Biểu diễn số nguyên không dấu (Unsigned Integer) với n bit

- Tất cả n bit ($a_{n-1}a_{n-2}\ldots a_1a_0$) đều dùng để biểu diễn độ lớn: $A = \sum_{i=0}^{n-1} a_i 2^i$.
- **Dải biểu diễn của số nguyên không dấu n bit:** Từ 0 **đến** $2^n - 1$.
  - Ví dụ: Với 8 bit: $[0, 255]$; với 9 bit: $[0, 511]$; với 10 bit: $[0, 1023]$; với 16 bit: $[0, 65\,535]$.
  - Bài toán ngược: Để đếm hoặc mã hóa M đối tượng không âm (ví dụ đếm M sinh viên hoặc M chỗ ngồi), số bit tối thiểu n cần dùng phải thỏa mãn $2^n \geq M$ (hoặc $2^n - 1 \geq M$ nếu tính cả giá trị 0).

2. Biểu diễn số nguyên có dấu (Signed Integer) theo Mã bù hai (Two's Complement)

Trong máy tính, số nguyên có dấu n bit ($A = a_{n-1}a_{n-2}\ldots a_1a_0$) được biểu diễn theo **Mã bù hai**. Bit ngoài cùng bên trái (a_{n-1}) là **bit dấu (Sign bit)**:

- Nếu $a_{n-1} = 0$: A là số không âm (≥ 0). Giá trị tính như số không dấu: $A = \sum_{i=0}^{n-2} a_i 2^i$.

- Nếu $a_{n-1} = 1$: A là số nguyên âm (< 0).
- **Công thức tổng quát xác định giá trị của số bù hai n bit:**

$$A = -a_{n-1}2^{n-1} + \sum_{i=0}^{n-2} a_i 2^i$$

- **Dải biểu diễn của số nguyên có dấu n bit:** Từ -2^{n-1} **đến** $+(2^{n-1} - 1)$.
- **Quy tắc tìm Mã bù một và Mã bù hai (Phép đảo dấu số nguyên):**
  1. **Bước 1 (Tìm số bù một):** Đảo giá trị tất cả các bit của số ban đầu ($0 \to 1$ và $1 \to 0$).
  2. **Bước 2 (Tìm số bù hai):** Lấy số bù một **cộng thêm 1** ở bit cuối cùng bên phải:

$$\text{Số bù hai của } A = (\text{Số bù một của } A) + 1 = 2^n - A = -A$$

- **Quy tắc mở rộng bit cho số nguyên (Sign-extension vs. Zero-extension):**
  - *Mở rộng không dấu (Zero-extended):* Luôn thêm các bit `0` vào bên trái.
  - *Mở rộng có dấu (Sign-extended):* Giữ nguyên giá trị của bit dấu đẩy sang toàn bộ các bit mới ở bên trái (số dương thêm toàn bit `0`, tức chữ Hexa `0`; số âm thêm toàn bit `1`, tức chữ Hexa `F`).

Bảng dải biểu diễn của Số nguyên có dấu và Mẹo tính nhẩm nhanh

Số lượng bit (n)	**Dải giá trị thập phân**	**Số âm nhỏ nhất**	**Số dương lớn nhất**
6 bit	$[-32, +31]$	$100000_{(2)}$ (-32)	$011111_{(2)}$ $(+31)$
8 bit (`char`)	$[-128, +127]$	`0x80` (-128)	`0x7F` $(+127)$
9 bit	$[-256, +255]$	$1\,0000\,0000_{(2)}$ (-256)	$0\,1111\,1111_{(2)}$ $(+255)$
12 bit (`imm`)	$[-2048, +2047]$	`0x800` (-2048)	`0x7FF` $(+2047)$
16 bit (`short`)	$[-32\,768, +32\,767]$	`0x8000` $(-32\,768)$	`0x7FFF` $(+32\,767)$
32 bit (`int`)	$[-2^{31}, +2^{31}-1]$	`0x80000000`	`0x7FFFFFFF`

Ba quy tắc tính nhẩm nhanh số bù hai trong phòng thi:

1. **Với số 8-bit có dấu ($a_7a_6a_5a_4a_3a_2a_1a_0$):** Bit đầu a_7 mang giá trị $-\mathbf{128}$, các bit còn lại mang giá trị dương $+64, +32, +16, +8, +4, +2, +1$. (Ví dụ: `0x8E` $= 1000\,1110_2 = -128 + 8 + 4 + 2 = -114$).
2. **Với các số Hexa có dạng toàn chữ `F` ở đầu (như `0xFFFFFFF3` hoặc `0xFFA`):** Vì `0xFFFFFFFF` $= -1$, nên nếu tất cả các chữ số phía trước đều là `F`, chỉ cần lấy **chữ số Hexa cuối cùng trừ đi 16** (Ví dụ: `0xFFFFFFF3` $\implies 3 - 16 = -13$; `0xFFA` $\implies 10 - 16 = -6$).
3. **So sánh nhanh các số nguyên có dấu viết ở dạng Hexa:**
   - Chữ số Hexa đầu tiên từ `0` đến `7`: Bit dấu bằng 0 $\implies$ **Số dương** (hoặc 0).
   - Chữ số Hexa đầu tiên từ `8` đến `F`: Bit dấu bằng 1 $\implies$ **Số âm** (luôn nhỏ hơn mọi số dương).

4.1.4 Cộng / Trừ / Nhân / Chia số nguyên và Hiện tượng Tràn số học (Overflow)

1. Cộng số nguyên không dấu và Tràn nhớ (Carry Out)

- Khi cộng hai số nguyên **không dấu** n bit, nếu tổng vượt quá $2^n - 1$ thì phát sinh bit nhớ ra ngoài cùng $\mathbf{C_{out} = 1}$, làm kết quả n bit bị sai (gọi là tràn nhớ). Nếu $C_{out} = 0$ thì kết quả đúng.

2. Cộng/Trừ số nguyên có dấu và Hiện tượng Tràn số học (Overflow)

- Phép trừ hai số nguyên được phần cứng đưa về phép cộng thông qua lấy bù hai: $X - Y = X + (-Y) = X + \overline{Y} + 1$.

- Khi cộng hai số nguyên **có dấu** n bit, kết quả đúng hay sai **không phụ thuộc vào bit nhớ** C_{out}:
  - Khi cộng hai số nguyên **trái dấu nhau**: Kết quả **luôn luôn đúng** (không bao giờ xảy ra tràn số học).
  - Khi cộng hai số nguyên **cùng dấu nhau**: Nếu bit dấu của kết quả giống với bit dấu của hai số hạng ban đầu thì kết quả đúng. Nếu bit dấu của kết quả **ngược lại** với bit dấu của hai số hạng ban đầu (Cộng hai số dương ra số âm, hoặc Cộng hai số âm ra số dương) thì xảy ra **Tràn số học (Overflow)** và kết quả bị sai!

Hai dạng bài toán Tràn số học (Overflow) trọng tâm trong đề thi

Dạng 1: Tính giá trị hiển thị khi cộng hai biến `char` (8-bit có dấu) bị tràn
Biến kiểu `char` (hoặc `int8_t`) có dải giá trị $[-128, +127]$. Tính tổng toán học thông thường $S = x + y$:

- Nếu $S \in [-128, +127]$: Không tràn, kết quả chính là S.
- Nếu $S > +127$ (tràn dương): Lấy $S - 256$ (Ví dụ: $85 + 52 = 137 > 127 \implies 137 - 256 = -119$).
- Nếu $S < -128$ (tràn âm): Lấy $S + 256$ (Ví dụ: $(-91) + (-67) = -158 < -128 \implies -158 + 256 = +98$).

Dạng 2: Tìm phạm vi của thanh ghi thứ hai gây tràn khi thực hiện lệnh `add rd, rs1, rs2`
Giả sử thanh ghi thứ nhất đang giữ một số nguyên dương $K > 0$ (ví dụ $K = 512 =$ `0x200` hoặc $K = 1024 =$ `0x400`). Vì $K > 0$, phép cộng 32-bit `add` chỉ bị tràn khi thanh ghi thứ hai cũng là một số dương lớn sao cho tổng vượt quá số dương lớn nhất 32-bit là MAX = `0x7FFFFFFF`:

$$K + \texttt{rs2} > \texttt{0x7FFFFFFF} \iff \texttt{rs2} \geq \texttt{0x80000000} - K$$

Vậy phạm vi của thanh ghi thứ hai gây tràn luôn là: **Từ** (`0x80000000` $- K$) **đến** `0x7FFFFFFF`.
(*Lưu ý:* Điểm kết thúc luôn là `0x7FFFFFFF`, không bao giờ chọn đáp án kết thúc bởi `0x80000000` vì `0x80000000` là số âm -2^{31}, cộng với số dương K là hai số trái dấu nên không bao giờ tràn).

3. Nhân/Chia số nguyên và Quy tắc dấu của phép chia trong RISC-V (RV32M)

- **Nhân hai số 32-bit tạo tích 64-bit:** Lệnh `mul rd, rs1, rs2` lấy 32 bit thấp của tích; các lệnh `mulh` (2 số có dấu), `mulhu` (2 số không dấu), `mulhsu` (`rs1` có dấu, `rs2` không dấu) lấy 32 bit cao của tích.

- **Chia hai số nguyên 32-bit có dấu (`div` và `rem`):**

$$\text{Số bị chia} = (\text{Số chia} \times \text{Thương}) + \text{Số dư}$$

  - **Dấu của Thương (`div rd, rs1, rs2`):** Cùng dấu thì thương dương (+); khác dấu thì thương âm (−).
  - **Dấu của Số dư (`rem rd, rs1, rs2`):** Số dư luôn luôn **cùng dấu với SỐ BỊ CHIA (`rs1`)**, bất kể số chia âm hay dương.

4.1.5 Số thực dấu phẩy động chuẩn IEEE 754

1. Cấu trúc chuẩn IEEE 754 độ chính xác đơn 32-bit (Single Precision – Binary32)
Một số thực dấu phẩy động 32-bit gồm 3 trường từ trái sang phải (**1 bit − 8 bit − 23 bit**):

Bit 31 (1 bit)	**Bit 30:23 (8 bit)**	**Bit 22:0 (23 bit)**
Bit dấu S (Sign)	Số mũ lệch e (Biased Exponent)	Phần lẻ định trị m (Fraction)

Công thức xác định giá trị của số thực X ở trạng thái chuẩn hóa ($0 < e < 255$):

$$X = (-1)^S \times (1.m)_{(2)} \times 2^{e-127}$$

Trong đó:

- **Bit dấu S:** Nếu $S = 0$ thì $X > 0$ (số dương); nếu $S = 1$ thì $X < 0$ (số âm).
- **Số mũ thực E:** $E = e - 127 \iff e = E + 127$ (số 127 là hằng số độ lệch Bias của dạng 32-bit).
- **Phần định trị $M = (1.m)_{(2)}$:** Bit 1. trước dấu phẩy được ẩn đi (implied bit) để tiết kiệm 1 bit lưu trữ.
- (*Với dạng 64-bit Double Precision:* Gồm 1 bit dấu S, 11 bit mũ $e = E + 1023$, và 52 bit lẻ định trị m).

2. Các giá trị đặc biệt trong chuẩn IEEE 754 32-bit

- **Số ± 0:** Khi tất cả 8 bit mũ $e = 0$ (`00000000`) và tất cả 23 bit $m = 0$ (`0x00000000` là $+0$; `0x80000000` là -0).
- **Số Vô cùng ($\pm\infty$ – Infinity):** Khi tất cả 8 bit mũ $e = 255$ (`11111111`) và tất cả 23 bit $m = 0$:
  - $+\infty$ có mã Hexa là `0x7F800000` (3 chữ số Hexa đầu là `7F8`).
  - $-\infty$ có mã Hexa là `0xFF800000` (3 chữ số Hexa đầu là `FF8`).

- **Không phải là số (NaN – Not a Number):** Khi tất cả 8 bit mũ $e = 255$ (`11111111`) và phần lẻ định trị $\mathbf{m \neq 0}$ (có ít nhất 1 bit bằng 1 trong 23 bit của m).
  - *Tổng số cách biểu diễn NaN trong chuẩn 32-bit:* Trường $e =$ `11111111` có 1 cách; phần lẻ m (23 bit) phải khác 0 nên có $2^{23} - 1$ tổ hợp; bit dấu S có 2 cách chọn (0 hoặc 1). Vậy tổng số biểu diễn NaN là:

$$2 \times (2^{23} - 1) = \mathbf{2^{24} - 2}$$

4.2 Phần 2: Toàn bộ bài tập Chương 4 và Hướng dẫn giải

4.2.1 Nhóm 1: Hệ đếm, Chuyển đổi cơ số và Xác định số bit biểu diễn

Bài tập 4.1 (Đề Giữa kỳ 2018.2 – Mã đề 198 Câu 7 & Mã đề 199 Câu 2)
Số dương 1101011.1011 ở hệ cơ số 2 bằng với số nào trong các số thập phân sau đây?

A. $107.125_{(10)}$

B. $107.129_{(10)}$

C. $107.1875_{(10)}$

D. $107.6875_{(10)}$

Hướng dẫn giải:

- **Bước 1 (Tính phần nguyên $1101011_{(2)}$):**

$$1101011_{(2)} = 2^6 + 2^5 + 0 + 2^3 + 0 + 2^1 + 2^0 = 64 + 32 + 8 + 2 + 1 = 107$$

- **Bước 2 (Tính phần lẻ $.1011_{(2)}$):**

$$0.1011_{(2)} = 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} + 1 \times 2^{-4} = 0.5 + 0.125 + 0.0625 = 0.6875$$

- **Bước 3 (Ghép lại):** $1101011.1011_{(2)} = 107.6875_{(10)}$.
- **Kết luận:** Đáp án đúng là **D** ($107.6875_{(10)}$).

Bài tập 4.2 (Đề Giữa kỳ 2018.2 – Mã đề 198 Câu 3, Mã đề 199 Câu 12, 18, Mã đề 204 Câu 3, 20 & Đề Giữa kỳ 2019.2 – Mã đề 243 Câu 4, Mã đề 244 Câu 11)
Hãy trả lời chuỗi câu hỏi về dải biểu diễn và số lượng bit cần dùng cho số nguyên:

a) **(Mã đề 198 – Câu 3):** Sử dụng 9 bit để biểu diễn một số nguyên **có dấu**, dải biểu diễn sẽ là?
A. Từ -255 tới $+256$ B. Từ -256 tới $+255$ C. Từ 0 tới 255 D. Từ 0 tới 511

b) **(Mã đề 204 – Câu 3):** Sử dụng 9 bit để biểu diễn một số nguyên **không dấu**, dải biểu diễn sẽ là?
A. Từ 0 tới 255 B. Từ -256 tới $+255$ C. Từ -255 tới $+256$ D. Từ 0 tới 511

c) **(Mã đề 199 – Câu 18):** Sử dụng 10 bit để biểu diễn một số nguyên **không dấu**, dải biểu diễn sẽ là?

A. Từ -511 tới $+512$ B. Từ 0 tới 1023 C. Từ -512 tới $+511$ D. Từ 0 tới 511

d) **(Mã đề 199 – Câu 12):** Biết rằng 511 là số không dấu lớn nhất trong phạm vi biểu diễn. Vậy kiểu dữ liệu số nguyên này có bao nhiêu bit?
A. 9 bit B. 8 bit C. 10 bit D. 11 bit

e) **(Mã đề 204 – Câu 20):** Sử dụng 6 bit nhị phân để biểu diễn số nguyên **có dấu**. Hỏi giá trị lớn nhất có thể biểu diễn là bao nhiêu?
A. 15 B. 31 C. 16 D. 32

f) **(Mã đề 243 Câu 4 & Mã đề 244 Câu 11):**

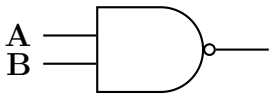
- Lớp học Kiến trúc máy tính có 150 sinh viên. Biến số nguyên để đếm số sinh viên cần tối thiểu mấy bit?
- Hội trường C2 có 1008 chỗ ngồi. Biến số nguyên để đếm số ghế cần tối thiểu mấy bit?

Hướng dẫn giải:

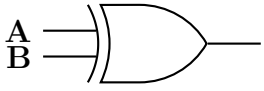
- **Câu a:** Với $n = 9$ bit có dấu, dải biểu diễn là $[-2^{9-1}, +2^{9-1} - 1] = [-2^8, +2^8 - 1] = [\mathbf{-256}, \mathbf{+255}] \rightarrow$ Chọn **B**.
- **Câu b:** Với $n = 9$ bit không dấu, dải biểu diễn là $[0, 2^9 - 1] = [\mathbf{0}, \mathbf{511}] \rightarrow$ Chọn **D**.
- **Câu c:** Với $n = 10$ bit không dấu, dải biểu diễn là $[0, 2^{10} - 1] = [\mathbf{0}, \mathbf{1023}] \rightarrow$ Chọn **B**.
- **Câu d:** Ta có $2^n - 1 = 511 \iff 2^n = 512 = 2^9 \implies n = \mathbf{9}$ bit $\rightarrow$ Chọn **A**.
- **Câu e:** Với $n = 6$ bit có dấu, giá trị dương lớn nhất là $2^{6-1} - 1 = 2^5 - 1 = \mathbf{31} \rightarrow$ Chọn **B**.
- **Câu f:**
  - Để đếm 150 sinh viên: Vì $2^7 - 1 = 127 < 150 \leq 2^8 - 1 = 255$, cần tối thiểu **8 bit**.
  - Để đếm 1008 chỗ ngồi: Vì $2^9 - 1 = 511 < 1008 \leq 2^{10} - 1 = 1023$, cần tối thiểu **10 bit**.

4.2.2 Nhóm 2: Nhận diện Cổng logic, Mạch tổ hợp và Bộ cộng 1-bit

Bài tập 4.3 (Ôn tập Cuối kỳ 2025.2 – P1, Câu 16 & Câu 74)
Hãy cho biết mỗi hình vẽ ký hiệu cổng logic dưới đây biểu diễn cho phép toán logic nào?



Hình a) Câu 16



Hình b) Câu 74

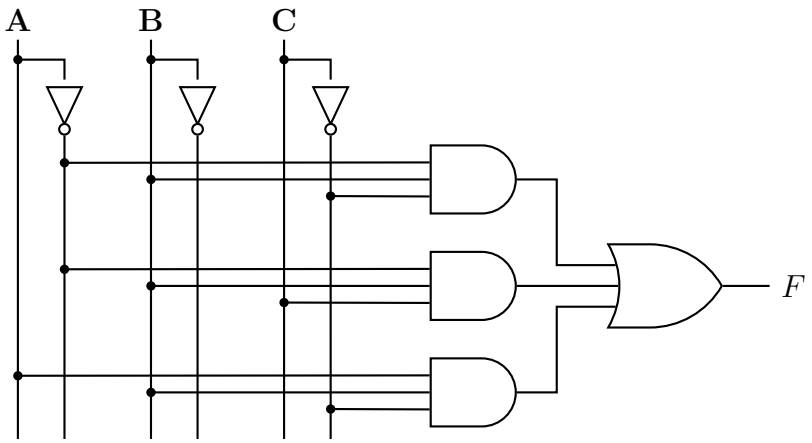
a) **(Câu 16 – Hình a):** A. NOR B. XOR C. AND D. NAND

b) **(Câu 74 – Hình b):** A. XOR B. AND C. NOR D. NAND

Hướng dẫn giải:

- **Giải câu a (Hình a):** Thân cổng có hình chữ D là cổng AND, ở đầu ra có thêm vòng tròn nhỏ đảo bit (NOT) tạo thành cổng **NAND** ($F = \overline{AB}$) $\rightarrow$ Chọn **D (NAND)**.
- **Giải câu b (Hình b):** Thân cổng có mũi nhọn giống cổng OR nhưng có thêm một đường cong tách rời ở phần đuôi là ký hiệu của cổng **XOR** ($F = A \oplus B$) $\rightarrow$ Chọn **A (XOR)**.

Bài tập 4.4 (Slide Chương 4 – Trang 30 & Ôn tập Cuối kỳ 2025.2 – P1, Câu 51) Mạch logic tổ hợp dưới đây có biểu thức đại số Boolean là gì? (Ký hiệu A' là NOT A):



A. $F = (A + B + C')(A + B' + C)(A' + B + C)$

B. $F = A'B'C' + A'BC + AB'C$

C. $F = ABC' + A'BC + AB'C$

D. $F = A'BC + A'BC' + ABC'$

Hướng dẫn giải:

- Mỗi đường tín hiệu A, B, C có 2 nhánh dọc: nhánh bên trái đi thẳng là tín hiệu gốc (A, B, C), nhánh bên phải đi qua cổng đảo NOT là tín hiệu đảo (A', B', C').
- Xác định biểu thức đầu ra của 3 cổng AND từ trên xuống dưới:
  - Cổng AND thứ nhất nối với: nhánh đảo của A (A'), nhánh thẳng của B (B), nhánh đảo của C (C') $\implies$ $\mathbf{A'BC'}$.
  - Cổng AND thứ hai nối với: nhánh đảo của A (A'), nhánh thẳng của B (B), nhánh thẳng của C (C) $\implies$ $\mathbf{A'BC}$.
  - Cổng AND thứ ba nối với: nhánh thẳng của A (A), nhánh thẳng của B (B), nhánh đảo của C (C') $\implies$ $\mathbf{ABC'}$.

- Cổng OR ở cuối cộng 3 tích trên lại: $F = A'BC' + A'BC + ABC'$ (hay $F = A'BC + A'BC' + ABC'$).
- **Kết luận:** Đáp án đúng là **D**.

Bài tập 4.5 (Slide Chương 4 – Trang 37, 38 & Ôn tập Cuối kỳ 2025.2 – P1, Câu 17 & Câu 25)

a) **(Câu 17):** Bản chất của bộ cộng bán phần 1-bit (Half Adder) là thực hiện hai phép toán logic nào?
A. Thực hiện phép XOR và AND B. Thực hiện phép OR và AND
C. Thực hiện phép NOR và NAND D. Thực hiện phép OR và NAND

b) **(Câu 25):** Bộ cộng toàn phần 1-bit (Full Adder) có đầu vào là (A, B, C) và đầu ra là (S, C_{out}). Với bộ giá trị đầu vào $(A, B, C) = (1, 1, 0)$, hãy xác định giá trị đầu ra tương ứng $(S, C_{out}) = ?$
A. $(0, 0)$ B. $(0, 1)$ C. $(1, 0)$ D. $(1, 1)$

Hướng dẫn giải:

- **Giải câu a:** Theo Slide Chương 4 (trang 37), bộ cộng bán phần 1-bit tính bit tổng $S = A \oplus B$ (phép **XOR**) và bit nhớ ra $C_{out} = A \cdot B$ (phép **AND**) $\rightarrow$ Chọn **A**.
- **Giải câu b:** Cộng 3 bit nhị phân: $A + B + C = 1 + 1 + 0 = 2_{(10)} = 10_{(2)}$, trong đó chữ số hàng đơn vị là bit tổng $S = 0$ và chữ số hàng cao là bit nhớ ra $C_{out} = 1$. Vậy $(S, C_{out}) = (0, 1) \rightarrow$ Chọn **B**.

4.2.3 Nhóm 3: Biểu diễn và So sánh Số nguyên có dấu (Mã bù 2)

Bài tập 4.6 (Đề Giữa kỳ 2018.2 – Mã đề 198 Câu 6, Mã đề 199 Câu 10 & Mã đề 204 Câu 8)

a) **(Mã đề 198 Câu 6 & Mã đề 199 Câu 10):** Số âm $-127_{(10)}$ bằng với số nào trong các số biểu diễn nhị phân 8-bit có dấu sau đây?
A. `1000 0000` B. `1000 0001` C. `-0111 1111` D. `0111 1111`

b) **(Mã đề 204 – Câu 8):** Số nguyên $-77_{(10)}$ được biểu diễn dưới dạng số nhị phân 8-bit bù 2 là?
A. `0100 1101` B. `1011 0011` C. `1011 1011` D. `1011 0010`

Hướng dẫn giải:

- **Giải câu a (Biểu diễn -127 bằng 8 bit):**
  - Bước 1: Số dương $+127 =$ `0111 1111`$_{(2)}$.
  - Bước 2 (Đảo bit tìm bù 1): `1000 0000`$_{(2)}$.

  - Bước 3 (Cộng 1 tìm bù 2): $\texttt{1000 0000}+1 = \texttt{1000 0001}_{\mathbf{(2)}}$ (Kiểm tra: $-128+1=-127$) $\rightarrow$ Chọn **B**.

- **Giải câu b (Biểu diễn -77 bằng 8 bit):**
  - Bước 1: Số dương $+77 = 64+8+4+1 = \texttt{0100 1101}_{(2)}$.
  - Bước 2 (Đảo bit tìm bù 1): $\texttt{1011 0010}_{(2)}$.
  - Bước 3 (Cộng 1 tìm bù 2): $\texttt{1011 0010} + 1 = \texttt{1011 0011}_{\mathbf{(2)}} \rightarrow$ Chọn **B**.

Bài tập 4.7 (Đề Giữa kỳ 2019.2 – Mã đề 243 Câu 9 & Mã đề 244 Câu 15)
Số nguyên 8-bit nào sau đây có dạng biểu diễn ở hệ 16 là `0xB2`?

a) **(Mã đề 243):** A. Là số `10110010` ở hệ 2 B. Là số 177 ở hệ 10, không dấu C. Là số `00100010` ở hệ 2 D. Là số 179 ở hệ 10, không dấu

b) **(Mã đề 244):** A. Là số 177 ở hệ 10, không dấu B. Là số `00100010` ở hệ 2 C. Là số `10100000` ở hệ 2 D. Là số -78 ở hệ 10, có dấu

Hướng dẫn giải:

- Đổi `0xB2` sang hệ nhị phân 8-bit: $\texttt{B} = 11 = \texttt{1011}_2$ và $\texttt{2} = \texttt{0010}_2 \implies \texttt{0xB2} = \texttt{1011 0010}_{\mathbf{(2)}}$.
- Nếu xét là số **không dấu**: $128+32+16+2 = 178_{(10)}$.
- Nếu xét là số **có dấu** (bù 2): $-128+32+16+2 = \mathbf{-78_{(10)}}$.
- **Kết luận:** Câu a (Mã đề 243) chọn **A**; Câu b (Mã đề 244) chọn **D**.

Bài tập 4.8 (Đề Cuối kỳ 2020.2 – Trang 2 & Đề Cuối kỳ 2021.1 – Trang 2)
Giả sử biến `i` là một biến số nguyên có dấu kích thước 8-bit được lưu trữ với giá trị Hexa là $\texttt{i} = \texttt{0x8E}$. Hãy cho biết giá trị tương ứng của biến số nguyên này trong hệ thập phân (cơ số 10)?

A. 142

B. -114

C. -113

D. -14

Hướng dẫn giải:

- Đổi `0x8E` sang nhị phân 8-bit: $\texttt{0x8E} = \texttt{1000 1110}_{(2)}$.
- Vì bit dấu đầu tiên bằng 1 (trọng số -128) và các bit 1 còn lại ở vị trí $8, 4, 2$:

$$i = -128+8+4+2 = -128+14 = -114$$

- **Kết luận:** Đáp án đúng là **B (-114)**.

Bài tập 4.9 (Đề Cuối kỳ 2022.2 – #f9c617, 2024.2 & Ôn tập Cuối kỳ 2025.2 – P1, Câu 69)
Giả thiết `k` là biến số nguyên có dấu 16-bit, cho $k = -68$. Hãy cho biết nội dung `k` biểu diễn theo số Hexa?

A. `0xFFBC`

B. `0x00BC`

C. `0x8044`

D. `0xFF44`

Hướng dẫn giải:

- **Bước 1:** Biểu diễn số dương +68 ở dạng 16-bit: $68 = 64 + 4 = \texttt{0100 0100}_{(2)} = \texttt{0x0044}$:

$$+68 = \texttt{0000 0000 0100 0100}_{(2)}$$

- **Bước 2 (Tìm bù 1 – đảo tất cả các bit):** $\texttt{1111 1111 1011 1011}_{(2)}$.
- **Bước 3 (Tìm bù 2 – cộng thêm 1):**

$$-68 = \texttt{1111 1111 1011 1100}_{(2)} = \texttt{0xFFBC}$$

- **Kết luận:** Đáp án đúng là **A** (`0xFFBC`).

Bài tập 4.10 (Đề Cuối kỳ 2022.2 – #7151, 2024.1 – #0182da, 2024.2 & Ôn tập 2025.2 – Câu 45, Câu 73)

a) **(Đề CK 2022.2, 2024.2 – #7151 & Ôn tập 2025.2 Câu 73):** Giả thiết `i` là biến số nguyên có dấu 32-bit, cho nội dung `i` = `0xFFFFFFF3`. Giá trị của `i` theo số thập phân là bao nhiêu?
A. -4 B. -3 C. -13 D. -12

b) **(Đề CK 2024.1 – #0182da & Ôn tập 2025.2 Câu 45):** Giả thiết `i` là biến số nguyên có dấu 32-bit, cho nội dung `i` = `0xFFFFFFF4`. Giá trị của `i` theo số thập phân là bao nhiêu?
A. -11 B. -12 C. -4 D. 12

Hướng dẫn giải:

- Áp dụng quy tắc tính nhanh cho các số âm có 7 chữ số đầu đều là `F`: lấy chữ số Hexa cuối cùng trừ đi 16:
- **Câu a:** Với `0xFFFFFFF3`, chữ số cuối là 3 $\implies i = 3 - 16 = \mathbf{-13}$ → Chọn **C**.
- **Câu b:** Với `0xFFFFFFF4`, chữ số cuối là 4 $\implies i = 4 - 16 = \mathbf{-12}$ → Chọn **B**.

Bài tập 4.11 (Đề Cuối kỳ 2023.1 – #581f03, 2023.2, Giữa kỳ 2024.1 – #30a33e & Ôn tập 2025.2 – Câu 48)

a) **(Đề CK 2023.1 – #581f03):** Cho lệnh hợp ngữ RISC-V: `addi t0, s0, 0xFFA` (với toán hạng tức thời hằng số 12-bit được biểu diễn dưới dạng bù 2). Giá trị thập phân tương ứng của toán hạng tức thời này là bao nhiêu?
A. 4010 B. −4010 C. 6 D. −6

b) **(Đề GK 2024.1 – #30a33e & Ôn tập 2025.2 Câu 48):** Giá trị thập phân của toán hạng immediate trong lệnh `addi t0, s0, 0x1FF` là bao nhiêu?
A. −511 B. 2047 C. −2047 D. 511

Hướng dẫn giải:

- **Giải câu a:** Hằng số 12-bit `0xFFA` có 2 chữ đầu là `FF` (bit dấu số 11 bằng 1, số âm) và chữ số cuối là `A` = 10. Lấy $10 - 16 = \mathbf{-6}$ → Chọn **D** (-6).
- **Giải câu b:** Hằng số 12-bit `0x1FF` có chữ số đầu là `1` = $\texttt{0001}_2$ (bit dấu số 11 bằng 0, số dương). Giá trị thập phân là: $1 \times 16^2 + 15 \times 16^1 + 15 = 256 + 240 + 15 = \mathbf{511}$ → Chọn **D** (511).

Bài tập 4.12 (Đề Giữa kỳ 2023.2 – Đề 1 Câu 2 & Đề Giữa kỳ 2022.2 – Đề 0008 Câu 4, Đề 0006 Câu 2)

a) **(Đề GK 2023.2):** Biểu diễn các số nguyên có dấu +105 và −42 theo mã bù hai 8-bit và 16-bit dưới dạng số Hexa.

b) **(Đề GK 2022.2 – Đề 0008 & 0006):** Hoàn thành bảng chuyển đổi số nguyên 8-bit có dấu giữa Hệ 10, Hệ 2 bù 2 và Hệ 16 bù 2 cho các giá trị: +77; −86; −19; và $\texttt{0011 1101}_{(2)}$ (`3D`).

Hướng dẫn giải:

- **Giải câu a:**
  - Với +105: $105 = 64 + 32 + 8 + 1 = \texttt{0110 1001}_{(2)} = \texttt{0x69}$. Biểu diễn 8-bit là `0x69`; mở rộng dấu dương 16-bit là `0x0069`.
  - Với −42: Ta có $+42 = 32 + 8 + 2 = \texttt{0010 1010}_{(2)} \xrightarrow{\text{bù 1}} \texttt{1101 0101}_{(2)} \xrightarrow{+1} \texttt{1101 0110}_{(2)} = \texttt{0xD6}$. Biểu diễn 8-bit là `0xD6`; mở rộng dấu âm 16-bit (thêm `FF` vào bên trái) là `0xFFD6`.
- **Giải câu b (Bảng kết quả chuyển đổi 8-bit có dấu):**

Hệ 10	Hệ 2 (Bù 2)	Hệ 16	Giải thích cách tính
+77	0100 1101	4D	$64+8+4+1=77$
−86	1010 1010	AA	$+86 = 0101\ 0110_2 \xrightarrow{\text{bù 2}} 1010\ 1010_2$
−19	1110 1101	ED	$+19 = 0001\ 0011_2 \xrightarrow{\text{bù 2}} 1110\ 1101_2$
+61	0011 1101	3D	$32+16+8+4+1=+61$

Bài tập 4.13 (Đề Giữa kỳ 2018.2 – Mã đề 198 Câu 16, Mã đề 199 Câu 11 & Mã đề 204 Câu 17)

a) **(Mã đề 198 – Câu 16):** Cho 4 số nguyên 16-bit **có dấu** có mã Hexa tương ứng như sau: 8792, 2763, F652, A876. Số nào là số lớn nhất?
A. 2763 B. A876 C. F652 D. 8792

b) **(Mã đề 204 – Câu 17):** Cho 4 số nguyên 16-bit **không dấu** có mã Hexa tương ứng như sau: 8792, 2763, F652, A876. Số nào là số lớn nhất?
A. 8792 B. A876 C. 2763 D. F652

c) **(Mã đề 199 – Câu 11):** Cho 4 số nguyên 16-bit **không dấu** có mã Hexa: 79F1, 2AB7, 8000, FFAA. Số nào là số nhỏ nhất?
A. 8000 B. 79F1 C. 2AB7 D. FFAA

Hướng dẫn giải:

- **Giải câu a (Xét số có dấu):** Ba số 8792, F652, A876 đều bắt đầu bằng chữ số Hexa ≥ 8 nên có bit dấu bằng 1 (đều là các số nguyên âm). Chỉ có duy nhất số 2763 bắt đầu bằng chữ số $2 < 8$ (bit dấu bằng 0) là số nguyên dương nên lớn nhất → Chọn **A (2763)**.
- **Giải câu b (Xét số không dấu – tìm số lớn nhất):** Khi không dấu, so sánh trực tiếp chữ số Hexa đầu tiên từ trái sang phải (F > A > 8 > 2). Vậy số lớn nhất là F652 → Chọn **D (F652)**.
- **Giải câu c (Xét số không dấu – tìm số nhỏ nhất):** So sánh chữ số Hexa đầu tiên ($2 < 7 < 8 <$ F). Vậy số nhỏ nhất là 2AB7 → Chọn **C (2AB7)**.

Bài tập 4.14 (Đề Giữa kỳ 2022.2 – Đề 0006 Câu 1, Đề 0008, 0009 Câu 2 & Đề Giữa kỳ 2025.2 – Đề 0013 Câu 73, 942, Đề 0012 Câu 106)
Hãy sắp xếp các bộ 3 số nguyên **8-bit có dấu** dưới đây theo đúng thứ tự yêu cầu của từng đề thi:

a) **(Đề GK 2022.2 – Đề 0006, Câu 1):** Sắp xếp theo thứ tự **giảm dần**: −108 (hệ 10), 0001 0001 (hệ 2), 91 (hệ 16).

b) **(Đề GK 2022.2 – Đề 0009, Câu 2):** Mảng A gồm 3 phần tử 8-bit có dấu ở địa chỉ cơ sở 0x40000E lần lượt là: 0xCE, 0x12, 0x20. Sắp xếp theo giá trị **tăng dần**.

c) **(Đề GK 2022.2 – Đề 0008, Câu 2):** Mảng `A` gồm 3 phần tử 8-bit có dấu ở địa chỉ cơ sở `0x400008` lần lượt là: `0x38`, `0x00`, `0xD6`. Sắp xếp theo giá trị **tăng dần**.

d) **(Đề GK 2025.2 – Đề 0013, Câu 73):** Mảng `A` gồm 3 phần tử 8-bit có dấu ở địa chỉ cơ sở `0x400009` (tra bảng Memory Map Đề 0013 tại `0x400009, 0x40000A, 0x40000B`) lần lượt là: `0xDE`, `0x74`, `0x00`. Sắp xếp theo giá trị **tăng dần**.

e) **(Đề GK 2025.2 – Đề 0013, Câu 942):** Mảng `A` gồm 3 phần tử 8-bit có dấu ở địa chỉ cơ sở `0x400014` (tại `0x400014, 0x400015, 0x400016`) lần lượt là: `0xB7`, `0x59`, `0x34`. Sắp xếp theo giá trị **tăng dần**.

f) **(Đề GK 2025.2 – Đề 0012, Câu 106):** Mảng `A` gồm 3 phần tử 8-bit có dấu ở địa chỉ cơ sở `0x400009` (tra bảng Đề 0012 tại `0x400009, 0x40000A, 0x40000B`) lần lượt là: `0x0B`, `0x8B`, `0x03`. Sắp xếp theo giá trị **tăng dần**.

Hướng dẫn giải:

- **Quy tắc chung:** Với số 8-bit có dấu viết dạng Hexa, các số bắt đầu bằng `8..F` là **số âm**; các số bắt đầu bằng `0..7` là **số dương** (hoặc 0).
- **Câu a:** Đổi về hệ 10: -108; $\texttt{0001 0001}_2 = +17$; $\texttt{0x91} = \texttt{1001 0001}_2 = -128 + 16 + 1 = -111$. Vì $+17 > -108 > -111$, thứ tự giảm dần là: `0001 0001` $> -108 >$ `91`.
- **Câu b:** Có `0xCE` là số âm (-50), còn `0x12`$(+18) <$ `0x20`$(+32) \implies$ Tăng dần: `0xCE` $<$ `0x12` $<$ `0x20`.
- **Câu c:** Có `0xD6` là số âm (-42), còn `0x00`$(0) <$ `0x38`$(+56) \implies$ Tăng dần: `0xD6` $<$ `0x00` $<$ `0x38`.
- **Câu d (Đề 0013 – Câu 73):** Có `0xDE` là số âm (-34), còn `0x00`$(0) <$ `0x74`$(+116) \implies$ Tăng dần: `0xDE` $<$ `0x00` $<$ `0x74`.
- **Câu e (Đề 0013 – Câu 942):** Có `0xB7` là số âm (-73), còn `0x34`$(+52) <$ `0x59`$(+89) \implies$ Tăng dần: `0xB7` $<$ `0x34` $<$ `0x59`.
- **Câu f (Đề 0012 – Câu 106):** Có `0x8B` là số âm (-117), còn `0x03`$(+3) <$ `0x0B`$(+11) \implies$ Tăng dần: `0x8B` $<$ `0x03` $<$ `0x0B`.

4.2.4 Nhóm 4: Phép cộng/trừ số nguyên và Hiện tượng Tràn số học (Overflow)

Bài tập 4.15 (Đề Giữa kỳ 2018.2 – Mã đề 198 Câu 8 & Mã đề 199 Câu 9)
Máy tính thực hiện phép toán cộng 2 số 8-bit không dấu ở hệ cơ số 2 sau đây, kết quả (lưu trong thanh ghi 8-bit) sẽ bằng bao nhiêu:

$$\texttt{1010 1000} + \texttt{1111 1010}$$

A. `1000 0010`

B. `1000 0100`

C. `1010 0100`

D. `1010 0010`

Hướng dẫn giải:

- Thực hiện cộng từng cột bit từ phải sang trái:
  - 4 bit thấp: `1000` + `1010` = `1 0010` (viết `0010`, nhớ 1 sang 4 bit cao).
  - 4 bit cao: `1010` + `1111` + 1 (nhớ) = `1010` + `1 0000` = `1 1010` (trong 8 bit giữ lại `1010`, bit nhớ tràn ra ngoài $C_{out} = 1$).
- Ghép 8 bit kết quả lại ta được: `1010 0010`.
- **Kết luận:** Đáp án đúng là **D** (`1010 0010`).

Bài tập 4.16 (Ôn tập Cuối kỳ 2025.2 – P1, Câu 2)
Khẳng định nào dưới đây là đúng khi thực hiện phép cộng hai số nguyên n-bit?

A. Khi cộng hai số nguyên không dấu n-bit, nếu $C_{out} = 1 \rightarrow$ kết quả sai.

B. Khi cộng hai số nguyên có dấu n-bit, nếu $C_{out} = 1$ và hai số ban đầu cùng dấu nhau $\rightarrow$ kết quả sai.

C. Khi cộng hai số nguyên có dấu n-bit, nếu hai số ban đầu trái dấu nhau $\rightarrow$ kết quả đúng.

D. Khi cộng hai số nguyên có dấu n-bit, nếu hai số ban đầu cùng dấu nhau và bit cao nhất của kết quả giống bit cao nhất của hai số ban đầu $\rightarrow$ chưa thể kết luận tính đúng sai.

Hướng dẫn giải:

- Theo Slide Chương 4 (trang 70 và trang 74):
  - Khi cộng hai số nguyên **không dấu** n-bit, nếu có nhớ ra ngoài ($C_{out} = 1$) thì tổng vượt quá $2^n - 1$ nên kết quả n-bit bị sai $\implies$ **Khẳng định A đúng**.
  - Khi cộng hai số nguyên **có dấu** n-bit, nếu hai số ban đầu **trái dấu nhau** thì tổng luôn nằm trong dải biểu diễn nên kết quả luôn luôn đúng $\implies$ **Khẳng định C đúng**.
  - (Khẳng định B sai vì khi cộng hai số âm không bị tràn như $(-1) + (-1) = -2$, luôn có $C_{out} = 1$ nhưng kết quả vẫn đúng; Khẳng định D sai vì khi hai số cùng dấu mà kết quả vẫn cùng dấu thì chắc chắn kết quả đúng).
- **Kết luận:** Chọn cả **A** và **C**.

Bài tập 4.17 (Đề Giữa kỳ 2019.2 – Mã đề 243 Câu 12 & Mã đề 244 Câu 5)
Tổng của 2 số 8-bit có dấu nào dưới đây sẽ bị hiện tượng tràn số học (Overflow)?

a) **(Mã đề 243 – Câu 12):**

A. `01010001` + `00101100` B. `10010001` + `01001000`

C. `00010001` + `11001000` D. `10010001` + `11001000`

b) **(Mã đề 244 – Câu 5):**

A. `00010001` + `11001000` B. `00011001` + `00111000`

C. `01010001` + `01001000` D. `10010001` + `01001000`

Hướng dẫn giải:

- **Bước 1 (Loại ngay các cặp trái dấu):** Các phép cộng có một số bắt đầu bằng bit `0` và một số bắt đầu bằng bit `1` là cộng hai số trái dấu nên không bao giờ tràn (Loại B, C ở câu a; loại A, D ở câu b).
- **Bước 2 (Kiểm tra các cặp cùng dấu):**
  - Ở câu a: Phương án A cộng hai số dương $81 + 44 = +125 \leq 127$ (không tràn). Phương án D cộng hai số âm: $\texttt{1001 0001}_2(-111) + \texttt{1100 1000}_2(-56) = -167 < -128$ (kết quả 8-bit ra $\texttt{0101 1001}_2$ có bit dấu bằng `0` – lật thành số dương) $\Longrightarrow$ **Bị tràn âm (Chọn D)**.
  - Ở câu b: Phương án B cộng $25 + 56 = 81 \leq 127$ (không tràn). Phương án C cộng hai số dương: $\texttt{0101 0001}_2(+81) + \texttt{0100 1000}_2(+72) = +153 > 127$ (kết quả 8-bit ra $\texttt{1001 1001}_2$ có bit dấu bằng `1` – lật thành số âm) $\Longrightarrow$ **Bị tràn dương (Chọn C)**.

Bài tập 4.18 (Đề Cuối kỳ 2022.2 – #6d82ac, 2024.2 & Ôn tập Cuối kỳ 2025.2 – P1, Câu 66)

Trong ngôn ngữ lập trình C, biến kiểu `char` là biến dữ liệu số nguyên có dấu 8-bit. Cho chương trình C dưới đây, hãy cho biết kết quả của biến `sum` hiển thị trên màn hình khi chạy chương trình?

```
#include <stdio.h>
int main() {
    char a = -91;
    char b = -67;
    char sum = a + b;
    printf("sum = %d", sum); // hien thi sum dang thap phan
    return 0;
}
```

A. -98

B. -158

C. 158

D. 98

Hướng dẫn giải:

- Tổng toán học thông thường: $S = (-91) + (-67) = -158$.
- Vì biến `sum` thuộc kiểu `char` (8-bit có dấu, chỉ biểu diễn được từ -128 đến $+127$), nên giá trị $-158 < -128$ bị tràn âm (Overflow).
- Áp dụng công thức tràn âm 8-bit (cộng thêm $2^8 = 256$):

$$\texttt{sum} = -158 + 256 = +98$$

- **Kết luận:** Đáp án đúng là **D (98)**.

Bài tập 4.19 (Đề Cuối kỳ 2025.1 – #e3221e, Giữa kỳ 2024.1 – #f2d536, Giữa kỳ 2022.2 – Đề 0007 Câu 1 & Ôn tập 2025.2 – Câu 40)
Giả thiết tất cả các biến đều là số nguyên 8-bit có dấu (`signed char`). Hãy tính giá trị thập phân lưu trong biến kết quả sau khi thực hiện các phép cộng sau:

a) **(Đề CK 2025.1 – #e3221e):** Cho $x = 85, y = 52$. Tính $z = x + y$.

b) **(Đề GK 2024.1 – #f2d536 & Ôn tập 2025.2 Câu 40):** Tính $+65 + 65$.

c) **(Đề GK 2022.2 – Đề 0007, Câu 1a):** Tính $5 + 125$.

d) **(Đề GK 2022.2 – Đề 0007, Câu 1b):** Tính $(-108) + (-42)$.

Hướng dẫn giải:
Dải biểu diễn của số nguyên 8-bit có dấu là $[-128, +127]$. Nếu tổng lớn hơn $+127$ thì trừ đi 256; nếu tổng nhỏ hơn -128 thì cộng thêm 256:

- **Câu a:** $S = 85 + 52 = 137 > 127$ (tràn dương) $\implies z = 137 - 256 = \mathbf{-119}$.
- **Câu b:** $S = 65 + 65 = 130 > 127$ (tràn dương) $\implies$ Kết quả $= 130 - 256 = \mathbf{-126}$.
- **Câu c:** $S = 5 + 125 = 130 > 127$ (tràn dương) $\implies$ Kết quả $= 130 - 256 = \mathbf{-126}$.
- **Câu d:** $S = (-108) + (-42) = -150 < -128$ (tràn âm) $\implies$ Kết quả $= -150 + 256 = \mathbf{+106}$.

Bài tập 4.20 (Đề Cuối kỳ 2023.1 – #cd9132, 2023.2, Giữa kỳ 2024.1 – #e83dac & Ôn tập 2025.2 – Câu 7)

a) **(Đề CK 2023.1 – #cd9132 & Ôn tập 2025.2 Câu 7):** Giả sử thanh ghi `s0` đang lưu trữ giá trị thập phân là 1024. Hãy cho biết phạm vi giá trị của thanh ghi `s1` để gây ra hiện tượng tràn số học (overflow) khi thực hiện lệnh `add s2, s0, s1`?
A. `0x7FFFFC00 - 0x7FFFFFFF` B. `0x7FFFFC01 - 0x80000000`
C. `0x7FFFFBFF - 0x7FFFFFFF` D. `0x7FFFFC00 - 0x80000000`

b) **(Đề GK 2024.1 – #e83dac):** Giả sử thanh ghi `t0` đang lưu trữ giá trị thập phân là

512. Phạm vi giá trị nào của thanh ghi `t1` sẽ gây ra tràn số học khi thực hiện lệnh `add t2, t0, t1`?

A. `0x7FFFFD00 - 0x7FFFFFFF` B. `0x7FFFFDFF - 0x7FFFFFFF`

C. `0x7FFFFE00 - 0x80000000` D. `0x7FFFFE00 - 0x7FFFFFFF`

Hướng dẫn giải:

- **Giải câu a (Với $K = 1024 = 2^{10} =$ `0x00000400`):**
  - Giá trị nhỏ nhất của `s1` bắt đầu làm tổng vượt quá `0x7FFFFFFF` là:

$$\texttt{s1}_{\min} = \texttt{0x80000000} - \texttt{0x400} = \texttt{0x7FFFFC00} \quad (\text{vì } \texttt{0x1000} - \texttt{0x400} = \texttt{0xC00})$$

  - Giá trị dương lớn nhất của thanh ghi 32-bit là `0x7FFFFFFF`.
  - Kết luận: Chọn **A** (`0x7FFFFC00 - 0x7FFFFFFF`).
- **Giải câu b (Với $K = 512 = 2^9 =$ `0x00000200`):**
  - Giá trị nhỏ nhất của `t1` bắt đầu gây tràn là:

$$\texttt{t1}_{\min} = \texttt{0x80000000} - \texttt{0x200} = \texttt{0x7FFFFE00} \quad (\text{vì } \texttt{0x1000} - \texttt{0x200} = \texttt{0xE00})$$

  - Kết luận: Chọn **D** (`0x7FFFFE00 - 0x7FFFFFFF`).

Bài tập 4.21 (Ôn tập Cuối kỳ 2025.1 – Câu 15 & Ôn tập 2025.2 – P2, Câu 59) Đoạn lệnh nào sau đây có xảy ra hiện tượng tràn số học (OVERFLOW)?

A. `li x8, 0xFFFFFFFF` → `li x9, 0xFFFFFFFF` → `add x10, x8, x9`

B. `li x8, 0xFFFFFFFF` → `li x9, 0x80000000` → `add x10, x8, x9`

C. `li x8, 0xFFFFFFFF` → `li x9, 0x80000000` → `sub x10, x8, x9`

D. `li x8, 0xFFFFFFFF` → `li x9, 0xFFFFFFFF` → `sub x10, x8, x9`

Hướng dẫn giải:

- Ta có `0xFFFFFFFF` $= -1$ và `0x80000000` $= -2^{31}$ (số nguyên âm nhỏ nhất của 32-bit).
- Kiểm tra từng phương án:
  - Phương án A: $(-1) + (-1) = -2$ (nằm trong dải, không tràn).
  - Phương án B: Cộng hai số âm $(-1) + (-2^{31}) = -(2^{31} + 1) < -2^{31}$. Vì `x9` $= -2^{31}$ đã ở kịch biên âm của 32-bit, cộng thêm -1 sẽ vượt ra ngoài dải biểu diễn và lật thành số dương `0x7FFFFFFF` $\implies$ **Xảy ra tràn số học (Overflow)**.
  - Phương án C: $(-1) - (-2^{31}) = +2^{31} - 1 =$ `0x7FFFFFFF` (vừa đúng biên dương lớn nhất, chưa tràn).

– Phương án D: $(-1) - (-1) = 0$ (không tràn).

- **Kết luận:** Đáp án đúng là **B**.

Bài tập 4.22 (Đề Giữa kỳ 2021.1 – IT3030E, Problem 4)
Hai thanh ghi `s0` và `s1` đang giữ các giá trị:

`s0 = 0x80000000` ; `s1 = 0xD0000000`

Hãy xác định giá trị Hexa của thanh ghi `t0` và cho biết có xảy ra tràn số học (Overflow) hay không cho 3 đoạn lệnh sau:

4.1. `add t0, s0, s1`

4.2. `sub t0, s0, s1`

4.3. `add t0, s0, s1` → `add t0, t0, s0`

Hướng dẫn giải:

- Nhận xét dấu ban đầu: Cả `s0` = `0x80000000` (chữ đầu `8` = 1000_2) và `s1` = `0xD0000000` (chữ đầu `D` = 1101_2) đều có bit dấu số 31 bằng 1, tức **cả `s0` và `s1` đều là số nguyên âm**.
- **Giải ý 4.1 (`add t0, s0, s1`):**
  - Cộng chữ số Hexa đầu tiên: `8` + `D` = $8 + 13 = 21 = 16 + 5 = 15_{(16)}$ (tràn chữ số 1 ra ngoài 32-bit, còn lại chữ số `5`).
  - Giá trị lưu trong `t0` là: `t0 = 0x50000000`.
  - Vì cộng hai số âm (`s0 < 0, s1 < 0`) mà kết quả thu được lại là số dương (`t0` = `0x50000000` có bit dấu bằng 0), nên **CÓ xảy ra tràn số học (Overflow)**.
- **Giải ý 4.2 (`sub t0, s0, s1`):**
  - Thực hiện phép trừ: `s0` − `s1` = `0x80000000` − `0xD0000000`. Đổi −`s1` bằng cách lấy bù 2 của `0xD0000000` là `0x30000000` (số dương).
  - Cộng lại: `0x80000000` + `0x30000000` = `0xB0000000`.
  - Vì đây là phép cộng hai số trái dấu (`s0` < 0 và $-$`s1` > 0), nên **KHÔNG xảy ra tràn số học**.
- **Giải ý 4.3 (`add t0, s0, s1` rồi `add t0, t0, s0`):**
  - Sau lệnh đầu tiên, `t0` = `0x50000000` (có tràn ở lệnh 1).
  - Ở lệnh thứ hai, cộng `t0` + `s0` = `0x50000000` + `0x80000000` = `0xD0000000` (bản thân lệnh thứ hai cộng số dương `0x50000000` với số âm `0x80000000` nên lệnh thứ hai không tràn).

4.2.5 Nhóm 5: Lệnh Chia và Lấy phần dư số nguyên (`div, rem`)

Bài tập 4.23 (Đề Giữa kỳ 2024.1 – #054a4a)
Giá trị của các thanh ghi `t0, t1` sau khi thực hiện đoạn mã RISC-V dưới đây là gì?

```
li  s0, 67
li  s1, -13
div t0, s0, s1
rem t1, s0, s1
```

A. `t0` = `0xfffffffb`, `t1` = `0x00000010`

B. `t0` = `0x00000005`, `t1` = `0xfffffffd`

C. `t0` = `0xfffffffb`, `t1` = `0x00000005`

D. `t0` = `0xfffffffb`, `t1` = `0x00000002`

Hướng dẫn giải:

- Ta có số bị chia `s0` $= +67$ và số chia `s1` $= -13$.
- **Tính thương `t0` (`div t0, s0, s1`):**
  - Chia độ lớn: $67 \div 13 = 5$ (vì $13 \times 5 = 65$).
  - Vì hai số trái dấu nên thương mang dấu âm: `t0` $= -5$.
  - Đổi -5 sang Hexa 32-bit (lấy $16 - 5 = 11 =$ `b`): `t0` = `0xfffffffb`.
- **Tính số dư `t1` (`rem t1, s0, s1`):**
  - Độ lớn phần dư: $67 - 65 = 2$.
  - Theo quy tắc: **Số dư luôn cùng dấu với số bị chia** (`s0` $= +67 > 0$), nên số dư mang dấu dương: `t1` $= +2 =$ `0x00000002`.
- **Kết luận:** Đáp án đúng là **D** (`t0` = `0xfffffffb`, `t1` = `0x00000002`).

4.2.6 Nhóm 6: Số thực dấu phẩy động chuẩn IEEE 754 (32-bit)

Quy trình 4 bước giải nhanh bài toán IEEE 754 32-bit

- **Bài toán 1: Cho mã Hexa 32-bit → Tìm giá trị thập phân X**
  1. Đổi 3 hoặc 4 chữ số Hexa đầu tiên sang nhị phân (12 hoặc 16 bit đầu).
  2. Tách **1 bit đầu** là dấu S ($0 \to +$; $1 \to -$); **8 bit tiếp theo** là số mũ lệch e; các bit còn lại là phần lẻ định trị m.
  3. Tính số mũ thực: $E = e - 127$.
  4. Ghép bit ẩn 1. vào trước m được $(1.m)_{(2)}$, rồi dịch dấu phẩy sang phải E vị trí (nếu $E > 0$) và đổi sang hệ 10.
- **Bài toán 2: Cho số thực thập phân X → Tìm mã Hexa 32-bit**
  1. Xác định bit dấu S ($X > 0 \implies S = 0$; $X < 0 \implies S = 1$).
  2. Đổi $|X|$ sang nhị phân, rồi dịch dấu phẩy về ngay sau bit 1 đầu tiên để đưa về dạng chuẩn $(1.m)_{(2)} \times 2^E$.
  3. Tính số mũ lệch $e = E + 127$ và đổi e ra 8 bit nhị phân (nhớ rằng $128 = 1000\,0000_2$).
  4. Ghép [S (1 bit) | e (8 bit) | m (23 bit)], rồi nhóm từng cụm 4 bit từ trái sang phải để viết thành 8 chữ số Hexa.

Bài tập 4.24 (Đề Giữa kỳ 2019.2 – Mã đề 243 Câu 2 & Mã đề 244 Câu 17)
Số thực dấu phẩy động (chuẩn IEEE 754) **không thể** biểu diễn chính xác tuyệt đối được số nào dưới đây?

a) **(Mã đề 243 – Câu 2):** A. 0.5 B. 0.4 C. 12 D. 2

b) **(Mã đề 244 – Câu 17):** A. -8 B. -1 C. 1 D. 0.7

Hướng dẫn giải:

- Các số nguyên nhỏ $(1, -1, 2, -8, 12)$ và số thực có phần lẻ là lũy thừa âm của 2 (như $0.5 = 2^{-1} = 0.1_{(2)}$) đều biểu diễn chính xác tuyệt đối bằng hữu hạn bit nhị phân.
- Các số thập phân có phần lẻ là $0.4 = \dfrac{2}{5}$ hoặc $0.7 = \dfrac{7}{10}$ (mẫu số chứa thừa số nguyên tố $5 \neq 2$) khi đổi sang nhị phân sẽ ra chuỗi bit tuần hoàn vô hạn ($0.4_{(10)} = 0.01100110011\ldots_{(2)}$), nên bị làm tròn và không thể biểu diễn chính xác tuyệt đối.
- **Kết luận:** Câu a chọn **B** (0.4); Câu b chọn **D** (0.7).

Bài tập 4.25 (Đề Giữa kỳ 2018.2 – Mã đề 198, Câu 5)

Số dấu phẩy động 32-bit sau có giá trị thập phân bằng bao nhiêu?

`1100 0000 1000 0000 0000 0000 0000 0000`

A. -5

B. -4

C. -2.5

D. -8

Hướng dẫn giải:

- Tách 3 trường (1 bit – 8 bit – 23 bit):

$$\underbrace{\texttt{1}}_{S=1} \mid \underbrace{\texttt{1000 0001}}_{e=129} \mid \underbrace{\texttt{000 0000...}}_{m=0}$$

- Bit dấu $S = 1 \implies$ Số âm $(-)$.
- Số mũ thực: $E = e - 127 = 129 - 127 = 2$.
- Phần định trị: $1.m = 1.0_{(2)} = 1$.
- Giá trị số thực: $X = -1.0 \times 2^2 = \mathbf{-4}$.
- **Kết luận:** Đáp án đúng là **B** (-4).

Bài tập 4.26 (Đề Cuối kỳ 2022.2 – #bd4c95, 2024.1 – #e73be8, 2024.2 & Ôn tập 2025.2 – Câu 39, Câu 53)

a) **(Đề CK 2022.2, 2024.2 – #bd4c95 & Ôn tập 2025.2 Câu 53):** Giả thiết X là biến số dấu phẩy động theo chuẩn IEEE 754 32-bit, cho nội dung $X =$ `0x40400000`. Hãy xác định giá trị của X dưới dạng thập phân?
A. 3 B. -3 C. 2 D. 4

b) **(Đề CK 2024.1 – #e73be8 & Ôn tập 2025.2 Câu 39):** Cho nội dung $X =$ `0x40A00000`. Hãy xác định giá trị của X dưới dạng thập phân?
A. 6 B. 5 C. -5 D. 4

Hướng dẫn giải:

- **Giải câu a (Với $X =$ `0x40400000`):**
  - Đổi 3 chữ số Hexa đầu (`404`) sang 12 bit nhị phân:

$$\texttt{404}_{(16)} = \texttt{0100 0000 0100}_{(2)} = \underbrace{\texttt{0}}_{S=0} \mid \underbrace{\texttt{1000 0000}}_{e=128} \mid \underbrace{\texttt{100...}}_{m}$$

– Ta có: $S = 0$ (số dương); $E = 128 - 127 = 1$; $1.m = 1.1_{(2)} = 1.5$.

– Giá trị: $X = +1.5 \times 2^1 = \mathbf{3} \rightarrow$ Chọn **A (3)**.

- **Giải câu b (Với $X =$ `0x40A00000`):**

– Đổi 3 chữ số Hexa đầu (`40A`) sang 12 bit nhị phân:

$$\texttt{40A}_{(16)} = \texttt{0100 0000 1010}_{(2)} = \underbrace{\texttt{0}}_{S=0} \mid \underbrace{\texttt{1000 0001}}_{e=129} \mid \underbrace{\texttt{010...}}_{m}$$

– Ta có: $S = 0$ (số dương); $E = 129 - 127 = 2$; $1.m = 1.01_{(2)} = 1.25$.

– Giá trị: $X = +1.25 \times 2^2 = 1.25 \times 4 = \mathbf{5} \rightarrow$ Chọn **B (5)**.

Bài tập 4.27 (Đề Giữa kỳ 2024.1 – #94ec4c & Ôn tập Cuối kỳ 2025.2 – P1, Câu 30)

a) **(Đề GK 2024.1 – #94ec4c):** Cho số thực dấu phẩy động IEEE 754 độ chính xác đơn 32-bit được biểu diễn bởi word `0xC11A0000`. Giá trị của số này ở hệ thập phân là gì?

b) **(Ôn tập 2025.2 – P1, Câu 30):** Giả thiết X là số thực dấu phẩy động IEEE 754 32-bit được biểu diễn bởi `0xC1E00000`. Giá trị của X ở hệ thập phân là gì?

Hướng dẫn giải:

- **Giải câu a (Với `0xC11A0000`):**

– Đổi 4 chữ số Hexa đầu (`C11A`) sang 16 bit nhị phân:

$$\texttt{C11A}_{(16)} = \texttt{1100 0001 0001 1010}_{(2)} = \underbrace{\texttt{1}}_{S=1} \mid \underbrace{\texttt{1000 0010}}_{e=130} \mid \underbrace{\texttt{0011010...}}_{m}$$

– Ta có: $S = 1$ (số âm); $E = 130 - 127 = 3$; $1.m = 1.001101_{(2)}$.

– Dịch dấu phẩy sang phải 3 vị trí:

$$X = -1.001101_{(2)} \times 2^3 = -1001.101_{(2)} = -(8 + 1 + 0.5 + 0.125) = \mathbf{-9.625}$$

- **Giải câu b (Với `0xC1E00000`):**

– Đổi 3 chữ số Hexa đầu (`C1E`) sang 12 bit nhị phân:

$$\texttt{C1E}_{(16)} = \texttt{1100 0001 1110}_{(2)} = \underbrace{\texttt{1}}_{S=1} \mid \underbrace{\texttt{1000 0011}}_{e=131} \mid \underbrace{\texttt{110...}}_{m}$$

– Ta có: $S = 1$ (số âm); $E = 131 - 127 = 4$; $1.m = 1.11_{(2)}$.

– Dịch dấu phẩy sang phải 4 vị trí:

$$X = -1.1100_{(2)} \times 2^4 = -11100_{(2)} = -(16 + 8 + 4) = \mathbf{-28}$$

Bài tập 4.28 (Đề Cuối kỳ 2021.2 – Mã đề B5, Câu 1.1 & Slide Chương 4 – Trang 98, 99)

Chuyển đổi các số biểu diễn dưới dạng Hexa sau về số thực dấu phẩy động theo chuẩn IEEE 754 32-bit:

a) `0x42A78000`

b) `0xC1560000`

c) `0x3FC00000` (`0011 1111 1100 0000 ...`)

Hướng dẫn giải:

- **Giải câu a (Với `0x42A78000`):**
  - Đổi 5 chữ số Hexa đầu (`42A78`) sang 20 bit nhị phân:

$$\texttt{0100 0010 1010 0111 1000}_{(2)} = \underbrace{\texttt{0}}_{S=0} \mid \underbrace{\texttt{1000 0101}}_{e=133} \mid \underbrace{\texttt{01001111000...}}_{m}$$

  - Ta có: $S = 0$ (số dương); $E = 133 - 127 = 6$; $1.m = 1.01001111_{(2)}$.
  - Dịch dấu phẩy sang phải 6 vị trí:

$$X = +1010011.11_{(2)} = (64 + 16 + 2 + 1) + (0.5 + 0.25) = +\mathbf{83.75}$$

- **Giải câu b (Với `0xC1560000`):**
  - Đổi 4 chữ số Hexa đầu (`C156`) sang 16 bit nhị phân:

$$\texttt{1100 0001 0101 0110}_{(2)} = \underbrace{\texttt{1}}_{S=1} \mid \underbrace{\texttt{1000 0010}}_{e=130} \mid \underbrace{\texttt{1010110...}}_{m}$$

  - Ta có: $S = 1$ (số âm); $E = 130 - 127 = 3$; $1.m = 1.101011_{(2)}$.
  - Dịch dấu phẩy sang phải 3 vị trí:

$$X = -1101.011_{(2)} = -(8 + 4 + 1 + 0.25 + 0.125) = -\mathbf{13.375}$$

- **Giải câu c (Với `0x3FC00000`):**
  - Đổi 3 chữ số Hexa đầu (`3FC`) sang 12 bit nhị phân:

$$\texttt{0011 1111 1100}_{(2)} = \underbrace{\texttt{0}}_{S=0} \mid \underbrace{\texttt{0111 1111}}_{e=127} \mid \underbrace{\texttt{100...}}_{m}$$

  - Ta có: $S = 0$; $E = 127 - 127 = 0$; $X = +1.1_{(2)} \times 2^0 = +\mathbf{1.5}$.

Bài tập 4.29 (Đề Giữa kỳ 2024.1 – #386374)

Biểu diễn các giá trị thập phân sau sang số dấu phẩy động chuẩn IEEE 754 độ chính xác đơn (32-bit) dưới dạng số Hexa:

a) -13.75

b) -46.375

Hướng dẫn giải:

- **Giải câu a (Biểu diễn -13.75):**
  - Bước 1: Vì $-13.75 < 0$ nên bit dấu $S = 1$.
  - Bước 2: Đổi 13.75 sang nhị phân và chuẩn hóa:

$$13 = 1101_{(2)} \ ; \ 0.75 = 0.11_{(2)} \implies 13.75 = 1101.11_{(2)} = 1.10111_{(2)} \times 2^3$$

  - Bước 3: Tính số mũ lệch $e = E + 127 = 3 + 127 = 130 = 128 + 2 = \texttt{1000 0010}_{(2)}$.
  - Bước 4: Ghép $[S \mid e \mid m]$ và nhóm 4 bit một thành chữ số Hexa:

$$\underbrace{\texttt{1}}_{S} \ \underbrace{\texttt{1000 0010}}_{e} \ \underbrace{\texttt{101 1100 0000...}}_{m} = \texttt{1100 0001 0101 1100 0000...}_{(2)} = \texttt{0xC15C0000}$$

- **Giải câu b (Biểu diễn -46.375):**
  - Bước 1: Số âm nên $S = 1$.
  - Bước 2: $46 = 32 + 8 + 4 + 2 = 101110_{(2)}$ và $0.375 = 0.25 + 0.125 = 0.011_{(2)}$:

$$46.375 = 101110.011_{(2)} = 1.01110011_{(2)} \times 2^5$$

  - Bước 3: Tính số mũ lệch $e = 5 + 127 = 132 = 128 + 4 = \texttt{1000 0100}_{(2)}$.
  - Bước 4: Ghép $[S \mid e \mid m]$ và đổi sang Hexa:

$$\underbrace{\texttt{1}}_{S} \ \underbrace{\texttt{1000 0100}}_{e} \ \underbrace{\texttt{011 1001 1000...}}_{m} = \texttt{1100 0010 0011 1001 1000 0000...}_{(2)} = \texttt{0xC2398000}$$

Bài tập 4.30 (Ôn tập Cuối kỳ 2025.2 – P1, Câu 46 & Câu 59)

a) **(Câu 59):** Cho số thực dấu phẩy động 32-bit theo chuẩn IEEE 754 viết theo hệ Hexa có 3 chữ số có ý nghĩa lớn nhất (bên trái) là `7F8`, phần còn lại chưa được biết. Khẳng định nào dưới đây có khả năng đúng? (Chọn tất cả các khẳng định đúng)
1. Số thực này bằng 0
2. Số thực này biểu diễn NaN (Not a Number)
3. Số thực này biểu diễn giá trị dương xác định
4. Số thực này biểu diễn vô cùng (Infinity)

b) **(Câu 46):** Có bao nhiêu cách biểu diễn NaN (Not a Number) khi dùng số thực dấu phẩy động 32-bit theo chuẩn IEEE 754?
A. 128 B. $2^{23}-1$ C. $2^{24}-2$ D. 2

Hướng dẫn giải:

- **Giải câu a:** Đổi 3 chữ số Hexa đầu `7F8` sang 12 bit nhị phân:

$$\texttt{7F8}_{(16)} = \texttt{0111 1111 1000}_{(2)} = \underbrace{\texttt{0}}_{S=0} \mid \underbrace{\texttt{1111 1111}}_{e=255} \mid \underbrace{\texttt{000...}}_{\text{3 bit đầu của } m}$$

  Vì cả 8 bit của trường mũ e đều bằng 1 ($e = 255$), theo quy ước đặc biệt của IEEE 754 (Slide Chương 4, trang 100):

  - Nếu 20 bit còn lại đều bằng 0 (tức toàn bộ 23 bit $m = 0$), thì số này biểu diễn $+\infty$ (**vô cùng**).
  - Nếu trong 20 bit còn lại có ít nhất 1 bit bằng 1 (tức $m \neq 0$), thì số này biểu diễn **NaN (Not a Number)**.
  - Vậy cả hai khẳng định **2 (biểu diễn NaN)** và **4 (biểu diễn vô cùng)** đều có khả năng đúng.

- **Giải câu b:**
  - Một chuỗi 32-bit là NaN khi trường mũ $e =$ `11111111` (1 cách) và trường định trị m (23 bit) khác 0.
  - Số cách chọn 23 bit của m sao cho không đồng thời bằng 0 là: $2^{23}-1$.
  - Bit dấu S ở đầu có thể bằng 0 hoặc bằng 1 (2 cách chọn).
  - Tổng số cách biểu diễn NaN là: $2 \times (2^{23}-1) = \mathbf{2^{24}-2} \rightarrow$ Chọn **C** $(2^{24}-2)$.